



Degree Project in Machine Learning

Second cycle, 30 credits

Channel Estimation via One-Step Flow-Matching-Based Generative Models

Conditional Flows for Pilot-Contaminated Wideband
MIMO-OFDM Channels

FREDRIK NGUYEN

Channel Estimation via One-Step Flow-Matching-Based Generative Models

Conditional Flows for Pilot-Contaminated Wideband MIMO-OFDM Channels

FREDRIK NGUYEN

Master's Programme, Machine Learning, 120 credits

Date: July 3, 2026

Supervisors: Hossein Azizpour, Diana Wang

Examiner: Hedvig Kjellström

School of Electrical Engineering and Computer Science

Host company: Huawei Technologies Sweden AB

Swedish title: Kanalestimering via enstegsmodeller baserade på flödesmatchning

Swedish subtitle: Villkorade flöden för pilotkontaminerade bredbandiga MIMO-OFDM-kanaler

Abstract

Accurate channel state information is essential for reliable wideband massive MIMO-OFDM systems, but pilot contamination and noise make channel estimation difficult, while practical deployment also favors estimators with low inference cost. This thesis studies whether conditional flow-matching-based one-step estimators can improve channel estimation under pilot contamination in wideband massive MIMO-OFDM systems.

The task is formulated as supervised recovery of the desired channel from projected noisy least-squares pilot observations. The observations are processed in the delay-beam domain, and the learned estimators are compared with a hard-windowing baseline and an ideal upper bound. The study evaluates one-step variants of standard flow matching, Mean Flow, data-parameterized flow, and consistency-style endpoint models, together with ablations over source-state choice, context conditioning, backbone architecture, training signal-to-noise-ratio distribution, and teacher-student distillation. Evaluation combines offline reconstruction diagnostics with end-to-end symbol error rate in a simulator.

The results show that the strongest reported one-step flow-based models yield modest gains over hard-windowing when the model design is aligned more closely with the channel-estimation task. Mean Flow improves over naive one-step standard flow, while the strongest reported one-step result is achieved by combining observation-anchored denoising, consistency-style endpoint learning, focused low-signal-to-noise-ratio training, and teacher-student distillation. This distilled one-step student, compressed from a roughly 4 million parameter teacher to roughly 50 thousand parameters, achieves a gain of 0.66 dB at symbol error rate 0.1 over hard-windowing, corresponding to about 14% less required linear signal-to-noise ratio. The thesis therefore shows that learned one-step channel estimation can improve end-to-end communication performance in the studied simulated pilot-contaminated setting.

Keywords

Channel estimation, Wireless communications, Massive multiple-input multiple-output systems, Orthogonal frequency-division multiplexing, Pilot contamination, Flow matching, Consistency models, Mean Flow models, Distillation, Delay-beam domain, End-to-end communication evaluation, Low-latency inference, One-step inference

Sammanfattning

Noggrann kanalinformation är avgörande för tillförlitlig drift av bredbandiga massiva MIMO-OFDM-system, men pilotkontaminering och brus försvårar kanalestimering samtidigt som praktisk användning ställer krav på låg inferenskostnad. Denna avhandling undersöker om villkorade enstegsestimater baserade på flödesmatchning kan förbättra kanalestimering under pilotkontaminering i bredbandiga massiva MIMO-OFDM-system.

Problemet formuleras som övervakad rekonstruktion av den önskade kanalen från projicerade, brusiga least-squares-observationer av pilotsignalerna. Observationerna bearbetas i delay-beam-domänen, och de lärda estimatorerna jämförs med en hard-windowing-baslinje och en ideal övre gräns. Studien utvärderar enstegsvarianter av standard flow matching, Mean Flow, data-parametriserat flöde och konsistensbaserade endpointmodeller, tillsammans med ablationer av val av starttillstånd, kontextvillkorning, modellarkitektur, träningsfördelning över signal-brus-förhållande och lärar-student-destillation. Utvärderingen kombinerar offline-diagnostik för rekonstruktion med end-to-end-symbolfelsfrekvens i en simulator.

Resultaten visar att de starkaste rapporterade flödesbaserade enstegsmodellerna ger måttliga förbättringar jämfört med hard-windowing när modellutformningen anpassas närmare kanalestimeringsuppgiften. Mean Flow förbättrar resultatet jämfört med en naiv enstegsvariant av standard flow, medan det starkaste rapporterade enstegsresultatet uppnås genom att kombinera observationsförankrad denoising, konsistensbaserad endpointinlärning, fokuserad träning vid låga signal-brus-förhållanden och lärar-student-destillation. Denna destillerade enstegsstudent, komprimerad från en lärarmodell med ungefär 4 miljoner parametrar till ungefär 50 tusen parametrar, uppnår en vinst på 0.66 dB vid symbolfelsfrekvens 0.1 jämfört med hard-windowing, vilket motsvarar ungefär 14% lägre krav på linjärt signal-brus-förhållande. Avhandlingen visar därmed att lärd enstegsbaserad kanalestimering kan förbättra kommunikationsprestandan i hela kedjan i den studerade simulerade pilotkontaminerade miljön.

Nyckelord

Kanalestimering, Trådlös kommunikation, Massiva system med flera in- och utgångar, Ortogonal frekvensdelningsmultiplexering, Pilotkontaminering, Flödesmatchning, Konsistensmodeller, Mean Flow-modeller, Destillation, Delay-beam-domän, End-to-end-utvärdering av kommunikationsprestanda,

Inferens med låg latens, Enstegs-inferens

Acknowledgments

I would like to thank my supervisor, Diana Wang, who hosted me at Huawei Technologies Sweden AB and provided invaluable guidance and support throughout this thesis project. I would also like to thank my KTH supervisor Hossein Azizpour for providing input for the project. I am also grateful to my examiner, Hedvig Kjellström, for her continuous feedback and encouragement to reach the finish line.

Special thanks to my opponent Nikita Suprun for their valuable feedback, to my fellow interns at Huawei Technologies Sweden AB, who encouraged me along the way and made this experience enjoyable, and to my family and friends for their support.

Disclosure: AI tools were used to edit, improve and assist with the text and language of the thesis. It was also used to create tables, images and pseudocode. The use of AI was limited to these tasks and did not contribute to the intellectual content or original research presented in this thesis.

Stockholm, July 2026

Fredrik Nguyen

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	2
1.2.1	Research question	3
1.2.2	Original problem and definition	3
1.2.3	Scientific and engineering issues	3
1.3	Purpose	3
1.4	Goals	4
1.5	Research Methodology	4
1.6	Delimitations	5
1.7	Structure of the thesis	5
2	Background	7
2.1	Wireless Communication	7
2.1.1	Massive MIMO systems	7
2.1.2	OFDM	8
2.2	Channel estimation	10
2.2.1	System modeling	10
2.2.1.1	Wideband MIMO-OFDM model with additive white Gaussian noise (AWGN)	10
2.2.1.2	Wideband MIMO-OFDM model with pilot contamination (PC)	11
2.2.1.3	Model simplification	12
2.2.2	Classical channel estimation methods	13
2.2.2.1	LS and LMMSE estimator	13
2.2.2.2	Windowing estimator	14
2.2.3	Motivation for Generative Neural Networks	15
2.3	Flow Matching	16
2.3.1	Conditional vs. Unconditional Generative Flows	17

2.3.2	Rectified Flow as a Special Case of Flow Matching . . .	18
2.3.3	Flow-Model Families	19
2.3.3.1	Mean Flow	19
2.3.3.2	Data-Parameterized Flow	22
2.3.3.3	Consistency Objectives	22
2.3.4	Distillation	24
2.3.4.1	Consistency Mid-Training	24
2.4	Related Work	25
2.4.1	Channel Estimation Using Generative Models	25
2.4.1.1	Other generative models	25
2.4.1.2	Flow matching for channel estimation	27
2.4.2	Flow Matching Models	28
2.4.2.1	General flow matching models for inverse problems	28
2.4.2.2	One-step and few-step methods	29
2.5	Summary	29
3	Methods	31
3.1	Research Process and Paradigm	31
3.2	Data Collection	32
3.2.1	Dataset Generation and Channel Modeling	32
3.2.2	Data Preprocessing and Interference Modeling	36
3.2.3	Domain Transformations	38
3.3	Experimental design and Measurements	44
3.3.1	Test environment	44
3.3.2	Model design	45
3.3.3	Design Choice: Unconditional vs. Conditional Flow	49
3.4	Assessing reliability and validity of the data collected	50
3.4.1	Validity of method	50
3.4.2	Reliability of method	50
3.4.3	Data validity	51
3.4.4	Reliability of data	51
3.4.5	Data Analysis	51
3.5	Evaluation Framework	52
3.5.1	Evaluation metrics	52
3.6	System documentation	53

4	Implementation	55
4.1	Experiment implementation	55
4.1.1	Implemented preprocessing	56
4.1.2	A. Flow type	57
4.1.3	B. Context inputs	57
4.1.4	C. Flow-model family	58
4.1.5	D. Backbone comparison	59
4.1.6	E. CMT implementation	60
4.1.7	F. Training-distribution ablations	61
4.1.8	G. Final model implementation	62
4.2	Backbone implementations	63
4.2.1	U-Net family	63
4.2.2	SiT transformer family	65
4.3	Training pipeline and experiment management	66
4.3.1	Software and hardware environment	66
4.3.2	Configuration presets	66
4.3.3	EMA checkpoint selection	68
4.4	Baseline implementations	68
5	Results and Analysis	71
5.1	Major results	71
5.2	Reliability Analysis	78
5.3	Validity Analysis	78
6	Discussion	81
6.1	Interpretation of the Main Findings	81
6.2	Training-Distribution Specialization	82
6.3	SER Versus Reconstruction Metrics	83
6.4	Complexity and Deployment Trade-offs	84
7	Conclusions and Future Work	87
7.1	Conclusions	87
7.2	Limitations	88
7.3	Future Work	89
7.4	Reflections	90
	References	93

A	Auxiliary Results	99
A.1	Supplementary Diagnostic Metric Definitions	99
A.1.1	Companion Diagnostic Tables for the Main Results . .	100
A.2	Exploratory Pilot and Huber Auxiliary-Loss Sweeps	103
A.3	Auxiliary CMT Comparisons	105
A.4	Supplementary Runtime Measurements	106
A.5	Auxiliary High-SNR Simulator Diagnostics	107
A.6	No-Interference Sanity Check	108
A.7	Supplementary Backbone Diagram	108
A.8	Additional Data and Domain Diagnostics	110
B	Supporting Materials	112
B.1	Implementation pseudocode	112
B.2	Least-Squares (LS) Projection	117
B.3	LMMSE Estimator	119

List of Figures

2.1	Multi-cell massive MIMO. Each Base Station (BS) is equipped with a large antenna array and serves multiple user equipment (UEs) in its cell. Because pilot orthogonality is not guaranteed across neighboring cells, the received pilots are corrupted by thermal noise and coherent inter-cell interference from pilot contamination, which degrades the resulting Channel State Information (CSI) estimates.	8
2.2	The OFDM resource grid, where the vertical axis represents subcarriers and the horizontal axis represents time. Each square denotes a resource element that may carry uplink data, downlink data, or pilot symbols. A block of such resource elements corresponds to a coherence block, within which the channel can be assumed to be approximately constant.	10
2.3	Conceptual illustration of hard-windowing in the delay-beam domain. The left panel shows a representative delay profile, where most clean-channel energy is concentrated in a small number of delay bins close to the edges, while the noisy estimate contains a broader disturbance floor. The highlighted edge regions indicate the delay support retained by a simple hard window. The rest of the delay bins are suppressed (set to zero) to reduce noise and interference.	15
2.4	Conceptual comparison between deterministic flow and stochastic diffusion trajectories. The smooth red trajectory illustrates a continuous flow path from the source point x_0 toward the target point x_1 , while the jagged blue trajectory illustrates a diffusion-style path with noisy intermediate states. The figure is intended as intuition for the difference in trajectory geometry rather than as a formal depiction of a specific sampler.	16

2.5	Geometric intuition for two fast-model ideas used in this thesis. Left: Mean Flow summarizes a finite interval with one averaged displacement. Right: consistency training maps different points on the same trajectory to the same endpoint. .	24
3.1	Comparison of train and test spread statistics in the delay and beam domains over 256 representative samples from each set. Here, spread denotes the RMS spread of the normalized channel magnitude over delay-bin index or beam-index, respectively, and the y-axis shows histogram density rather than counts. The Cluster Delay Line B (CDL-B) evaluation set is more tightly concentrated in delay spread and slightly shifted in beam spread relative to the QuaDRiGa training set, illustrating an additional train/test distribution shift beyond the sampled Signal-to-Noise Ratio (SNR)/Interference-to-Noise Ratio (INR) ranges.	34
3.2	Conceptual comparison of Quadrature Phase Shift Keying (QPSK) and 16-QAM constellations. In nearest-neighbor detection, a noisy or predicted complex symbol is assigned to the valid constellation point closest to it in the complex plane. Because QPSK has fewer constellation points than 16-QAM, its decision regions are larger, so the same perturbation is less likely to cross a decision boundary and cause a symbol error. This illustrates why low-order modulation is generally more robust to noise and interference in the noisy regime considered in this thesis.	35
3.3	Representative 1D magnitude slices of the clean and noisy channels at SNR 0 dB in the frequency-antenna domain. . . .	39
3.4	3D magnitude of the clean and noisy channels at SNR 0 dB under the default INR configuration in the frequency-antenna domain.	39
3.5	Normalized cumulative retained-energy curves computed over 256 representative channel samples in the frequency-antenna and delay-beam domains, plotted against the percentage of coefficients retained after sorting coefficients by magnitude. The y-axis therefore shows the fraction of total sample energy captured by the retained coefficients rather than physical energy units. The delay-beam representation concentrates most of the energy into a much smaller fraction of coefficients.	40

3.6	Representative 1D magnitude slices of the clean and noisy channels at SNR 0 dB in the delay-beam domain.	42
3.7	Representative 1D magnitude slices of the clean and noisy channels at SNR -20 dB in the delay-beam domain.	42
3.8	Representative 1D magnitude slices of the clean and noisy channels at SNR 30 dB in the delay-beam domain.	43
3.9	3D magnitude of the clean and noisy channels at SNR 0 dB under the default INR configuration in the delay-beam domain.	43
4.1	Implemented Consistency Mid-Training (CMT) pipeline. A standard-flow teacher checkpoint is rolled out with 10 Euler steps to produce either a grid or interpolated trajectory representation. The reported one-step CMT runs use this trajectory to form beginning-to-end teacher supervision from the noisy start to the teacher endpoint. The dashed connection indicates an optional student checkpoint or warm-start used to initialize the CMT stage before post-CMT retraining.	61
4.2	Schematic U-Net-style backbone. The architecture is organized as encoder, bottleneck, and decoder stages over the delay-beam grid. Dashed arrows denote skip concatenations, and the time/context embedding is shown schematically rather than repeated for every residual stage. UNet-mini uses $c = 8$, one residual block per level, channel multipliers $[1, 1, 2]$, and only the middle attention block. UNet-Lite uses the same layout with $c = 16$. The standard U-Net uses $c = 32$, two residual blocks per level, multipliers $[1, 2, 4]$, and attention at resolution 32 in addition to the middle block.	65
5.1	End-to-end Symbol Error Rate (SER) versus SNR for hard-windowing, the ideal upper-bound estimator, and the strongest reported one-step learned estimator from Table 5.7, namely the CMT-distilled UNet-mini. In the legend, CE denotes the channel-estimation module, R_{uu} denotes the covariance estimate used in the detector, and MD=lmmse indicates that the final symbol-estimation stage uses an Linear Minimum Mean-Squared Error (LMMSE) detector.	76

5.2	Accuracy-cost trade-off for selected final neural models from Table 5.7. The x-axis uses the reported backbone GMAC from Table 5.8; for the 10-step teacher, the plotted cost is approximated as ten times the one-step U-Net cost because the same backbone is evaluated over ten sampling steps. Hard-windowing is omitted because no directly comparable profiler-based GMAC estimate is available for that non-neural baseline.	77
A.1	Supplementary SiT-4M backbone schematic. A patchify convolution converts the delay-beam grid into 512 spatial tokens, four time/context prefix tokens are concatenated before the transformer stack, and the prefix tokens are discarded before patch projection and unpatchifying back to the dense output grid. The transformer blocks use RMSNorm, RoPE attention, SwiGLU feed-forward layers, and zero-initialized residual gates.	109
A.2	Supplementary four-panel magnitude summary of the same representative sample in the frequency-antenna and delay-beam domains, for both clean and noisy observations. This figure complements the main-text visualizations by gathering the qualitative comparison into a single overview.	110
A.3	Supplementary delay-beam phase visualizations for the same representative sample used in the main text. These plots are included for completeness. They show that there is an inherent phase structure in the delay-beam channel representation, and that noise affects this structure in a nontrivial way.	111

List of Tables

2.1	Qualitative comparison of channel-estimation paradigms discussed in this chapter.	30
2.2	Flow-matching extensions, fast-model families, and training enhancements discussed in this chapter.	30
4.1	Compact shorthand naming for the reported experiment groups and auxiliary CMT mode variants documented in the implementation chapter.	56
4.2	Explicit C-series parameter settings not already specified by the model definitions.	59
4.3	Documented neural backbone variants used in the reported experiments.	63
4.4	Remaining shared base training settings not already specified above.	67
5.1	Classical and reference simulator baselines. Per-SNR SER values are reported across the evaluation grid, together with $\text{Gain}_{0.1}$ (the SNR advantage over hard-windowing at $\text{SER} = 0.1$, where positive means less SNR is required) and Normalized Mean Squared Error (NMSE).	72
5.2	Generative versus denoising task formulation for one-step standard FM with the UNet-Lite backbone. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. . .	72
5.3	Context-embedding ablation. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. $\text{Gain}_{0.1}$ is measured against hard-windowing using the simulator interpolation convention.	73
5.4	One-step and few-step flow model comparison. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. . .	73

5.5	Backbone comparison. Per-SNR SER, $\text{Gain}_{0.1}$, NMSE, and parameter count are reported for the standard-flow backbone variants.	74
5.6	Focused low-SNR training distribution ablation for one-step standard FM with the UNet-Lite backbone. The standard row is included as the wide-range reference. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. . .	74
5.7	Selected final integrated joint-Demodulation Reference Signal (DMRS) runs. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. The no-flow/standard rows and the consistency/CMT rows are separated for readability, and the best values within each section are shown in bold. Detailed training schedules are documented in Chapter 4.	75
5.8	Model complexity for the joint-DMRS backbones used in the main comparisons, measured with ptflops and corrected for the scaled dot-product attention operations omitted by the profiler.	77
A.1	Supplementary diagnostics for the baseline table in Chapter 5.	101
A.2	Supplementary diagnostics for the task-formulation table in Chapter 5.	101
A.3	Supplementary diagnostics for the context-embedding table in Chapter 5.	101
A.4	Supplementary diagnostics for the flow-model table in Chapter 5.	101
A.5	Supplementary diagnostics for the backbone table in Chapter 5.	102
A.6	Supplementary diagnostics for the focused low-SNR training-distribution table in Chapter 5.	102
A.7	Final integrated supplementary diagnostics	102
A.8	Exploratory pilot-guided auxiliary-loss sweep for one-step standard flow matching. SER is averaged over -14 to -6 dB. All rows are independent one-epoch sensitivity checks.	105
A.9	Exploratory pseudo-Huber auxiliary-loss sweep for one-step standard flow matching. SER is averaged over -14 to -6 dB. All rows are independent one-epoch sensitivity checks.	105

A.10 Auxiliary CMT mode comparison for a consistency student. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. These rows are used mainly to compare grid and interpolated trajectory representations before the final integrated CMT study.	106
A.11 Representative joint-DMRS simulator-path latency. $\text{Gain}_{0.1}$ is reported relative to hard-windowing, and latency is the overall per-sample channel-estimation latency reported by the simulator for the full estimator path.	106
A.12 Auxiliary high-SNR simulator diagnostics on the separate [5,15,25] dB grid. Since all listed rows achieve zero SER throughout this grid, the table reports only reconstruction quality through NMSE.	108
A.13 Auxiliary side-by-side low-SNR gain summary for the with-interference and no-interference settings. $\text{Gain}_{0.1}$ is measured relative to hard-windowing at the target SER of 0.1.	108

List of Algorithms

1	Physics-collator preprocessing	113
2	Standard flow-matching training step	113
3	Mean Flow and AlphaMeanFlow training step	114
4	Data-parameterized flow training step	114
5	Consistency-model training step	114
6	CMT teacher-student distillation	115
7	Learned-estimator inference in the simulator adapter	116

List of acronyms and abbreviations

This document is incomplete. The external file associated with the glossary ‘acronym’ (which should be called `thesis_final_fredrik_nguyen.acr`) hasn’t been created.

Check the contents of the file `thesis_final_fredrik_nguyen.acn`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "thesis_final_fredrik_nguyen"
```

- Run the external (Perl) application:

```
makeglossaries "thesis_final_fredrik_nguyen"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

Chapter 1

Introduction

This chapter introduces the context and scope of the thesis. It first motivates the channel-estimation problem in wideband massive **Multiple-Input Multiple-Output (MIMO)-Orthogonal Frequency-Division Multiplexing (OFDM)** systems and the practical difficulties caused by noise, pilot contamination, and implementation constraints. It then formulates the specific problem studied in this work, states the purpose and goals of the thesis, outlines the research methodology, and clarifies the main delimitations.

1.1 Background

Modern wireless communication systems are evolving rapidly to support increasing data rates, massive connectivity, and stringent reliability requirements. Technologies such as massive **MIMO** and **OFDM** play a central role in enabling these capabilities [1, 2]. A fundamental prerequisite for reliable communication in such systems is accurate **Channel State Information (CSI)** [3, 4]. In this context, the wireless channel describes how a transmitted signal is altered as it propagates from transmitter to the receiving side, including attenuation, phase shifts, and multipath propagation [5]. Accurate **CSI** is required for downstream tasks such as signal detection, beamforming, and resource allocation [3, 4]. In practice, **CSI** is acquired from pilot observations, but these observations are degraded by thermal noise and by interference from neighboring cells, which leads to pilot contamination in multi-cell networks [5, 6].

Recent generative modeling approaches have shown promise for channel estimation, but many high-performing methods rely on iterative inference and therefore suffer from high computational latency. This includes score-based

and diffusion-style channel estimators, which depend on sequential posterior sampling or repeated denoising steps to recover the channel [7, 8].

Channel estimation in practical wireless systems is, in addition to accuracy constraints, subject to strict latency requirements, since CSI must be acquired and used while the channel remains sufficiently stable [3, 5]. Consequently, channel estimation methods must not only mitigate noise and pilot contamination, but also operate with sufficient computational efficiency to be deployable in real-time wireless systems [4]. Together, these requirements create a setting in which neither simple low-cost estimators nor slower iterative generative samplers are fully satisfactory.

1.2 Problem

Channel estimation can be viewed as a high-dimensional inverse problem: the estimator receives noisy and potentially contaminated pilot measurements and must recover the underlying channel. Traditional estimators used for this task, such as Least Squares (LS) and Linear Minimum Mean-Squared Error (LMMSE)-based approaches, have important practical limitations [5, 6]. While LS is simple but often inaccurate, LMMSE can improve estimation quality at the cost of high computational complexity for large channel dimensions.

Recent advances in machine learning, particularly in generative modeling, have introduced a data-driven alternative. Generative models can learn complex channel distributions and exploit this learned structure for denoising and channel reconstruction, thereby improving robustness under challenging channel conditions [7, 9]. The central unresolved problem is therefore how to achieve robust channel estimation under wideband pilot contamination with learned one-step estimators that remain practical to deploy under tight latency and inference-cost constraints.

To address this problem, the thesis investigates conditional Flow Matching (FM)-based estimators as a candidate approach. Flow matching is chosen because its deterministic transport formulation and compatibility with one-step or very-low-step inference make it a plausible route to simple inference while still retaining expressive generative modeling capacity. The one-step focus is motivated by the latency-sensitive deployment setting outlined above: if accurate channel estimation requires many iterative sampling steps, the resulting estimator becomes harder to use in practice.

1.2.1 Research question

This thesis is guided by the following research question: can conditional **FM**-based one-step estimators provide accurate channel estimation under noise and pilot contamination, and which design choices make them competitive with practical baselines?

1.2.2 Original problem and definition

The original industrial problem proposed by Huawei Technologies Sweden AB was broader than the final thesis framing. In the initial proposal, the main question was whether flow matching could serve as a useful generative approach for channel estimation in wideband **MIMO-OFDM** systems under noise and pilot contamination. At that stage, the problem was not yet formulated in terms of one-step inference or explicit runtime-oriented evaluation. The thesis problem studied in this report emerged by refining that broader industrial question into a more specific and practically constrained form.

1.2.3 Scientific and engineering issues

Refining the original problem in this way brings the scientific and engineering issues into sharper focus through the tension between estimation quality and direct one-step inference. Since **CSI** must be acquired and used within strict coherence-time constraints, there is a practical incentive to favor methods that avoid heavy iterative sampling [3, 5]. The scientific and engineering challenge is therefore to determine whether a generative model can retain enough modeling capacity to handle noise and pilot contamination while still working well in a single learned update.

1.3 Purpose

The purpose of this thesis is to evaluate the hypothesis that conditional **FM**-based one-step estimators can improve channel estimation and end-to-end communication performance under noise and pilot contamination in a practically relevant one-step setting motivated by latency-sensitive deployment constraints.

A practically useful method for channel estimation is relevant from an industrial perspective because channel estimation must be accurate enough to

be useful and simple enough to fit within practical deployment constraints. In deployed wireless systems, a channel estimator is only valuable if it can be integrated into the physical-layer signal-processing pipeline without introducing excessive computational burden, energy cost, or implementation overhead. This is particularly relevant because accurate and efficient channel estimation is an integral part of the physical-layer signal-processing pipeline in next-generation 5G and prospective 6G systems, which are active development directions in the wireless industry, including at Huawei [10, 11]. It is also relevant from an academic perspective because it studies how a recent generative modeling framework can be adapted to a structured inverse problem in wireless communications. In addition, lower computational burden may be beneficial from a sustainability perspective through reduced energy use in communication infrastructure.

1.4 Goals

To address this purpose, the thesis is structured around the following goals:

1. Implement conditional **FM**-based channel estimation models and evaluate whether they can improve reconstruction quality and end-to-end **Symbol Error Rate (SER)** relative to baseline methods.
2. Investigate how modeling choices, including context-input design, flow objective, and backbone architecture, affect reconstruction accuracy, robustness, and overall model practicality.
3. Perform ablation studies to assess estimation quality under different noise and pilot-contamination conditions.

The intended outputs of the thesis are the implemented models, a well-documented experimental framework, and an empirical comparison against practical and classical reference baselines.

1.5 Research Methodology

The thesis follows a quantitative and experimental methodology based on simulation-driven evaluation and iterative model development. Conditional **FM** is adopted as the main model family under study, and different context-input variants, flow objectives, model variants, and backbone configurations are implemented and compared empirically. The design choices are guided by

prior literature, the one-step design target, available computational resources, and observed performance in controlled experiments. Models are trained on simulated channel data and evaluated in terms of reconstruction accuracy, robustness across **Signal-to-Noise Ratio (SNR)** and interference conditions, and downstream communication performance.

1.6 Delimitations

This thesis is limited to **FM** and **FM**-based methods for channel estimation. The experimental work focuses on one-step estimators, since the central question is whether a single learned update can already outperform strong practical baselines such as hard-windowing while remaining compatible with latency-sensitive deployment. Multi-step generative inference is therefore treated as supplementary context rather than as the main design axis.

Other machine learning and generative model families are discussed in relation to prior work but are not implemented or experimentally evaluated. The study is also restricted to simulated channel data. No real measured channel data is used for training or evaluation. In addition, the experimental workflow does not include exhaustive hyperparameter optimization due to limited available GPU resources and overall compute budget. Instead, a limited number of small sweeps and pilot experiments are used to choose reasonable default settings before sequential ablations are carried out. Where explicit search ranges were needed, they were chosen as informed, narrow guesses based on the original papers and then checked through a small number of practical trial runs rather than through a broad automated optimization study. This is an important limitation because flow-map-style models such as Mean Flow and consistency-based models, as well as teacher-student procedures such as **Consistency Mid-Training (CMT)**, are known to be unstable and sensitive to hyperparameters during training [12, 13, 14]. These delimitations mean that the thesis evaluates methodological feasibility under controlled simulation conditions rather than claiming immediate deployment readiness on measured real-world channels.

1.7 Structure of the thesis

The remainder of this thesis is organized as follows. Chapter 2 introduces the wireless communication setting, the wideband **MIMO-OFDM** channel-estimation problem under pilot contamination, and the flow-matching

literature relevant to this work. Chapter 3 describes the research process, the simulation-based data pipeline, the experimental design, and the evaluation framework. Chapter 4 documents the implemented model families, the main model and experiment configurations, the training setup, backbone choices, and the classical reference baselines used in the experiments. Chapter 5 presents the empirical results, including reconstruction metrics, simulator-based communication performance, and ablation studies. Supplementary runtime measurements and related implementation notes are collected in Appendix A.4. Chapter 6 interprets the main findings in relation to the thesis objectives and limitations, while Chapter 7 concludes the thesis and outlines directions for future work. The appendices provide supplementary visualizations, supporting material, and implementation details.

Chapter 2

Background

This chapter introduces the physical-layer setting and channel-estimation models used throughout the thesis. It first reviews the wireless-communication background needed for wideband massive **MIMO-OFDM** channel estimation, then presents the observation models under noise and pilot contamination together with the main classical baselines. The chapter next introduces flow matching and its one-step extensions, and concludes with related work on generative channel estimation and flow-based inverse-problem methods.

2.1 Wireless Communication

2.1.1 Massive MIMO systems

Massive **MIMO** is a multi-user technology in which each **Base Station (BS)** is equipped with a large number N_r of active antenna elements and simultaneously serves N_{UE} single-antenna or multi-antenna **user equipment (UE)** terminals, each with N_t antennas, over the same time-frequency resources [1, 5]. Here, a **BS** is the network-side radio node, whereas a **UE** is the communicating terminal or device served by it. By processing signals jointly across the antenna array, the **BS** can steer downlink transmissions toward the intended users, which is referred to as precoding. In the uplink, it can combine the received antenna signals to better separate transmissions from different **UEs**, which is referred to as receive combining.

Massive MIMO systems are often operated in **Time-Division Duplex (TDD)** mode, where uplink and downlink transmissions share the same frequency band but are separated in time. This setup makes it possible to exploit channel reciprocity, meaning that the same physical propagation

channel is observed in both directions. Consequently, the **BS** can estimate the channel in the uplink and reuse the acquired **CSI** for both uplink detection and downlink precoding, although some hardware calibration is required in practice. Accordingly, this thesis focuses on the uplink perspective of the wireless communication pipeline [1].

A key advantage of **TDD**-based Massive **MIMO** is scalability. The channel estimation overhead scales with the number of terminals N_{UE} , but is independent of the number of **BS** antennas N_r . Moreover, increasing N_r does not increase this overhead, while larger antenna arrays can provide better angular resolution and more pronounced spatial structure that can be exploited in channel estimation [1, 5]. These properties make Massive **MIMO** particularly suitable for high-spectral-efficiency, i.e. the amount of information transmitted per unit bandwidth, and scalable multi-user communication [1]. Figure 2.1 illustrates the multi-cell setting considered in this thesis and how the lack of guaranteed pilot orthogonality across neighboring cells gives rise to pilot contamination.

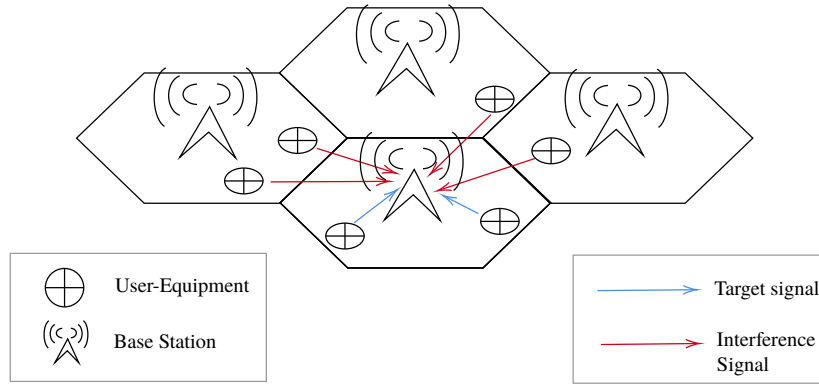


Figure 2.1: Multi-cell massive MIMO. Each **BS** is equipped with a large antenna array and serves multiple **UEs** in its cell. Because pilot orthogonality is not guaranteed across neighboring cells, the received pilots are corrupted by thermal noise and coherent inter-cell interference from pilot contamination, which degrades the resulting **CSI** estimates.

2.1.2 OFDM

In wireless communication systems, transmitted signals propagate along multiple paths due to reflections and scattering in the environment. These paths arrive with different delays and attenuations, resulting in multipath fading and frequency-selective channels [15, 5].

OFDM addresses these challenges by decomposing a wideband frequency-selective channel into multiple orthogonal subcarriers, each of which experiences approximately flat fading. The data stream is divided into parallel low-rate symbol streams that are independently modulated on each subcarrier using digital modulation schemes such as phase-shift keying (PSK) or **Quadrature Amplitude Modulation (QAM)**. This makes **OFDM** well suited for representing and estimating wideband channels on a structured time-frequency grid [15, 5].

When combined with **MIMO**, which enables multiple data streams to be transmitted over multiple antennas on the same time-frequency resources, **MIMO-OFDM** improves robustness to multipath fading while increasing spectral efficiency. This combination underlies modern wireless systems such as 5G and prospective 6G systems and forms the foundation of wideband **CSI** estimation in contemporary wireless communications [1, 2].

Because fading causes the wireless channel to vary over both time and frequency, channel estimation and data transmission must be confined to a time-frequency region over which the channel can be approximated as quasi-static. This ensures that the acquired **CSI** remains sufficiently accurate to support reliable and spectrally efficient communication. The dimensions of this region are determined by the coherence bandwidth B_c in frequency and the coherence time T_c in time. Together, these quantities define a coherence block, *i.e.*, a two-dimensional time-frequency region over which the channel can be assumed approximately constant [1, 5].

In an **OFDM** system, the coherence bandwidth corresponds to a set of adjacent subcarriers over which the channel frequency response varies negligibly, while the coherence time corresponds to a number of consecutive **OFDM** symbols over which the channel remains approximately constant in time. The channel is therefore represented on a two-dimensional time-frequency grid defined by subcarriers and **OFDM** symbols. The **Transmission-Time Interval (TTI)** corresponds to a scheduling block comprising several consecutive **OFDM** symbols. In the setting considered here, the channel is assumed quasi-static within one **TTI**, and each **TTI** is treated as one channel realization. Figure 2.2 illustrates this resource grid and shows how the coherence bandwidth B_c and coherence time T_c define a coherence block across frequency and time [1, 5].

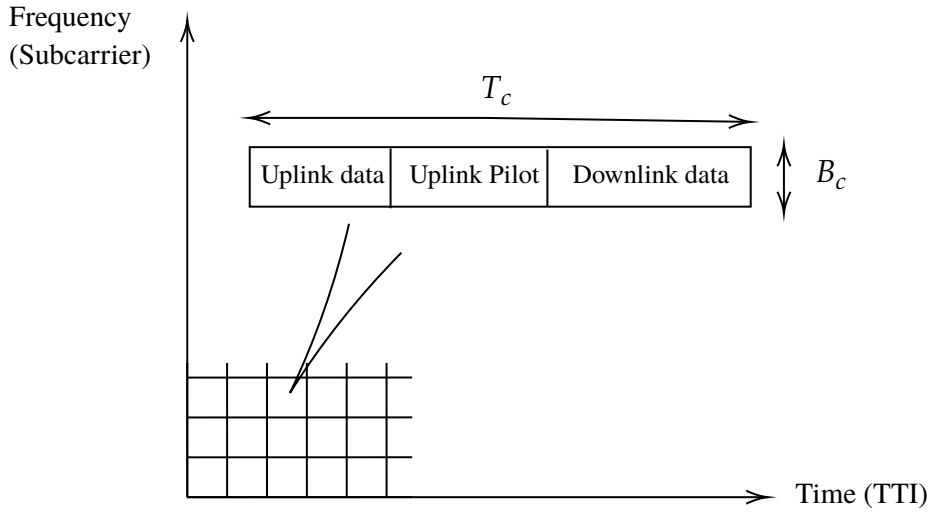


Figure 2.2: The OFDM resource grid, where the vertical axis represents subcarriers and the horizontal axis represents time. Each square denotes a resource element that may carry uplink data, downlink data, or pilot symbols. A block of such resource elements corresponds to a coherence block, within which the channel can be assumed to be approximately constant.

2.2 Channel estimation

2.2.1 System modeling

We consider channel estimation in a wideband massive **MIMO-OFDM** system. The task is formulated as an inverse problem in which the objective is to recover the underlying **CSI** from noisy and potentially contaminated pilot observations. The system model is presented first for a clean setting without pilot contamination and then for a more realistic setting with pilot contamination [5, 6].

2.2.1.1 Wideband MIMO-OFDM model with additive white Gaussian noise (AWGN)

During each **TTI**, a subset of the time-frequency resources across subcarriers is reserved for transmitting known pilot sequences. These pilot signals enable the **BS** to compare transmitted and received signals and thereby estimate the channel response [5].

The coherence time and coherence bandwidth constrain the size of the coherence block, which in turn limits the available pilot length. This limits

the number of mutually orthogonal pilot sequences that can be accommodated within one coherence interval. As a result, pilot orthogonality cannot in general be maintained across base stations, which leads to pilot contamination in multi-cell networks [6]. Throughout this thesis, we assume that:

- Within each cell, pilot sequences are orthogonal and do not interfere with each other.
- Across cells, pilot sequences are not in general orthogonal, and pilot contamination is present.

The detailed **LS** projection from the per-subcarrier pilot matrix model is given in Appendix **B.2**. In the main text, the important outcome is the projected observation model used by the estimators. After applying the **LS** projection independently on each subcarrier and stacking the result over K subcarriers, the clean wideband observation can be written as

$$\hat{H}_{\text{LS}} = H + \tilde{N}, \quad (2.1)$$

where H is the underlying wideband channel, $\hat{H}_{\text{LS}}$ is the projected noisy channel estimate, and $\tilde{N}$ is the effective projected noise [5]. In the general matrix notation these quantities are defined over subcarriers, receive antennas, and transmit antennas. In the simplified setup considered in this thesis, the target subproblem is an effective per-user model with $N_t = 1$ and one retained pilot symbol, as described in Section **2.2.1.3**.

2.2.1.2 Wideband MIMO-OFDM model with pilot contamination (PC)

In multi-cell scenarios, pilots from neighboring cells introduce structured interference [6]. Without loss of generality, let the desired user/channel be indexed by 0. Then the projected observation on subcarrier k can be written as

$$\hat{H}_{\text{LS},0,k} = H_{0,k} + \sum_{i \in \mathcal{I}} H_{i,k} P_{i,k} P_{0,k}^H (P_{0,k} P_{0,k}^H)^{-1} + \tilde{N}_k, \quad k = 1, \dots, K, \quad (2.2)$$

where $\mathcal{I}$ is a set of interfering users, $H_{0,k}$ and $P_{0,k}$ denote the desired channel and pilot, $H_{i,k}$ and $P_{i,k}$ denote the corresponding interfering channels and pilots, and $\hat{H}_{\text{LS},0,k}$ denotes the projected estimate of the desired channel. The residual pilot factors remain because $P_{i,k}$ and $P_{0,k}$ are generally non-orthogonal. In the effective per-user approximation used below, where $N_t =$

1, $\tau_p = 1$, and under the unit-power pilot convention $P_{0,k}P_{0,k}^H = 1$, the pilot factor reduces to the scalar cross-correlation

$$\rho_{i0,k} := P_{i,k}P_{0,k}^H,$$

and the projected per-subcarrier observation becomes

$$\hat{h}_{\text{LS},0,k} = h_{0,k} + \sum_{i \in \mathcal{I}} \rho_{i0,k} h_{i,k} + \tilde{n}_k.$$

This residual term captures the structured interference caused by pilot contamination. Standard data signals are not coherent with the desired pilot sequence and are therefore treated as part of the unstructured noise term after projection rather than as coherent pilot contamination.

2.2.1.3 Model simplification

The considered scenario is a multi-cell massive **MIMO** setup. In the adopted model, multi-antenna **UEs** are treated as cross-polarized antenna pairs with negligible cross-correlation between antenna ports. Under this approximation, each **UE** antenna can be modeled as an independent transmitter. We further assume that there is no intra-cell interference.

These assumptions do not change the overall multi-cell massive **MIMO** setting. Rather, they yield an effective per-user **Single-Input Multiple-Output (SIMO)** approximation from the base-station perspective, where the transmit-antenna dimension is absorbed into the user index and the subproblem considered from this point onward has $N_t = 1$. Unless stated otherwise, in the setup used in this thesis, only one of the pilot-carrying **OFDM** symbols is retained for each sample, so the effective pilot length is $\tau_p = 1$. The projected **LS** observation and the corresponding target channel are then rescaled relative to the disturbance level. As a result, additional uncorrelated data-induced noise is absorbed into the effective noise term rather than tracked explicitly in this background model.

The final channel-estimation model is therefore

$$\hat{H}_{\text{LS},0} = H_0 + \sum_{i \in \mathcal{I}_{\text{eff}}} H_i \odot \rho_{i0} + \tilde{N}, \quad (2.3)$$

where $\mathcal{I}_{\text{eff}}$ denotes the effective set of interferers overlapping with the selected pilot pattern, $H_0, H_i, \hat{H}_{\text{LS},0}, \tilde{N} \in \mathbb{C}^{K \times N_r}$, $\rho_{i0} \in \mathbb{C}^K$ collects the per-subcarrier pilot cross-correlation factors and $\odot$ denotes subcarrier-wise

multiplication broadcast across receive antennas. Throughout the thesis, channel estimation under pilot contamination is therefore treated as the task of recovering H_0 from the projected observation $\hat{H}_{LS,0}$. These simplifications and the resulting model are discussed further in Section 3.2 in relation to the data-collection process.

2.2.2 Classical channel estimation methods

Classical channel estimation in OFDM systems has been studied extensively [5]. In general, these methods can be categorized into four groups: traditional channel estimation based on the Channel Frequency Response (CFR), parametric model-based channel estimation, iterative channel estimation, and channel estimation specifically designed for MIMO systems.

Among the classical estimation methods, this thesis reviews traditional CFR-based estimators, particularly LS, LMMSE, and windowing techniques, as reference methods for channel estimation [5]. In the reported simulator comparisons, the practical channel-estimation baseline is hard-windowing, while ideal channel estimation is used as an upper-bound reference. The LS estimate is the projected noisy observation that feeds the hard-windowing and learned estimators, and LMMSE is discussed as an unweighted textbook statistical reference rather than used as a reported channel-estimation baseline. Unless stated otherwise, the textbook derivations in this subsection use the uncontaminated wideband symbol $\hat{H}_{LS}$ for readability. In the pilot-contaminated thesis setting, the corresponding projected observation is $\hat{H}_{LS,0}$.

2.2.2.1 LS and LMMSE estimator

The LS projection of the channel has been introduced above. The full per-subcarrier derivation is provided in Appendix B.2.

Under additive white Gaussian noise, the LS estimator is equivalent to the maximum-likelihood estimator. Since neither LS nor maximum likelihood requires statistical knowledge of the CFR, which is treated as deterministic but unknown, their estimation accuracy is often limited [5].

In contrast, the LMMSE estimator treats the channel as a random variable with known second-order statistics and uses the covariance between the desired channel and the observation to construct the optimal linear minimum-mean-square-error estimator [5, 16]. The full LMMSE expression is provided in Appendix B.3.

Although the LMMSE estimator can substantially improve estimation performance compared with LS, its computational complexity is higher

because of the required matrix inversion. It also relies on accurate channel statistics, such as user covariance matrices, which may not be available in practice [5]. Given the scope of this thesis, covariance-based **LMMSE** is therefore not used as a reported channel-estimation baseline. The reported comparisons instead focus on hard-windowing and flow-matching-based learned estimators.

2.2.2.2 Windowing estimator

Windowing-based channel estimation is another classical way to refine the wideband **LS** estimate $\hat{H}_{\text{LS}}$ [17]. The idea is to move the estimate to the delay domain, keep the delay taps where the physical channel is expected to have energy, and then transform back to the frequency domain. This works because a frequency-selective wireless channel usually has limited delay spread, so most of the channel energy is concentrated in a relatively small number of delay taps, whereas additive noise is spread more broadly across the delay domain. The delay-domain representation used here is discussed further in Section 3.2.

Let H_{delay} denote the delay-domain version of $\hat{H}_{\text{LS}}$, obtained by an inverse discrete Fourier transform along the subcarrier axis. Hard-windowing can then be written as

$$\begin{aligned} H_{\text{delay}} &= \text{IDFT}_K(\hat{H}_{\text{LS}}), \\ \hat{H}_{\text{HW}} &= \text{DFT}_K(w \odot H_{\text{delay}}), \end{aligned} \tag{2.4}$$

where $w \in \mathbb{R}^K$ is a delay-domain window broadcast across receive antennas. A hard window keeps the selected delay taps and suppresses the rest. Figure 2.3 illustrates this idea on representative delay-beam magnitude slices: the clean channel energy is concentrated in a small set of delay bins near the edges, while the noisy observation has a broader disturbance floor, which motivates windowing in the delay domain.

Compared with **LMMSE**, this approach is computationally simple and does not require full covariance-matrix inversion. Its performance, however, depends on how well the retained support matches the true channel, and it cannot remove interference components that overlap with the desired channel within the retained support region. The implementation-specific details are described later in Chapter 4.

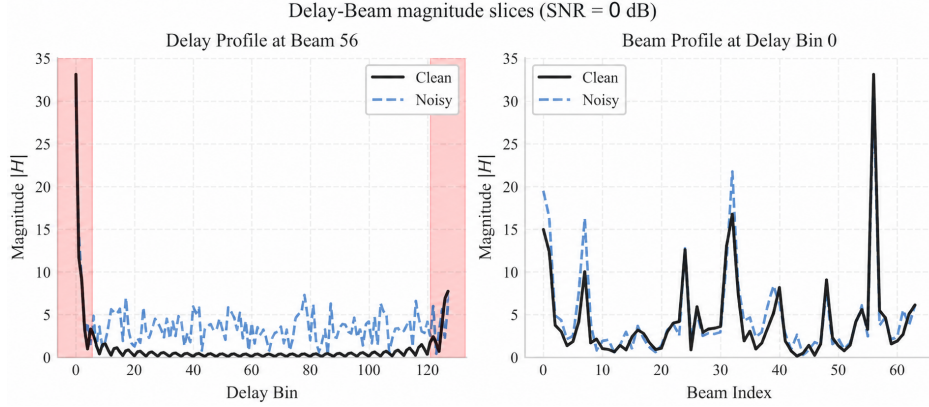


Figure 2.3: Conceptual illustration of hard-windowing in the delay-beam domain. The left panel shows a representative delay profile, where most clean-channel energy is concentrated in a small number of delay bins close to the edges, while the noisy estimate contains a broader disturbance floor. The highlighted edge regions indicate the delay support retained by a simple hard window. The rest of the delay bins are suppressed (set to zero) to reduce noise and interference.

2.2.3 Motivation for Generative Neural Networks

Although the projected observation model in (2.3) is linear, recovering the clean channel H_0 from the projected observation $\hat{H}_{LS,0}$ is an ill-posed inverse problem. Traditional linear estimators, such as the **LMMSE** estimator, rely on the assumption that uncertainty can be adequately described through Gaussian noise and second-order statistics [5]. Under pilot contamination, however, the interference is structured and coherent rather than purely unstructured noise [6, 16].

Because linear estimators can be limited in their ability to disentangle superimposed structured channel components, their performance may degrade under strong pilot contamination. This limitation motivates the use of deep generative neural networks. By learning the complex, non-linear distribution of channel realizations, a generative model can exploit the geometric structure of the target channel and potentially separate it from the structured interference more effectively than conventional linear methods [16, 7, 9].

2.3 Flow Matching

Flow matching is a recent paradigm for generative modeling [18] in which the objective is to learn a continuous flow, represented by a velocity field, that transports a simple prior distribution to a complex data distribution along a predefined probability path. Sampling is then performed by solving an ordinary differential equation (ODE) defined by the learned velocity field. Figure 2.4 gives a qualitative comparison between a smooth deterministic flow trajectory and a more stochastic diffusion-style path.

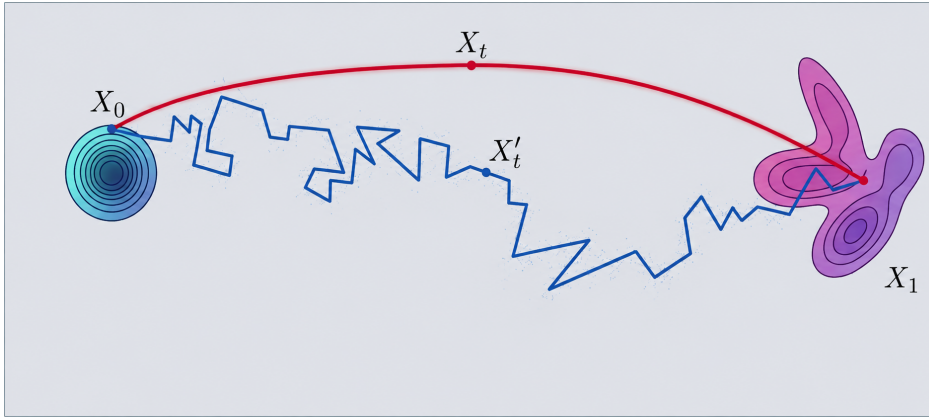


Figure 2.4: Conceptual comparison between deterministic flow and stochastic diffusion trajectories. The smooth red trajectory illustrates a continuous flow path from the source point x_0 toward the target point x_1 , while the jagged blue trajectory illustrates a diffusion-style path with noisy intermediate states. The figure is intended as intuition for the difference in trajectory geometry rather than as a formal depiction of a specific sampler.

Formally, let $x_1 \sim q(x_1)$ denote samples from an unknown data distribution and let $x_0 \sim p_{\text{prior}}(x_0)$ denote samples from a known prior distribution. We define a family of intermediate distributions $\{p_t(x)\}_{t \in [0,1]}$ such that $p_0 = p_{\text{prior}}$ and $p_1 = q$. This endpoint condition is exact in the theoretical probability-path construction. The approximation enters later when the neural network v_θ is trained to approximate the corresponding velocity field. Associated with this probability path is a velocity field $v(x, t)$ that satisfies the continuity equation and controls the evolution of samples along the path.

Given access to $v(x, t)$, one could train a neural network $v_\theta(x, t)$ to

approximate it by minimizing

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x_t} \|v_{\theta}(x_t, t) - v(x_t, t)\|^2. \quad (2.5)$$

However, this objective is intractable in practice, since the marginal velocity field $v(x_t, t)$ depends on all trajectories that pass through x_t at time t .

To address this issue, **Conditional Flow Matching (CFM)** is introduced. Here, “conditional” refers to conditioning the probability path on endpoint samples from the source and target distributions, not on measurement observations from an inverse problem. Instead of regressing directly onto the marginal velocity field, **CFM** considers a conditional probability path $p_t(x | x_0, x_1)$ with a corresponding conditional velocity field $v(x_t, t | x_0, x_1)$. This leads to the surrogate objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1, x_t} \|v_{\theta}(x_t, t) - v(x_t, t | x_0, x_1)\|^2. \quad (2.6)$$

It can be shown that $\mathcal{L}_{\text{CFM}}$ and $\mathcal{L}_{\text{FM}}$ have the same gradient with respect to θ [18], and therefore optimizing either objective yields the same solution. Crucially, unlike the FM objective, the **CFM** objective is tractable: it allows for unbiased Monte Carlo estimates of the marginal velocity by sampling from the conditional path $p_t(x | x_0, x_1)$ and evaluating $v(x_t, t | x_0, x_1)$, without requiring access to the marginal distribution $p_t(x)$. Once the velocity field has been learned, sampling is performed by solving the probability flow ODE

$$\frac{d}{dt}x_t = v_{\theta}(x_t, t), \quad (2.7)$$

starting from an initial condition $x_0 \sim p_{\text{prior}}$, using a numerical ODE solver such as Euler’s or Heun’s method.

2.3.1 Conditional vs. Unconditional Generative Flows

The conditioning in **CFM** should not be confused with conditioning on observed measurements in an inverse problem. When applying flow-based generative modeling to channel estimation, there are two main design choices. The first is unconditional generative modeling, which learns a prior over clean channel realizations and the corresponding probability path. The second is observation-conditioned generative modeling, where the model learns a conditional distribution over clean channel states given the observed noisy measurements.

In unconditional generative flows, the prior distribution is typically chosen

as a standard Gaussian, $p_{\text{prior}}(x) = \mathcal{N}(0, I)$. For channel estimation, the model is trained to learn a velocity field $v_{\theta}(x_t, t)$ that transports samples from this prior to the clean-channel distribution $p_{\text{data}}(x)$, independently of the noisy measurements. At inference time, the learned prior is combined with the observations, and the inverse problem is then solved using an inference procedure such as **Maximum A Posteriori (MAP)** estimation or posterior sampling.

The alternative design choice is to formulate channel estimation conditionally on the observed noisy measurements. In this case, the generative model is trained to learn the posterior over clean channel states. The model learns a velocity field $v_{\theta}(x_t, t \mid c)$ that transports samples from a prior distribution to the posterior distribution over clean channel states, conditioned on the observed measurements c . During inference, the model can directly generate channel estimates that are consistent with the observations, without requiring a separate optimization step.

Section 2.4 includes examples of both unconditional and conditional flow-based models for channel estimation. In particular, [9] is closer to the unconditional setting: it learns a prior over clean channels and then incorporates that prior into a **MAP**-type optimization procedure at inference time. In contrast, [19] performs explicit measurement conditioning by providing the noisy observations as an additional input to the neural network.

2.3.2 Rectified Flow as a Special Case of Flow Matching

The efficiency of the sampling phase depends heavily on the straightness of the learned trajectories. Highly curved trajectories require many integration steps (high **Number of Function Evaluations (NFE)**) to keep discretization errors small, leading to high latency. Recent extensions to flow matching therefore aim to straighten these paths and enable few-step or even one-step generation.

A particularly important special case of flow matching is rectified flow [20], which uses a linear interpolation between source and target samples. This produces straight conditional trajectories in sample space, in contrast to more general probability paths such as Gaussian paths. Formally, the conditional path is defined as

$$x_t = a_t x_1 + b_t x_0, \quad (2.8)$$

where a_t and b_t are predefined scalar schedules satisfying the boundary conditions $a_1 = 1$, $b_1 = 0$, $a_0 = 0$, and $b_0 = 1$. The simplest such trajectory

is a direct linear interpolation:

$$x_t = tx_1 + (1 - t)x_0. \quad (2.9)$$

The corresponding conditional velocity field along this path is given by

$$v_t(x_t | x_1, x_0) = \dot{a}_t x_1 + \dot{b}_t x_0 = x_1 - x_0. \quad (2.10)$$

Flow matching then trains a neural network $v_\theta(x_t, t)$ to approximate the intractable marginal velocity field by regressing onto samples of this conditional velocity. The resulting rectified-flow objective is

$$\mathcal{L}_{\text{RF}}(\theta) = \mathbb{E}_{t, x_1, x_0} \|v_\theta(x_t, t) - (x_1 - x_0)\|^2. \quad (2.11)$$

Because rectified flow encourages straighter trajectories, the resulting **Ordinary Differential Equation (ODE)** can often be integrated accurately with fewer steps. This property is central for one-step or few-step channel estimation, where inference cost is tied directly to the number of network evaluations.

2.3.3 Flow-Model Families

This subsection introduces the main one-step or near-one-step flow-model families used later in the thesis. These models are designed to learn straight trajectories that enable fast sampling, which is important for the one-step focus of this thesis. All of these models can be viewed as special cases of flow matching with a linear interpolation path, but they differ in their specific parameterization and training objectives.

2.3.3.1 Mean Flow

Mean Flow [21] reformulates flow modeling in terms of average velocity over a finite time interval rather than instantaneous velocity at a single time. Mean Flow defines the interval-averaged velocity as

$$u(x_r, r, t) = \frac{1}{t - r} \int_r^t v(x_\tau, \tau) d\tau, \quad r < t, \quad (2.12)$$

Parameterizing the average velocity by a network u_θ , and writing a sampled state on the path as x_t , the Mean Flow objective can be expressed as

$$\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E} \left[\left\| u_\theta(x_t, r, t) - \text{sg}(u_{\text{tgt}}) \right\|_2^2 \right], \quad (2.13)$$

where

$$u_{\text{tgt}} = v(x_t, t) - (t - r) (v(x_t, t) \partial_x u_\theta(x_t, r, t) + \partial_t u_\theta(x_t, r, t)). \quad (2.14)$$

Here, $\text{sg}(\cdot)$ denotes a stop-gradient operation, $\partial_x u_\theta$ is the derivative with respect to the state, and $\partial_t u_\theta$ is the time derivative. For the straight paths used in this thesis, the corresponding direct velocity is $v^* = x_1 - e$. Sampling then replaces explicit time integration by the learned average velocity,

$$x_t = x_r + (t - r)u_\theta(x_r, r, t), \quad (2.15)$$

so one-step generation from the chosen start state e becomes $\hat{x}_1 = e + u_\theta(e, 0, 1)$. Intuitively, u summarizes the net transport over a finite interval, so learning u makes it possible to model a large jump from an intermediate state directly toward the endpoint.

The left panel of Figure 2.5 illustrates this idea: Mean Flow replaces many local tangent directions by one interval-averaged displacement between r and t , which is why it is naturally suited to one-step or few-step sampling.

In the original Mean Flow loss, u_{tgt} is self-referential: it is the target for $u_\theta(x_t, r, t)$, but it is itself computed using the derivatives $\partial_x u_\theta$ and $\partial_t u_\theta$ of the same network being trained. Improved Mean Flow moves this derivative correction from the target side to the prediction side [12]. The model still learns the interval-averaged velocity u_θ , which is the quantity used for one-step sampling, but training compares a corrected prediction to the simple straight-path velocity. Let $v_\theta(x_t, t)$ denote an instantaneous-velocity prediction obtained from the same network. The implementation used in this thesis uses the boundary-condition form $v_\theta(x_t, t) = u_\theta(x_t, t, t)$, although the paper also discusses an auxiliary v -head. Define

$$\dot{u}_{\theta, \text{sg}}(x_t, r, t) = \text{sg}(v_\theta(x_t, t) \partial_x u_\theta(x_t, r, t) + \partial_t u_\theta(x_t, r, t)), \quad (2.16)$$

and the compound velocity prediction

$$V_\theta(x_t, r, t) = u_\theta(x_t, r, t) + (t - r)\dot{u}_{\theta, \text{sg}}(x_t, r, t). \quad (2.17)$$

The improved Mean Flow objective is then

$$\mathcal{L}_{\text{iMF}}(\theta) = \mathbb{E} \left[\|V_\theta(x_t, r, t) - v^*\|_2^2 \right]. \quad (2.18)$$

In this form, the right-hand side of the loss is only the direct velocity $v^* = x_1 - e$. The Mean Flow correction is instead added to the model prediction through V_θ . The learned object used at inference is still the interval-averaged velocity u_θ , so this formulation is referred to simply as Mean Flow in the remainder of the thesis.

A further Mean-Flow-style extension is AlphaFlow [22]. AlphaFlow does not change the one-step average-velocity sampler. Instead, it changes how the Mean Flow objective is introduced during training. The idea is to start from an easier direct regression target and gradually move toward the full Mean Flow objective. The original Mean Flow loss can be decomposed as a direct average-velocity regression term $\mathcal{L}_{\text{TFM}}$ and a trajectory-consistency correction term $\mathcal{L}_{\text{TC}_c}$. The first term is easier to learn because it resembles ordinary velocity regression, while the second term adds the Mean Flow consistency correction. AlphaFlow then introduces a scheduled family of objectives,

$$\mathcal{L}_\alpha(\theta) = \mathbb{E}_{t,r,x_t} \left[\alpha^{-1} \|u_\theta(x_t, r, t) - (\alpha \tilde{v}_{s,t} + (1 - \alpha)u_{\theta^-}(x_s, r, s))\|_2^2 \right], \quad (2.19)$$

with

$$s = \alpha r + (1 - \alpha)t. \quad (2.20)$$

Here, x_s denotes the intermediate state at time s , and $\tilde{v}_{s,t}$ denotes the known straight-path velocity between the two times s and t . For the linear path used in this thesis,

$$\tilde{v}_{s,t} = \frac{x_t - x_s}{t - s} = x_1 - e = v^*. \quad (2.21)$$

The factor α^{-1} compensates for the residual shrinking as s approaches t when α becomes small, so the consistency signal remains visible in the loss. At $\alpha = 1$, the target reduces to the direct regression term $\mathcal{L}_{\text{TFM}}$. As $\alpha \rightarrow 0$, the gradient approaches the original Mean Flow objective. The AlphaMeanFlow implementation used in this thesis follows the same curriculum idea by interpolating between a direct regression target and the implemented Mean Flow objective with an annealed α schedule. It is therefore reported separately as an ablation of the Mean-Flow-style training objective rather than treated as a separate model family.

2.3.3.2 Data-Parameterized Flow

Inspired by [23], data-parameterized flow can be viewed as another flow-matching paradigm that keeps the same straight rectified-flow trajectory but replaces velocity prediction with direct endpoint prediction. On the straight path

$$x_t = (1 - t)e + tx_1, \quad (2.22)$$

the network directly predicts the clean endpoint as

$$f_\theta(x_t, t) \approx x_1, \quad (2.23)$$

and is trained with the regression objective

$$\mathcal{L}_{\text{Data}}(\theta) = \mathbb{E}_{t,e,x_1} \left[\|f_\theta(x_t, t) - x_1\|_2^2 \right]. \quad (2.24)$$

Under the straight path $x_t = (1 - t)e + tx_1$, this induces the velocity proxy

$$\hat{v}_\theta(x_t, t) = f_\theta(x_t, t) - e, \quad (2.25)$$

so the model can still be interpreted as learning a finite jump toward the endpoint.

2.3.3.3 Consistency Objectives

Two closely related consistency formulations can be built on top of this endpoint-oriented view, but they optimize different learned objects. The first is an endpoint formulation, where consistency is imposed directly on the endpoint map $f_\theta(x_t, t)$ so that

$$f_\theta(x_{t_1}, t_1) \approx f_\theta(x_{t_2}, t_2) \approx x_1 \quad (2.26)$$

for points that lie on the same path. The right panel of Figure 2.5 shows this endpoint-consistency view: states observed at different times on the same trajectory should still map to the same clean endpoint. The endpoint map is written with the clean-boundary condition

$$f_\theta(x, 1) = x. \quad (2.27)$$

Following **improved consistency training (iCT)** [13], this boundary is enforced with the wrapper

$$f_{\theta}(x_t, t) = c_{\text{skip}}(t)x_t + c_{\text{out}}(t) \text{model}_{\theta}(x_t, t), \quad (2.28)$$

where the coefficients satisfy

$$c_{\text{skip}}(1) = 1, \quad c_{\text{out}}(1) = 0. \quad (2.29)$$

In the implementation, this is written directly in terms of the path parameter t through the remaining-noise factor $1 - t$:

$$c_{\text{skip}}(t) = \frac{\sigma_{\text{data}}^2}{(1 - t)^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(t) = \frac{(1 - t) \sigma_{\text{data}}}{\sqrt{(1 - t)^2 + \sigma_{\text{data}}^2}}, \quad (2.30)$$

with σ_{data} a fixed data-scale hyperparameter. This parameterization satisfies $c_{\text{skip}}(1) = 1$ and $c_{\text{out}}(1) = 0$, so the clean-boundary condition is enforced directly at the endpoint. Training then uses adjacent levels $t_1 < t_2$ on the same trajectory together with a Pseudo-Huber objective,

$$\mathcal{L}_{\text{cons}}(\theta) = \mathbb{E} [\rho_c(f_{\theta}(x_{t_1}, t_1) - \text{sg}(f_{\theta}(x_{t_2}, t_2)))], \quad (2.31)$$

with

$$\rho_c(z) = \sqrt{\|z\|_2^2 + c^2} - c. \quad (2.32)$$

In practice, the normalized time axis is discretized into a finite grid of levels rather than treated as fully continuous during training. Adjacent pairs (t_1, t_2) are then sampled as neighboring points on this grid, so the discrete-level schedule can be understood as the time-axis resolution used by the consistency objective: fewer levels give a coarser grid and larger adjacent gaps, whereas more levels give a finer grid and smaller adjacent gaps. Chapter 4 later summarizes the reported level range and related settings. Although this endpoint model does not parameterize a velocity field directly, it is still built on the same straight rectified-flow path $x_t = (1 - t)e + tx_1$ used by the other one-step flow families. In that sense, it is path-based and flow-derived: the model learns a path-consistent endpoint map rather than an explicit instantaneous velocity field. Inference is then reduced to a single evaluation $\hat{x}_1 = f_{\theta}(e, 0)$. The second formulation, Consistency Flow Matching (Consistency-FM) [24], instead places the consistency objective in velocity space. This constrains trajectories started at different times but ending at the same sample to agree through a consistent velocity field. The key difference is therefore the learned object: **iCT**-style consistency regularizes the endpoint map directly, whereas Consistency-FM regularizes a velocity

model and derives endpoint consistency from that velocity field. The model implemented and reported in the result tables is the **iCT**-style endpoint model, denoted simply as consistency. Consistency-FM is included here only as a related alternative, as it is closer to the standard flow-matching paradigm.

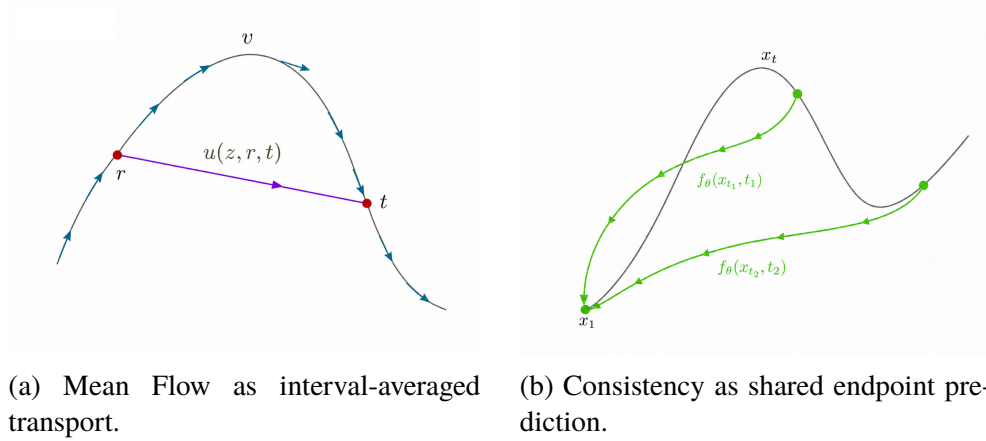


Figure 2.5: Geometric intuition for two fast-model ideas used in this thesis. Left: Mean Flow summarizes a finite interval with one averaged displacement. Right: consistency training maps different points on the same trajectory to the same endpoint.

2.3.4 Distillation

Not all low-step flow improvements come from changing the learned object or the inference rule itself. A separate line of work instead focuses on distillation techniques that improve the training procedure by having a teacher model provide guidance to a smaller student model.

2.3.4.1 Consistency Mid-Training

CMT [14] is not a separate generative family or a new sampling rule. It is a teacher-student training procedure for fast flow models such as consistency models and Mean Flow. The idea is to improve a fixed one-step or few-step student by replacing part of the hard direct supervision with trajectory-aligned targets from a stronger teacher. This also mitigates the geometric mismatch in direct one-step training, where the student must approximate a long finite-time jump while the teacher reaches the same endpoint through many smaller steps.

In this thesis that additional guidance is useful because the inference-time models must remain small to satisfy the cost budget, making pure one-step

training more challenging.

CMT inserts an intermediate stage between teacher pre-training and final student training. Given a teacher-generated trajectory $\{\hat{x}^{t_i}\}_{i=0}^M$ with $0 = t_0 < \dots < t_M = 1$, the student can be trained either on endpoint targets or on shorter interval targets. A consistency-style endpoint variant regresses a trajectory state to the teacher endpoint with a chosen endpoint loss d ,

$$\mathcal{L}_{\text{CMT-CM}}(\theta) = \mathbb{E}_i \left[d(f_\theta(\hat{x}^{t_i}, t_i), \hat{x}^{t_M}) \right], \quad (2.33)$$

while a Mean-Flow-style interval variant regresses the student average velocity against teacher finite differences over sampled trajectory times $r < t$,

$$\mathcal{L}_{\text{CMT-MF}}(\theta) = \mathbb{E}_{r < t} \left[\left\| u_\theta(\hat{x}^t, r, t) - \frac{\hat{x}^t - \hat{x}^r}{t - r} \right\|_2^2 \right]. \quad (2.34)$$

Both objectives reduce training to ordinary regression with fixed teacher labels, giving a more stable, trajectory-aligned initialization than random weights or pure diffusion pre-training.

The experiments in this thesis use the one-step specialization of this idea: both Mean-Flow and consistency students are supervised on the full jump from the noisy start of the teacher trajectory to the teacher endpoint, as detailed in Chapter 4. **CMT** therefore changes the training targets used to produce the student checkpoint, not the student inference rule itself.

2.4 Related Work

This section reviews prior research on channel estimation using generative machine learning methods. In particular, we discuss approaches that leverage generative models to mitigate noise and pilot contamination in **MIMO** systems. We also review recent flow matching extensions that are applicable to inverse problems, with particular emphasis on few-step and one-step flow matching models.

2.4.1 Channel Estimation Using Generative Models

2.4.1.1 Other generative models

Recent work has explored the use of generative neural networks to mitigate pilot contamination and noise in massive **MIMO** systems. In

[16], the authors propose a **Variational Autoencoder (VAE)**-based channel-estimation framework that models the joint distribution of channel realizations across cells and learns first- and second-order conditional statistics in order to compute conditional linear minimum mean-square error (**LMMSE**) estimates. By learning the conditionally Gaussian structure of both desired and interfering **UE** channels, their method reduces the impact of pilot contamination and outperforms classical estimators that ignore inter-cell interference. The same work also shows, using single-cell **VAE** models, that pilot contamination cannot be neglected or accurately approximated as **Additive White Gaussian Noise (AWGN)**.

However, this approach relies on using a **VAE** to parameterize statistical channel moments and subsequently deriving an estimator based on these moments, rather than directly learning a generative mapping from noisy observations (or noise) to channel realizations. As a result, the final estimation step is constrained by the **LMMSE** formulation and implicitly approximates the posterior distribution as Gaussian.

Another family of generative models for channel estimation is based on score-based diffusion models. These models do not suffer from the same limitation and can instead learn the data distribution directly. In [7], deep score-based models are used to estimate high-dimensional **MIMO** channel state information in a noise-limited interference regime by learning the gradient of the log-prior distribution and generating samples through posterior sampling. That work also presents theoretical results guaranteeing recovery of the correct estimate, assuming that a sufficient number of measurements are available when performing posterior sampling, even in the presence of train-test distributional mismatch. Experiments on simulated channels demonstrate competitive in-distribution performance and robustness to out-of-distribution scenarios, at the cost of multiple iterative sampling steps during inference.

In [8], Interference Cancellation Diffusion Models are proposed for a related task in wireless semantic communications with Rayleigh fading channel interference, which is conceptually related to pilot contamination in **CSI** recovery. There, dual diffusion models are used to iteratively denoise received signals and suppress interference.

In contrast, the flow-based approach considered in this thesis directly learns a velocity field that transports samples from a simple prior to the data distribution via deterministic **ODE**-based sampling. This enables few-step or one-step inference without reliance on stochastic iterative sampling, which can incur prohibitively high computational cost and limit the applicability of diffusion-based models in real-world scenarios. At the same time, the

proposed approach avoids restrictive explicit parametric assumptions on the posterior distribution, making it well suited for channel estimation. However, flow matching has not yet been widely explored in this context.

2.4.1.2 Flow matching for channel estimation

Flow matching-based generative models for **MIMO** channel estimation are presented in [19] and [9]. The method in [19] formulates channel estimation within the flow-matching framework by constructing a conditional probability path from the noisy channel distribution to the true channel distribution along a straight-line trajectory. Channel estimates are obtained by solving an **ODE** driven by the learned velocity field, effectively using flow matching as a denoising mechanism. In contrast, [9] employs flow matching within an optimization framework, where a learned prior over clean channels is incorporated into a **MAP** objective and integrated into a plug-and-play proximal gradient descent procedure to iteratively refine channel estimates. This iterative inference procedure can incur substantially higher inference latency than one-step estimators.

More recently, Recursive Flow (RC-Flow) is proposed in [25] as a generative framework that reformulates inference as a closed-loop fixed-point iteration. Unlike conventional open-loop flow-matching solvers that refine a single trajectory from noise, RC-Flow employs a serial restart mechanism with anchored trajectory rectification to correct drift. By nesting a dual-scheduling inner loop within an outer recursive anchor reset, RC-Flow alternates between flow-based prior estimation and a closed-form proximal projection. This design is intended to improve data consistency and stability while reducing noise-induced artifacts in standard open-loop solvers.

Finally, [26] proposes a one-step flow-matching model for channel estimation using a Mean Flow formulation. By directly modeling the average velocity field over the entire flow trajectory, this method enables one-step sampling without requiring a numerical **ODE** solver at inference time.

These works indicate that flow matching can reduce sampling overhead compared with iterative generative schemes, such as score-based methods, while retaining competitive estimation performance across different channel conditions. In particular, RC-Flow performs especially well in low-**SNR** regimes. However, the existing flow-matching channel-estimation studies considered here assume narrowband signal models and do not explicitly consider pilot contamination, which can severely degrade channel estimation performance in practical multi-user scenarios. This motivates the focus of

this thesis on wideband, pilot-contaminated channel estimation with one-step flow-based models. For that reason, the comparison to prior work in this thesis is primarily methodological rather than numerical. The cited papers differ in signal model, bandwidth assumption, interference setting, inference budget, and reported metric, so a raw paper-to-paper ranking would be misleading.

2.4.2 Flow Matching Models

2.4.2.1 General flow matching models for inverse problems

A **MAP** perspective aligns naturally with channel estimation: recovering the most probable channel given noisy observations using a generative prior. However, incorporating flow priors into **MAP** objectives is computationally prohibitive for high-dimensional **MIMO** channels, as computing likelihoods requires backpropagating through an **ODE** solver. To address this, [27] proposed Iterative Corrupted Trajectory Matching (ICTM). Instead of differentiating through the full **ODE** trajectory, ICTM approximates the global **MAP** objective by partitioning the flow trajectory into multiple discrete local **MAP** subproblems. These intermediate states are optimized sequentially using standard gradient steps, effectively matching a corrupted trajectory to an auxiliary path without full **ODE** backpropagation, accelerating flow-based **MAP** estimation.

An alternative to **MAP** optimization is to adapt posterior-sampling ideas within the flow-matching framework. [28] introduced Flow-Driven Posterior Sampling (FlowDPS), which extends diffusion-style inverse solvers to flow models by decomposing the flow **ODE** into clean-signal and noise components. At inference time, FlowDPS combines likelihood guidance with stochastic noise injection at each integration step to steer samples toward the posterior. While FlowDPS yields strong reconstruction quality for generic linear inverse problems without requiring additional training, it still relies on a multi-step numerical **ODE** solver and repeated likelihood-gradient evaluations, which can limit applicability when strict inference-cost constraints apply.

Related plug-and-play strategies have also been explored for flow models. For example, [29] introduced *Flower*, an iterative solver with three stages: flow-consistent destination estimation, projection onto a feasible set defined by the forward model, and time progression along the flow trajectory. Although this approach yields strong reconstruction quality and provides a principled link between plug-and-play updates and flow-based inverse solvers, it also fundamentally requires multiple neural network evaluations at inference time.

2.4.2.2 One-step and few-step methods

Beyond iterative inverse-problem solvers, recent work has focused on reducing the number of function evaluations of the flow model itself. A central difficulty is that, even when the conditional paths are defined by simple interpolations, the learned marginal vector field can still be curved because it averages over many source–target pairings. This makes very-low-step sampling sensitive to truncation error.

One way to alleviate this is to make the transport paths easier to integrate. **Optimal Transport (OT)-CFM** constructs an approximate optimal coupling between minibatches of source and target samples [30], reducing trajectory intersections and yielding a smoother marginal velocity field. **Reflow** [20] instead retrain a rectified-flow model on synthetic pairs generated from its own trajectories, progressively aligning the transport with the learned dynamics. A recent extension combines this straightening idea with Mean Flow training, using rectified trajectories to make efficient one-step generation easier [31]. Related one-step distillation work, such as **SC-Flow** [32], studies how to preserve sample quality when compressing a multi-step generator into one or a few long jumps.

2.5 Summary

This chapter introduced the wideband multi-cell massive **MIMO-OFDM** uplink setting, the projected observation model under **AWGN** and pilot contamination, and the classical reference estimators along with the reported hard-windowing and ideal channel-estimation comparisons. It then positioned flow matching as the main alternative for one-step channel estimation under structured interference, covering rectified flow, Mean Flow and its scheduled **AlphaMeanFlow** extension, data-parameterized and consistency-based models, and **CMT** as a teacher-student training enhancement for fast students. Table 2.1 summarizes the main channel-estimation paradigms discussed in the chapter, while Table 2.2 summarizes the flow-matching extensions, fast-model families, and training enhancements most relevant to the thesis. Overall, the chapter motivates the thesis focus on wideband, pilot-contaminated channel estimation with one-step learned estimators.

Table 2.1: Qualitative comparison of channel-estimation paradigms discussed in this chapter.

Paradigm	Representative Methods	Typical Inference Cost
Classical projection and refinement	LS, HW, LMMSE	Ultra-low to high
Moment-based generative estimation	VAE-based estimators [16]	Low to moderate
Score / diffusion models	Score-based and diffusion estimators [7, 8]	High
Iterative flow-based methods	Conditional FM, ICTM, FlowDPS, Flower, RC-Flow	Moderate to high
Fast flow-based methods	Mean Flow and AlphaMeanFlow, data-parameterized flow, consistency models	Low

Table 2.2: Flow-matching extensions, fast-model families, and training enhancements discussed in this chapter.

Technique	Key Distinction	Typical Use
ICTM [27]	Local flow subproblems as a MAP surrogate	Iterative inference
FlowDPS [28]	Likelihood-guided flow sampling with stochastic correction	Iterative inference
Flower [29]	Endpoint updates plus feasible-set projection	Iterative inference
RC-Flow [25]	Recursive anchor resets with proximal refinement	Iterative inference
OT-CFM [30]	Optimal-transport pairing for smoother training flows	Few-step inference
Reflow [20]	Self-generated retraining to straighten trajectories	Few-step inference
Mean Flow / AlphaMeanFlow [21, 12, 26, 22]	Interval-averaged velocity, with AlphaMeanFlow adding target scheduling	One-step inference
Data-parameterized flow [23]	Direct endpoint prediction	One-step inference
Consistency models [13, 24]	Endpoint or velocity consistency along one trajectory	One-step inference
SC-Flow [32]	Self-corrected distillation for compressed sampling	One-step inference
CMT [14]	Teacher-trajectory mid-training for fast students	Training-time distillation

Chapter 3

Methods

The purpose of this chapter is to describe how the thesis is carried out from a methodological and experimental perspective. It outlines the overall research process, the simulation-based data pipeline, the main model-design choices, and the evaluation principles used in the study.

3.1 Research Process and Paradigm

The methodology adopted in this thesis combines an iterative engineering workflow with quantitative experimental evaluation. The work is primarily deductive: candidate model classes and design hypotheses are derived from prior theory and literature on flow matching and channel estimation, and are then tested through controlled experiments, comparative studies, and ablations. Methodologically, the study aligns with a positivist paradigm in which model behavior is evaluated through reproducible measurements of reconstruction accuracy, robustness, computational cost, and downstream communication performance.

Because prior work on one-step or few-step flow-based channel estimation remains limited, the process is iterative rather than strictly linear. Implementation, empirical evaluation, and model refinement are repeated as the understanding of the problem improves.

The workflow therefore combines literature study, simulation-based data preparation, model development, and quantitative evaluation through the following core phases:

Step 1 Preparation and Planning: Establishing the theoretical foundation through literature review, preparing simulator-aligned data, and defining the initial experimental hypotheses and model configurations.

- Step 2 Iterative Experimentation:** Conducting experiments on the full pilot-contaminated formulation and refining the model design through comparative studies, ablations, and repeated adjustments to the context-input design, flow objective, and backbone architecture.
- Step 3 Evaluation and Simulator Integration:** Once promising model configurations have been identified, they are integrated into the company simulator for end-to-end evaluation. At the same time, local reconstruction metrics and comparative analyses are used to interpret model behavior and support the thesis conclusions.

3.2 Data Collection

In this section, we describe the data collection process, including the generation of synthetic datasets using channel simulation tools, the modeling of interference and noise, and the preprocessing steps applied to prepare the data for training and evaluation.

3.2.1 Dataset Generation and Channel Modeling

To generate the training dataset, we employ the **QuaDRiGa v2.8.1** channel simulation tool [33, 34]. QuaDRiGa is a geometry-based stochastic channel model designed to simulate realistic **MIMO** channels characterized by spatial consistency, time evolution, and three-dimensional propagation across diverse environments. The simulation follows the **3rd Generation Partnership Project (3GPP)** TR 38.901 spatial channel model, which provides the standardized propagation framework used for 5G channel evaluation [35]. Within that framework, the dataset generation uses an Urban Macro Non-Line-of-Sight (UMa-NLOS) scenario with a single **BS**. The **BS** is equipped with a 64-element dual-polarized **Uniform Planar Array (UPA)** configured with 4 rows and 8 columns ($4 \times 8 \times 2$). The **UEs** are equipped with dual-polarized antennas. In the dataset construction used here, the two **UE** antenna ports are modeled as spatially uncorrelated in the simulator setup. This modeling choice follows internal company measurement-based practice, although the underlying measurements were not available for inclusion or independent verification in this study. The transmission operates over an **OFDM** grid with 192 subcarriers.

The simulator yields 800 segments of different **UE** mapping, each with N_{TTI} time frames, producing four-dimensional complex-valued tensors of

channel realizations. Each raw tensor has shape $(800, N_{TTI} \times K \times N_{BS} \times N_{UE})$, specifically $(800 \times 51 \times 192 \times 64 \times 20)$, where N_{TTI} denotes the number of transmission-time intervals (temporal evolution), K the subcarriers (frequency domain), N_{BS} the base-station antenna elements, and N_{UE} the total **UE** antenna ports per simulation frame. The **UE** antenna dimension N_t is collapsed into this last dimension.

To evaluate the generalization capabilities of the model, the testing phase utilizes an independent evaluation dataset whose clean and interfering channels follow the standardized **3GPP Cluster Delay Line B (CDL-B)** profile [35]. The evaluation data follow the same tensor structure, but with shape $(1 \times 1000 \times 192 \times 64 \times 24)$. Evaluating the model on the **CDL-B** dataset after training on the QuaDRiGa dataset therefore probes robustness to varying delay profiles and out-of-distribution channel conditions.

During training, both **SNR** and **Interference-to-Noise Ratio (INR)** are sampled randomly from predefined ranges. During evaluation, **SNR** is examined on a predefined sweep of test points within the target range, while **INR** is held fixed for each test configuration. The exact distributions are documented in Chapter 4. Beyond this mismatch, Figure 3.1 shows a structural distribution shift in the transformed channels: the **CDL-B** evaluation set exhibits a narrower delay spread and a slightly shifted beam-spread distribution than the QuaDRiGa training set. The delay and beam domain are described below in Section 3.2.3.

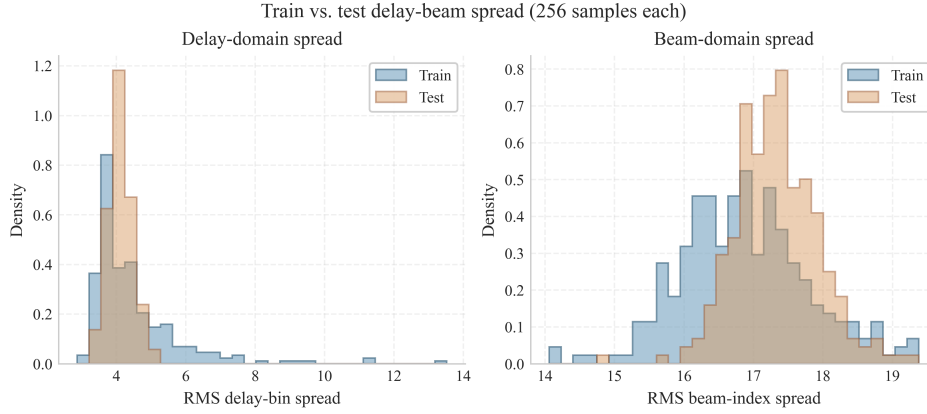


Figure 3.1: Comparison of train and test spread statistics in the delay and beam domains over 256 representative samples from each set. Here, spread denotes the RMS spread of the normalized channel magnitude over delay-bin index or beam-index, respectively, and the y-axis shows histogram density rather than counts. The **CDL-B** evaluation set is more tightly concentrated in delay spread and slightly shifted in beam spread relative to the QuaDRiGa training set, illustrating an additional train/test distribution shift beyond the sampled **SNR/INR** ranges.

For the pilot signals, we use **Quadrature Phase Shift Keying (QPSK)** modulation mapped to a Comb-2 pattern over 192 subcarriers. In the considered system, the pilot modulation is fixed, whereas the data-symbol modulation can vary between configurations. In the studied simulator setup, however, the data symbols also use **QPSK**, a low-order modulation scheme suited to noisy operating conditions. The evaluation therefore focuses on a low-**SNR** regime that is relevant for this setup and in which differences in channel-estimation quality are most clearly reflected in end-to-end **SER**. The exact implementation settings are reported in Chapter 4. Figure 3.2 illustrates the conceptual difference between **QPSK** and a higher-order modulation scheme, such as 16-QAM, through their constellation diagrams and decision regions. Because **QPSK** has fewer constellation points, it is generally more robust to noise and interference than higher-order alternatives in the considered noisy regime.

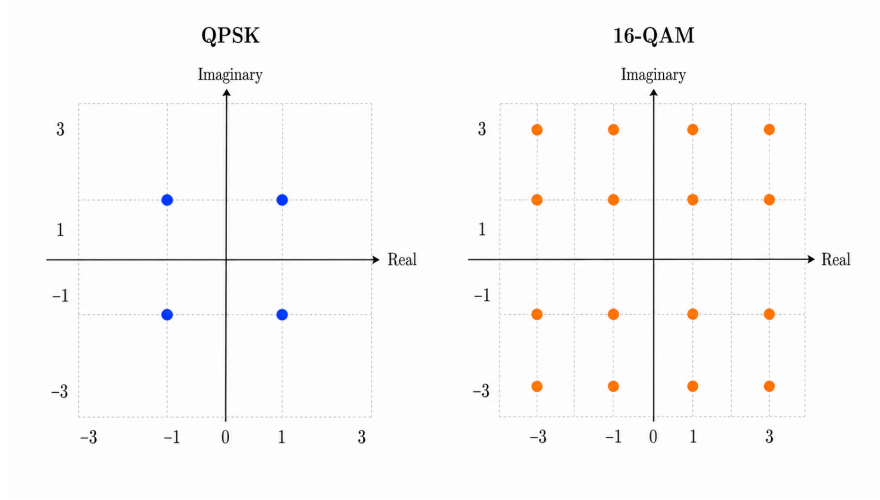


Figure 3.2: Conceptual comparison of **QPSK** and 16-QAM constellations. In nearest-neighbor detection, a noisy or predicted complex symbol is assigned to the valid constellation point closest to it in the complex plane. Because **QPSK** has fewer constellation points than 16-QAM, its decision regions are larger, so the same perturbation is less likely to cross a decision boundary and cause a symbol error. This illustrates why low-order modulation is generally more robust to noise and interference in the noisy regime considered in this thesis.

The QPSK mapping draws complex symbols from the constellation $s_k \in \{(\pm 1 \pm i)\}$. Normally, the symbols are normalized with a factor $\frac{1}{\sqrt{2}}$, giving constant modulus ($|s_k|^2 = 1$). In the implemented simulator, however, the pilot is normalized over the full 192-subcarrier Comb-2 grid, including the zero entries. The Comb-2 pattern maps these symbols to every other subcarrier, resulting in a structurally sparse grid of 96 active and 96 null subcarriers. Consequently, the proposed model operates exclusively on the 96 active subcarriers, focusing entirely on mitigating pilot contamination and noise. After restricting the pilot to these active subcarriers, the original full-grid pilot-power normalization no longer holds, but this is compensated for in the **LS** projection step. For presentation and downstream simulator use, the resulting estimates are interpolated back to the dense subcarrier grid using deterministic linear interpolation along the subcarrier axis. This post-processing step is parameter-free and exploits the strong correlation between adjacent subcarriers.

Pilot-associated tensors are written with axes corresponding to N_{TTI} , N_{DMRS} , K , N_{BS} , and N_{UE} . The stored target-pilot tensors have

shape $(3500, 2, 192, 64, 1)$ and the interference-pilot tensors have shape $(3500, 2, 192, 64, 4)$. The $N_{DMRS} = 2$ dimension indexes the two pilot-carrying **OFDM** symbols in the active slot configuration. Throughout this thesis, the *single-Demodulation Reference Signal (DMRS)* setting means that the estimator processes one of these two pilot symbols at a time, whereas the *joint-DMRS* setting means that both pilot symbols are supplied together and predicted in one forward pass. During training, this pilot bank is sampled on the fly rather than matched one-to-one to the collapsed channel samples, so the stored pilot realizations are randomly paired with many desired/interfering channel examples. In the default single-**DMRS** setup, one of the two pilot symbols is selected uniformly at random for each example. In the later joint-**DMRS** variants, both pilot symbols are retained and predicted together in one model call. During evaluation, in contrast, the pilot realizations are fixed by the test configuration rather than resampled on the fly.

3.2.2 Data Preprocessing and Interference Modeling

For training and validation, individual **TTIs** and **UEs** are treated as independent data samples. Hence, the dimensions are collapsed together in the first dimension. This is done not only to increase the number of supervised examples, but also to expose the model to many different random **UE-drop** realizations during training, which is intended to improve generalization across user positions and channel conditions. To facilitate training, the complex-valued entries are decomposed into their real and imaginary components. Consequently, the data is reshaped into a tensor of shape $(816000 \times 192 \times 64 \times 2)$, where the dimensions represent the total samples, subcarriers, base station antennas, and the real/imaginary components, respectively. The resulting dataset is split 99/1 into training and validation partitions. Because the collapsed dataset is large, this still leaves roughly 8,000 held-out validation examples, which was sufficient for checkpoint selection while preserving almost all samples for training.

For evaluation, this training-time collapse is not used: the **UE** dimension is kept explicit rather than collapsed into time. Performance is measured for a fixed evaluation **UE**. This asymmetry is deliberate. During training, collapsing **TTIs** and **UEs** exposes the model to a large variety of random **UE-drop** realizations and supports generalization. During evaluation, however, the simulator-facing test structure is retained so that the reported **SER** characterizes performance for a fixed evaluation **UE** and evaluation setting rather than only giving an average over collapsed samples.

In the main single-**DMRS** comparisons, one pilot-carrying symbol is processed at a time, matching the corresponding training setup. Single-**DMRS** is used in the main ablations as a controlled reduced interface that keeps training cost manageable while isolating model-design effects. In the later joint-**DMRS** variants, both **DMRS** realizations are supplied together and the model predicts both in one forward pass, so one network evaluation produces both pilot-symbol estimates. Joint-**DMRS** is reserved for the final integrated evaluation because the simulator ultimately provides both pilot symbols, and a deployment-style one-step estimator should therefore produce both estimates in one forward pass. For that reason, the final integrated comparison is trained and reported in joint-**DMRS** form rather than as two separate single-symbol estimates. Unlike training, evaluation does not resample pilots on the fly: the underlying evaluation channels are drawn from the separate **CDL-B** dataset with 1000 time frames and are paired with the fixed pilot realizations used in the test configuration, resulting in 1000 test frames with both **DMRS** realizations available.

To model realistic signal degradation, we account for two primary interference sources: additive environmental noise and pilot contamination. During training, the interfering realizations are sampled from the same single-**BS** QuaDRiGa channel pool as the desired channels, but from different **UE/TTI**. During evaluation, the clean **CDL-B** channels are combined with separately generated **QPSK** pilots and interference realizations. The environmental noise is modeled as a zero-mean Gaussian process with constant noise power $N_0 = \mathbb{E}[\|n\|^2] = \sigma^2 = 1$, where n is the noise vector. Given that the channels are normalized across subcarriers ($\mathbb{E}[\|h\|^2] = 1$) and the noise power is constant, changing the linear **SNR** is equivalent to scaling the channel by a factor of $\beta_{\text{snr}} = \sqrt{\text{SNR}}$. Different **SNR** conditions can therefore be simulated by adjusting the channel power.

For pilot contamination, we generate four interferers per sample. Because of the Comb-2 pilot allocation, only the interferers that occupy the same parity pattern as the selected target **DMRS** overlap on the active subcarriers. Hence, four interferers are present at the simulator level, but each effective projected observation is contaminated by only two interferers. Under the same unit-noise normalization, changing the linear **INR** for interferer i is equivalent to scaling that interferer by $\alpha_{\text{inr},i} = \sqrt{\text{INR}_i}$. The effective projected noisy observation used by the network can then be written as

$$y = \beta_{\text{snr}} H + \sum_{i=1}^4 \alpha_{\text{inr},i} (H_{\text{int},i} \odot \rho_i) + n, \quad (3.1)$$

where $\delta_i \in \{0, 1\}$ indicates whether interferer i shares the selected comb pattern, and $\rho_i = \delta_i \frac{P_{\text{int},i} \odot P_{\text{target}}^*}{|P_{\text{target}}|^2} \in \mathbb{C}^K$ collects the per-subcarrier pilot-overlap factors. For each sample, exactly two of the four δ_i are equal to one. Here, y denotes the projected observation seen by the estimator. This notation makes explicit that the simulator first forms the received pilot signal and then applies **LS** projection on each active subcarrier to obtain the observation used by the network. Since the neural estimator operates only on the 96 active subcarriers, the corresponding active-band slice is extracted for the network input. In the joint-**DMRS** variants, the same construction is applied to both pilot symbols and the two projected observations are stacked for a single forward pass.

3.2.3 Domain Transformations

For all neural models, the active-subcarrier representation is transformed into a delay-beam tensor represented on a 128-bin delay axis. This length is obtained by upscaling the 96 active-subcarrier representation to the nearest power of two before the transform, which gives a convenient FFT-compatible grid while preserving the same active-band information.

When visualizing the raw data in the frequency-antenna domain, the signal energy is spread over the whole grid with substantial fluctuations. Even at medium **SNR** levels, the target structure is only partially visible in the noisy observation, as illustrated by the representative 1D slices and 3D magnitude views in Figure 3.3 and Figure 3.4 at 0 dB **SNR**. This diffuse appearance is partly a consequence of observing the channel in a domain where delay structure and spatial structure are still mixed. We therefore transform the data into the delay-beam domain before feeding it to the network.

The transition from the frequency-antenna domain to the delay-beam domain follows the physical structure of the channel. Along the frequency axis, multipath components arrive with different delays. Applying an Inverse Discrete Fourier Transform (IDFT) across the subcarriers therefore maps the channel to a discrete delay domain, in which the energy concentrates into a limited set of delay bins associated with the dominant propagation components [5, 17].

Along the spatial axis, the base station employs a 2D antenna grid, namely a **UPA**. Signals arriving from different directions induce different spatial phase progressions across the array. Applying a 2D Discrete Fourier Transform (DFT) across the antenna dimensions maps this variation into beam bins associated with the dominant angles of arrival, following the usual array-domain interpretation of Massive **MIMO** channels [1]. Taken together, these

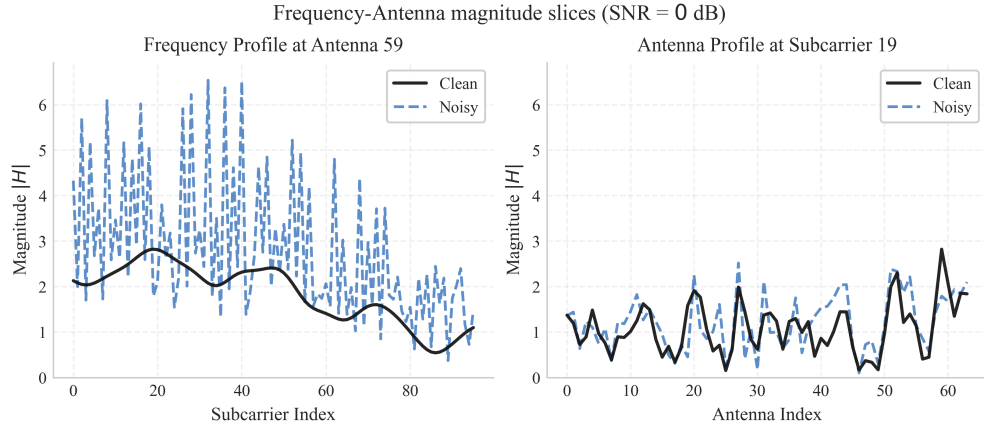


Figure 3.3: Representative 1D magnitude slices of the clean and noisy channels at SNR 0 dB in the frequency-antenna domain.

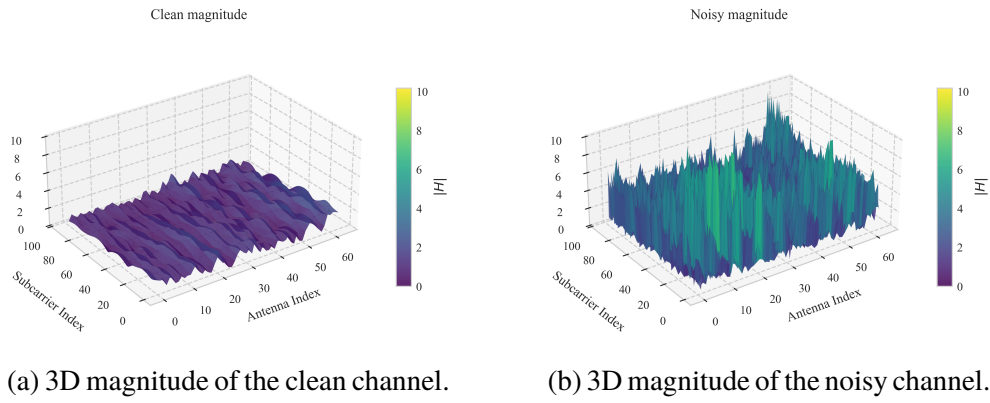


Figure 3.4: 3D magnitude of the clean and noisy channels at SNR 0 dB under the default INR configuration in the frequency-antenna domain.

two transforms convert a diffuse frequency-antenna representation into the more structured delay-beam representation used by the neural models.

Both transformations are linear and invertible, and therefore do not discard information by themselves. The signals become significantly sparser in the delay-beam domain because the dominant multipath components concentrate into a relatively small number of delay and beam bins, while the noise remains more diffuse as shown in Figure 3.5. This sparsity is advantageous for learning because it allows the model to focus on the most relevant physical structure of the channel. The same sparsity is also crucial for the hard-windowing baseline estimator introduced conceptually in Chapter 2. This wideband frequency selectivity is one of the key differences from narrowband channel-estimation settings, which cannot exploit the same delay-domain sparsity [5, 17].

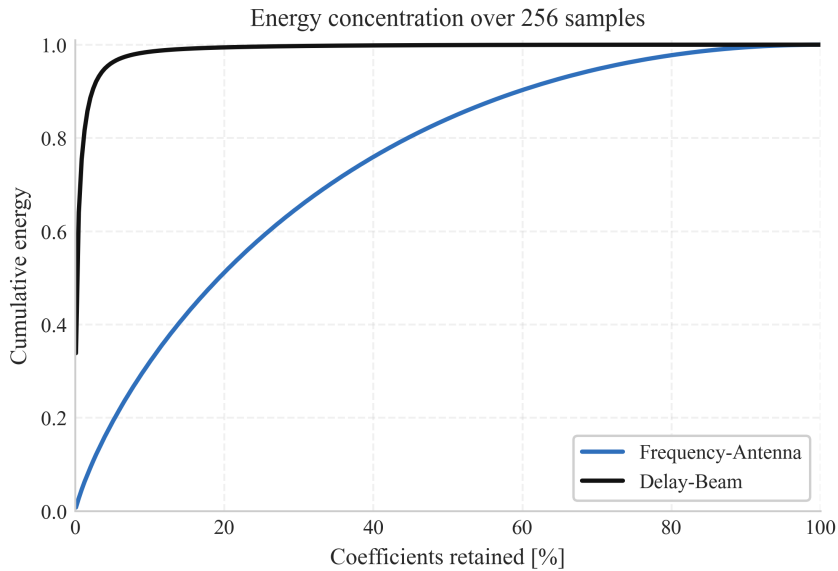


Figure 3.5: Normalized cumulative retained-energy curves computed over 256 representative channel samples in the frequency-antenna and delay-beam domains, plotted against the percentage of coefficients retained after sorting coefficients by magnitude. The y-axis therefore shows the fraction of total sample energy captured by the retained coefficients rather than physical energy units. The delay-beam representation concentrates most of the energy into a much smaller fraction of coefficients.

Due to the periodic nature of the DFT, delay peaks naturally wrap around and may appear at the extreme first and last taps of the delay axis as shown in Figure 2.3. Because these locations correspond to adjacent positions on the underlying circular grid, we apply an `fftshift` operation to move the

wrapped peak structure toward the center of the delay grid. This restores local spatial continuity that may be beneficial for convolutional processing.

Finally, it is important to note how these transformations affect the noise and interference. Because the Fourier transform is an orthogonal operation, strictly white Gaussian noise (AWGN) remains white and Gaussian in the delay-beam domain. The practical value of delay-domain processing comes instead from concentrating the channel energy relative to the broader disturbance floor [5, 17]. The structured interference from other users, however, behaves differently due to the mechanics of pilot contamination in the implemented simulator. During channel estimation, the received interfering signal H_{int} is weighted by the per-subcarrier pilot-overlap vector $\rho = \frac{P_{\text{int}} \odot P_{\text{target}}^*}{|P_{\text{target}}|^2}$ after projection onto the target pilot. Because these pilots are quasi-orthogonal, this pilot-overlap factor induces pseudo-random phase variation across the frequency subcarriers.

When the Inverse Fourier Transform is applied, this frequency-domain phase scrambling prevents the interfering energy from constructively interfering in the same way as an unweighted physical channel. In the simulator interpretation used here, the coherent multipath structure of the interferer is therefore dispersed across the delay bins and appears as structured, colored disturbance rather than as distinct physical peaks. This implementation-level interpretation is consistent with the qualitative delay-beam visualizations below: the neural network sees coherently compressed, sharp peaks from the target signal together with more dispersed energy from pilot contamination.

Figure 3.6 shows clean and noisy 1D slices of the same representative sample as in Figure 3.3, now expressed in the delay-beam domain. In this representation, the noisy slice preserves the dominant support of the clean slice more clearly than in the frequency-antenna domain, even though the disturbance floor remains elevated. The delay-beam domain is therefore used as the model input throughout the remainder of the thesis.

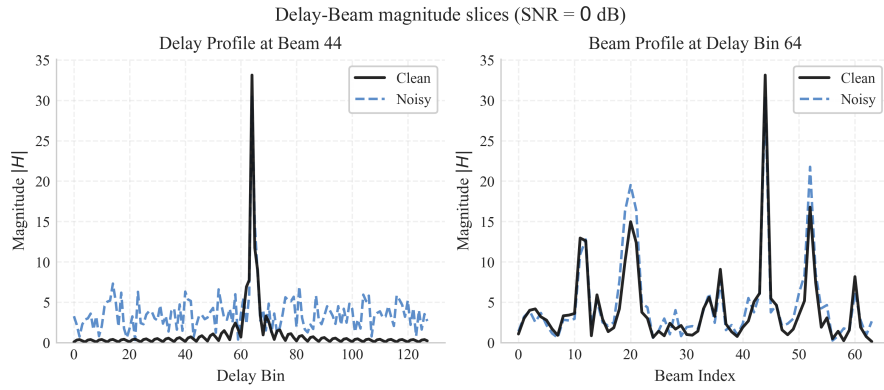


Figure 3.6: Representative 1D magnitude slices of the clean and noisy channels at SNR 0 dB in the delay-beam domain.

To further visualize the range of operating conditions, we compare representative delay-beam slices at -20 dB SNR in Figure 3.7 and Figure 3.8, respectively. At -20 dB SNR, the dominant peaks are barely distinguishable in the noisy slice, whereas at 30 dB SNR the noisy slice much more closely tracks the clean one. The model must therefore operate across a wide range of noise conditions, from extremely low-SNR regimes to comparatively clean observations.

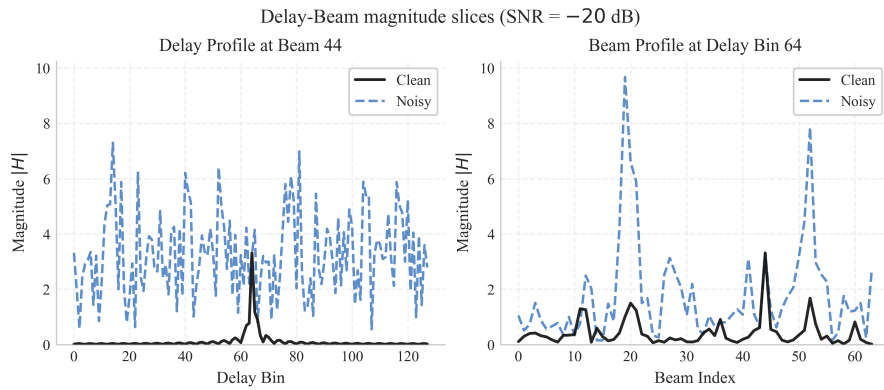


Figure 3.7: Representative 1D magnitude slices of the clean and noisy channels at SNR -20 dB in the delay-beam domain.

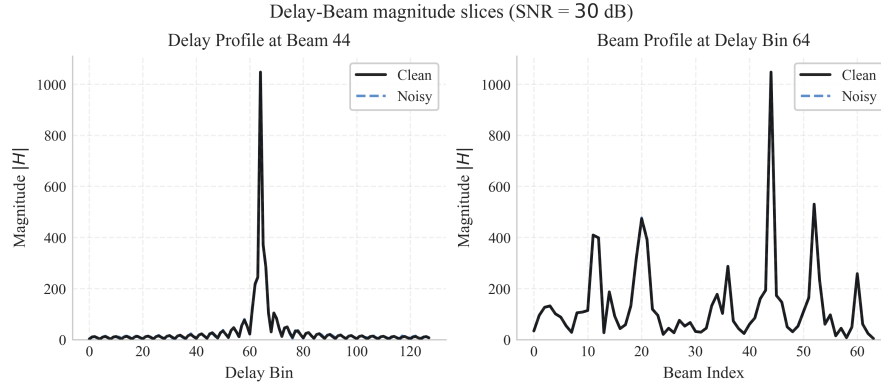


Figure 3.8: Representative 1D magnitude slices of the clean and noisy channels at SNR 30 dB in the delay-beam domain.

Until now, the visualizations have been 1D magnitude plots, but the data fundamentally consists of 2D complex-valued tensors. To further illustrate the structure of the data, Figure 3.9 shows 3D surface visualizations of the magnitude of the same representative signal in the delay-beam domain. In the magnitude view, the clean signal is localized in a small number of delay and beam bins, while the background noise remains broadly spread and the pilot contamination is more diffuse due to the phase scrambling described above. Supplementary phase visualizations are reported in Appendix A, Figure A.3. In the dominant bins, the target phase still reflects the underlying channel geometry, whereas low-magnitude noisy regions are less visually informative because of phase wrapping and disturbance.

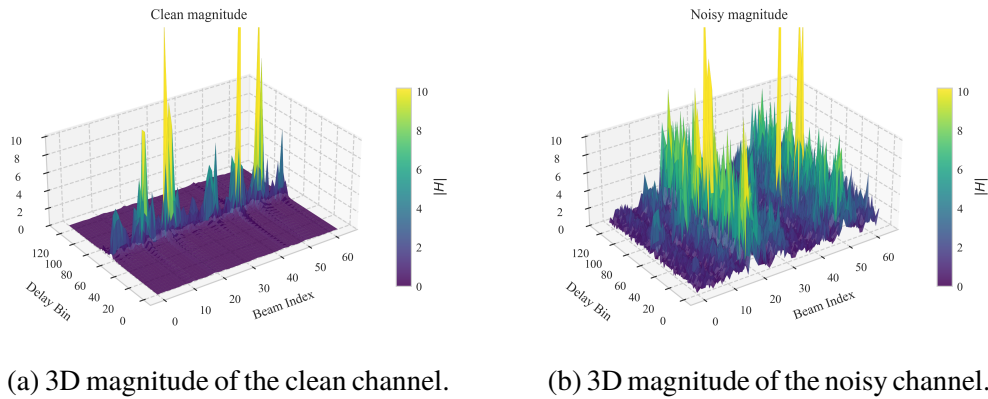


Figure 3.9: 3D magnitude of the clean and noisy channels at SNR 0 dB under the default INR configuration in the delay-beam domain.

Because the data is fully simulated, the data-collection process does not

involve personal data or human participants. However, the model is trained on simulated channels and may therefore not capture all complexities of real-world propagation environments. This can lead to performance degradation when the model is transferred to practical deployments.

3.3 Experimental design and Measurements

3.3.1 Test environment

This subsection specifies the experimental environment required to reproduce the study and to interpret the reported comparisons fairly. All neural estimators are implemented in Python using PyTorch and trained on the available GPU cluster described later in Chapter 4. The experimental test bed consists of the QuaDRiGa-generated training and validation data together with the independent **CDL-B** evaluation set introduced in Section 3.2. All compared estimators use the same underlying simulator setting, pilot-generation configuration, channel and interference realizations, and downstream detection chain, while each estimator follows its own simulator-integrated channel-estimation path.

The learned input-target pairs are generated with the same simulator-aligned transmission chain that is later used in the end-to-end evaluation rather than with a separate ad hoc corruption model. For each sample, the clean channel is scaled according to the sampled operating condition, corrupted by noise and pilot contamination, projected onto the selected **DMRS** symbol, or onto both symbols in the joint-**DMRS** variants, and converted into the noisy **LS** observation used by the network. The scaled supervised target is the corresponding simulator-aligned clean channel, which is referred to simply as the clean channel unless the distinction matters.

To ensure that differences in reported performance are attributable to model design rather than to system-level variation, all comparisons use the same GPU class, software stack, numerical precision mode, random seeds, data splits, and evaluation settings. Detailed package versions, hardware specifications, and hyperparameter settings are deferred to Chapter 4.

The experiments are run sequentially rather than as a single large hyperparameter search. The reference configuration is a one-step denoising flow-matching model with a UNet-Lite backbone and no auxiliary losses unless they are part of the experimental variable. Most experiment groups

therefore vary one design axis at a time, with method-specific deviations only where required. The next subsection describes this convention in more detail.

3.3.2 Model design

The experimental design is organized into six experiment families, labeled A–F for consistent reference across the implementation and result chapters. Families A–E form the main design comparisons: source-state choice, context inputs, flow-model family, backbone, and distillation. Family F is an auxiliary training-distribution study. To keep the main comparison centered on one-step estimators, the default comparison regime uses one-step inference unless step count is itself the experimental variable.

A. Generative vs. denoising formulations

There are two main conditional formulations considered here within the observation-conditioned setting introduced in Section 2.3.1. In the *generative* formulation, the flow starts from a simple prior and receives the noisy observation as side information. In the *denoising* formulation, it starts directly from the noisy observation and learns a transport from corrupted to clean channels. Let x_1 denote the clean target channel, y the projected noisy **LS** observation, and e the source state, so the trajectory runs from e at $t = 0$ to x_1 at $t = 1$. In the denoising case, $e = y$. In the generative case, $e \sim \mathcal{N}(0, I)$ and y is provided separately as conditioning information. Denoising can work well when the observation remains close to the target, but it may generalize less well under stronger corruption or very low-**SNR**. Starting instead from an uncorrelated Gaussian prior avoids anchoring the trajectory to the noisy input, but usually requires learning a more complex transport and can therefore be harder to realize with low-step inference. This comparison tests whether low-step estimation benefits more from an observation-anchored start or from conditional generation from a generic prior.

B. Context inputs

This axis concerns which additional inputs the model receives beyond the core noisy observation, rather than the endpoint conditioning used in **CFM**. The baseline uses no extra context beyond the timestep and noisy observation. Ablations then add the empirical log-standard deviation, explicit **SNR**, and combined **SNR/INR** context to test whether disturbance-aware context improves robustness. The log-standard-deviation term is attractive because it can be computed directly from the noisy observation without extra side information. Approximate **SNR** is also fairly plausible to obtain in practice, since it can often be estimated reasonably well from measured signal power

together with a noise-floor estimate. By contrast, a detailed per-interferer **INR** description is harder to estimate robustly online because it requires separating the total disturbance into individual interference contributions. Hence, the **INR** case is mostly exploratory. This extra context is especially relevant because the study does not use unconditional flow models: once the estimator is anchored primarily on the noisy observation, extra disturbance information may help when interference or noise conditions shift in ways that are not fully identifiable from the observation alone. This is further discussed in Section 3.3.3.

C. Flow-model variants

The compared flow-model families follow the formal definitions introduced in Chapter 2, especially Section 2.3.3, so the purpose here is only to state what is compared experimentally. Unless stated otherwise, the main experiments use the denoising setup described above and vary the learned object or inference rule rather than the surrounding data pipeline. Standard Flow Matching serves as the direct velocity-prediction baseline. Data-parameterized flow predicts the clean endpoint directly. Mean-Flow-style models predict interval-averaged transport. The reported Mean Flow row uses the improved Mean Flow formulation, while AlphaMeanFlow is treated as a scheduled-target ablation within the same family. The reported consistency model is the **iCT**-style endpoint formulation rather than the velocity-space Consistency-FM variant. The formal objectives and notation are not repeated here, and the implementation-specific details are deferred to Chapter 4.

D. Backbone architecture

The default backbone in the study is UNet-Lite, adapted from the U-Net architecture used by [9], because that family had already performed well on a closely related channel-estimation task and offers a compact practical reference. Within the same family, the study also considers a larger U-Net. A super-lightweight UNet-mini variant is then introduced only in the final stage to further test how much the **CMT** distillation setup can recover performance for a more aggressively compact student. Relative to the source architecture, the main thesis-specific changes are the addition of scalar context embeddings for the variables in family B and the use of 2D attention over the delay-beam grid instead of sequence-style 1D attention. As a cross-family check, the study further includes one channel-adapted SiT/DiT-style backbone following [21]. This is intended as a limited benchmark rather than an exhaustive architecture search, so exact layer counts, widths, and parameter counts are deferred to Chapter 4.

E. CMT implementation choice

This group documents how **CMT** is implemented before it is used in the final-model study. The main motivation is that direct one-step training can be unstable, so teacher-trajectory guidance may make optimization more stable and thereby improve final performance. This is especially relevant here because compact backbones are desirable at inference time, so distillation offers one of the few practical ways to transfer information from a stronger training-time model into a student that is still small enough to run easily. More generally, once inference is fixed to one step, there is limited room to recover performance through sampler design alone. Unlike approaches such as Recursive Flow [25], which improve results through additional iterative sampling corrections, this thesis keeps inference simple and instead studies guidance through teacher supervision. The **CMT** stage starts from a frozen standard-flow teacher that generates a raw rollout grid. Unlike the original implementation, this thesis also uses a linearly interpolated version of that rollout. The interpolation is used because the teacher rollout is otherwise relatively coarse. Densifying the stored path provides more candidate intermediate trajectory states before the one-step student is supervised on jumps from sampled teacher states to the teacher endpoint. This design choice is also a consequence of limited compute, since sampling many more teacher steps for every trajectory would be computationally infeasible. Together, these implementation variants test whether trajectory-aligned supervision improves the performance–complexity trade-off of one-step estimators.

F. Training-distribution ablations

These ablations keep the model family fixed and vary only the training **SNR** distribution. They are treated as auxiliary sensitivity studies rather than the main theory-driven or architecture-driven comparisons. Their role is to test a training-distribution shift toward a difficult low-**SNR** operating band and to determine whether such specialization should be carried into the final integrated model. In that sense, the earlier experiment families use broad-range training as the default generalist setting, whereas family F asks whether moving the training distribution closer to the target low-**SNR** evaluation regime yields a better specialized estimator. The motivation is twofold. First, the default model is trained over a broad range of conditions, whereas the focused variants deliberately spend more training mass on challenging low-**SNR** conditions. Second, as discussed earlier in this chapter, the considered **QPSK** data setting is motivated by robustness under very noisy pilot conditions, so prioritizing that low-**SNR** band is also application-motivated rather than only difficulty-motivated. Family F therefore tests whether a low-**SNR**-focused training distribution is more appropriate for the

target application regime, while the broad-range setup serves as the more robustness-oriented reference. The main axes are:

- **Low-SNR-focused training versus the default broad setup:** the shared default setup samples from a broad SNR range, whereas the focused variants restrict more training mass to a difficult low-SNR band.
- **Low-SNR sampler shape:** uniform and truncated-normal sampling change how probability mass is distributed within that low-SNR band, with truncated-normal sampling concentrating more weight around a representative difficult operating point rather than spreading samples uniformly across the interval.

The experiment families are presented in the order A–F because the study is designed as a sequential design-space narrowing rather than as one exhaustive grid over all design combinations. This ordering is methodological as well as practical. Methodologically, the thesis asks which design choices matter most for making one-step learned channel estimation competitive, so the later experiment groups are intended to build on the stronger findings from earlier ones instead of re-testing every possible combination from scratch. Practically, the available compute budget was too limited for a full combinatorial search over source-state choice, context inputs, flow-model family, backbone, distillation setup, training distribution, and DMRS interface.

The sequence also follows the logic of the one-step thesis question. Since the unconditional branch would have shifted the work toward a more iterative inference problem, the study first resolves the main design choices within the conditional one-step setting. Family A addresses the first formulation question: whether the estimator should start from the noisy observation itself or from a generic Gaussian source conditioned on that observation. Family B then tests whether extra disturbance-aware scalar context is useful once the estimator is anchored on the noisy channel. Family C compares the main one-step flow formulations under that shared conditional setting, while family D asks whether the observed limitations are mainly about the learned objective or whether stronger backbone capacity changes the picture materially. Family E is then treated as a training-procedure study rather than a separate model family, since CMT changes how a one-step student is trained rather than how it performs inference. Finally, family F checks whether shifting the training distribution toward the target low-SNR regime improves over the broad-range reference.

In that sense, the experiment order is part of the method rather than only part of the presentation. Earlier families answer broader design questions, middle families compare the main one-step alternatives under a shared setup, and later families study more specialized choices once a stronger base configuration has been identified. This keeps the comparisons interpretable while still allowing the final integrated model to combine the strongest ideas found along the way.

3.3.3 Design Choice: Unconditional vs. Conditional Flow

Preliminary prototyping informed a scoping decision about whether the study should include unconditional flow models in addition to conditional ones. In the unconditional setting, the network learns a global prior over clean channel realizations but does not receive the noisy observation during the transport itself. Inference then becomes an extra search problem: the model must find a channel that both lies on the learned prior and agrees with the measured pilots, typically through iterative **MAP**- or posterior-sampling-style updates. Including that family would therefore have shifted the thesis toward a more iterative inference regime than the main one-step question considered here, so unconditional models were not kept as a main experiment axis.

The study therefore focuses on conditional models. Giving the model the projected noisy channel provides a direct physical anchor, narrows the transport problem to a more local part of the channel space, and makes one-step or low-step inference more plausible. This matters especially in the present setting because the noisy observations do not come from one simple interference model. Instead, they span a broad family of channel-dependent distributions induced by varying **SNR**, **INR**, pilot-contamination patterns, and train–test channel mismatch. Conditioning on the noisy estimate therefore removes much of the burden of learning a global unconditional prior and then solving the inverse problem at inference time.

The drawback is that conditional models can become sensitive to distribution shift when the noisy channels seen at test time differ from those used during training. To study and partly mitigate this, the ablations include both empirical log-standard-deviation conditioning and explicit **SNR/INR** context. These additional variables tell the model more directly which operating regime it is in, so the network does not have to infer that regime from the noisy channel alone. This is expected to improve robustness under moderate out-of-distribution shifts. This does not mean that unconditional

priors are uninteresting, only that they fall outside the scope of the present one-step study.

3.4 Assessing reliability and validity of the data collected

This section summarizes how reliability and validity are assessed, separating what is controlled by the evaluation protocol from what remains limited by simulation, compute budget, and single-run experimental design.

3.4.1 Validity of method

The method is valid to the extent that it can answer the thesis question: whether one-step flow-based estimators can improve practical channel estimation under pilot contamination in the studied setting. Because this is a comparative design question, the method is to evaluate the main one-step alternatives and design choices, such as source-state choice, context inputs, flow-model family, backbone, distillation, and training distribution, within the same simulator-aligned pipeline and against the same practical hard-windowing baseline. This controlled ablation design tests both whether the general approach can be competitive and which modeling choices matter for the hypothesis. End-to-end simulator **SER** and $\text{Gain}_{0,1}$ are therefore the primary evidence, since the hypothesis concerns end-to-end communication performance in a one-step setting. **Normalized Mean Squared Error (NMSE)** and the auxiliary diagnostics are used only to interpret model behavior. The main limitation is that this validity is internal to the studied simulator setting and does not by itself establish correctness under measured real-world channels or outside the tested operating regime.

3.4.2 Reliability of method

Reliability is supported by fixed preprocessing, data splits, hardware platform, software stack, evaluation settings, and validation procedure. Inference is deterministic for a fixed checkpoint and input because stochastic **ODE** solvers are not used. However, most configurations are trained once and with limited hyperparameter tuning, so the ablations should be read as controlled single-run comparisons rather than estimates of mean performance and variance across seeds. Detailed training settings are reported in Chapter 4.

3.4.3 Data validity

The result data support the thesis claims because they include both downstream communication performance and channel reconstruction quality. Simulator **SER** measures the effect of channel estimation on downstream detection, while **NMSE** and diagnostics are computed against known ground truth after transformation back to the frequency-antenna domain. Comparing against the established hard-window baseline further anchors the reported gains in the intended application context. External validity is more limited: the results come from simulation, with a non-public **CDL-B** test set that differs from the QuaDRiGa training distribution but is still not measured real-world channel data. Also, the results are specific to the particular simulator, interference model, and evaluation protocol used here. The main value of the results is therefore in comparing different design choices within this fixed evaluation framework and in demonstrating the attainable potential of the approach in that setting, rather than in making general claims about absolute performance or real-world deployment. The full internal simulator and evaluation pipeline are not public because they belong to an internal proprietary environment. The thesis is therefore intended to describe the data representation, preprocessing chain, evaluation protocol, and reported metrics well enough that an external reader can implement an analogous study in another comparable environment.

3.4.4 Reliability of data

The result data are reliable within the fixed evaluation protocol: all estimators use the same evaluation dataset, interference configuration, **SNR/INR** points, preprocessing, interpolation rules, and simulator pipeline.

3.4.5 Data Analysis

The main design comparisons are carried out on the reported evaluation **SNR** grid at fixed **INR**. Each group is ranked primarily by simulator **SER** and $\text{Gain}_{0.1}$ relative to hard-windowing. Checkpoint selection within a training run uses quick-validation power-weighted **NMSE**, since evaluating full end-to-end simulator **SER** for every saved checkpoint would have been too expensive under the available budget. Because later results show that **NMSE** and **SER** are not perfectly aligned, this checkpoint rule should be read as a practical proxy rather than as a guarantee that every reported checkpoint is **SER**-optimal. Signed channel-gain bias, cosine similarity and **Maximum Mean Discrepancy**

(MMD) are supplementary diagnostics defined in Appendix A.1. After the sequential design study, the strongest settings are combined into a final retrained model. This final model is therefore a best achieved configuration within the explored design space, included to demonstrate the attainable potential of the approach in this setup rather than to serve as a strictly unbiased held-out model-selection result.

3.5 Evaluation Framework

The evaluation framework defines the metrics used for the comparisons summarized above. It combines end-to-end simulator SER with offline reconstruction diagnostics. Although the models operate in the delay-beam domain, all reported reconstruction metrics and simulator results are computed after transforming estimates back to the frequency-antenna domain.

3.5.1 Evaluation metrics

For the n th evaluation sample, let $\hat{H}^{(n)}$ and $H^{(n)}$ denote the predicted and target channels after inverse transformation to the frequency-antenna domain. The primary downstream metric is the Symbol Error Rate,

$$\text{SER} = \frac{N_{\text{err}}}{N_{\text{sym}}}, \quad (3.2)$$

where N_{err} is the number of incorrectly detected symbols and N_{sym} is the total number of detected symbols in the simulator. The simulator pipeline applies the selected SNR/INR scaling, estimates the channel with the method under test, performs the fixed downstream detection steps, and counts symbol errors. All compared estimators use the same downstream detection structure and ideal interference-plus-noise covariance estimate, so the reported SER differences mainly reflect channel-estimation quality. Because the simulator is internal, only the details needed to interpret the metrics are described. SER is evaluated over an SNR sweep at fixed INR and reported per point or through sweep-level summaries.

To compare against hard-windowing at a fixed operating target, the thesis also reports SNR gain at $\text{SER} = 0.1$. For each method, the required SNR is estimated by linear interpolation between the two neighboring simulated SNR points around $\text{SER} = 0.1$, and $\text{Gain}_{0.1}$ is then reported relative to hard-windowing. Positive values therefore mean that the learned estimator reaches

the target SER at a lower SNR than the hard-windowing baseline.

The main offline reconstruction metric is the power-weighted **NMSE**,

$$\text{NMSE}_{\text{pw}} = 10 \log_{10} \left(\frac{\sum_n \|\hat{H}^{(n)} - H^{(n)}\|_2^2}{\sum_n \|H^{(n)}\|_2^2} \right), \quad (3.3)$$

which measures total reconstruction error relative to total target-channel energy over the evaluation set. It is therefore a dataset-level normalized error metric: linear error power and linear target power are first accumulated over the dataset, and only the final ratio is converted to dB. This aggregation is more stable than an unweighted average of per-sample NMSE values, which can be dominated by samples with very small channel energy. Lower values indicate better reconstruction, and negative values mean that the total error power is below the total signal power.

For model-complexity reporting, the thesis also uses **Giga Multiply-Accumulate Operation (GMAC)**. The reported GMAC values are obtained from the `ptflops` library and then corrected for the scaled dot-product attention operations omitted by the profiler, while keeping the standard profiler counts for convolutions and projection layers unchanged. GMAC is used as a hardware-agnostic proxy for arithmetic cost. Supplementary prototype-path runtime measurements are discussed separately in Appendix A.4, since they cover the full channel-estimation module rather than only the neural-network forward pass.

3.6 System documentation

In this project, the thesis documents the study design, evaluation protocol, and main implementation choices, while the operational prototype documentation is distributed across two codebases and the saved experiment artifacts. The estimator codebase covers data handling, training, and offline evaluation, whereas the simulator codebase covers integration of the learned estimator into the end-to-end pipeline. Checkpoints, logs, tables, and simulator outputs provide traceability between a reported result and the configuration that produced it.

Together, the thesis, the codebases, and the saved artifacts are sufficient to reconstruct and evaluate the prototype internally. Because the codebases and simulator are not public, this documentation primarily supports reproducibility within the original environment. For external readers, the role of the thesis

is instead to describe the problem formulation, preprocessing chain, model interfaces, evaluation procedure, and reported metrics clearly enough that an analogous setup can be reproduced in another comparable environment.

Chapter 4

Implementation

This chapter summarizes the implemented environment, preprocessing pipeline, learned estimators, training setup, and simulator integration used to produce the reported results. The theoretical model definitions are given in Chapter 2, while the experimental rationale is given in Chapter 3. This chapter therefore focuses on concrete implementation choices and reproducibility settings.

4.1 Experiment implementation

To keep the experimental discussion compact, Table 4.1 summarizes the shorthand identifiers used for the main experiment groups documented in this chapter. The letter denotes the design axis and the number denotes the specific variant within that axis.

Table 4.1: Compact shorthand naming for the reported experiment groups and auxiliary **CMT** mode variants documented in the implementation chapter.

ID	Variant	ID	Variant
A1	Generative	A2	Denoise
B1	No scalar context	B2	Log-standard-deviation context
B3	Log-standard-deviation + SNR context	B4	Log-standard-deviation + SNR/INR context
C1	Standard FM	C2	Data-parameterized flow
C3	Mean Flow	C4	Consistency
C5	AlphaMeanFlow	D1	UNet-Lite
D2	U-Net	D3	SiT-4M
E1	Aux. CMT grid	E2	Aux. CMT interpolate
F1	Low- SNR -only	F2	Truncated-normal SNR
Final	Integrated CMT and retraining		

The following subsections summarize experiment families A to D and F, the **CMT** implementation choice, and the final integrated configuration.

4.1.1 Implemented preprocessing

Training and evaluation use the simulator-aligned pipeline described in Chapter 3, so only the main implementation choices are summarized here. For each sample, one target channel and four interferer channels are selected using deterministic seeds derived from the sample index. During training, the corresponding pilot realization is sampled on the fly from the stored pilot bank before the same pilot-transmission and **LS** projection logic used by the simulator is applied. During evaluation, the pilot realizations are instead fixed by the test configuration. The desired and interfering channels are then scaled according to the sampled **SNR** and **INR**, and the noisy **LS** observation is generated at batch time. During training, the single-**DMRS** configurations select one of the two available pilot symbols at random, while the later joint-**DMRS** variants keep both and predict them together in one forward pass. Before the neural model is applied, the active pilot-bearing subcarriers are transformed to the delay-beam domain. The single-**DMRS** path uses a two-channel real/imaginary tensor of size $(B, 2, 128, 64)$, while the joint-**DMRS** path stacks both pilot symbols in the same model call. Reported metrics and simulator evaluation are computed after transforming the estimate back to the physical frequency-antenna domain.

4.1.2 A. Flow type

All learned estimators combine a neural backbone with a chosen flow or consistency objective. The flow-type experiment changes the source state from which the learned transport starts and whether the noisy observation is also provided through conditional channels:

- **A1 Generative:** The source state is Gaussian noise. The noisy **LS** observation is supplied separately through real and imaginary conditional channels.
- **A2 Denoise:** The source state is the noisy **LS** observation itself. No additional conditional channels are used.

In the explicit conditioning case, the current flow state is first represented as the usual two-channel real/imaginary delay-beam tensor. The noisy **LS** observation is represented in the same way and concatenated with the current state along the channel axis at the input projection of the neural backbone. Thus, the generative conditional model receives both the current sampled state and the noisy channel observation at the first network stage. In the denoising setup, no such input concatenation is used. The noisy observation conditions the estimator by serving directly as the source state of the transport.

In the sequential workflow of this study, the later experiment groups are instantiated on the denoising line unless an override is stated. The reason this line is carried forward is given with the results discussion rather than in this implementation description.

4.1.3 B. Context inputs

The context-input experiments then vary only the scalar side information appended to the time embedding. Each scalar is embedded with sinusoidal features and a small **Multi-Layer Perceptron (MLP)**. Missing context slots are zero-padded for checkpoint compatibility. The log-standard deviation of the noisy input tensor is used as an empirical disturbance proxy and can be computed directly from the observation. When explicit **SNR** and **INR** context is used, the model receives the simulator operating point in dB. This should be read as an exploratory side-information study: approximate **SNR** is more practically plausible because it can often be estimated reasonably well, whereas detailed interference estimates are harder to obtain, so the **INR** input is summarized by the mean across interferers as a coarse proxy. The implemented variants are:

- **B1 No scalar context:** The model receives only the flow time variable.
- **B2 Log-standard-deviation context:** The model also receives $\log(\text{std}(y))$ computed from the noisy **LS** input.
- **B3 Log-standard-deviation + SNR context:** The current **SNR** value is appended to the log-standard-deviation context.
- **B4 Log-standard-deviation + SNR/INR context:** The current **SNR** value and the mean **INR** value are appended to the log-standard-deviation context.

For continuity with the later experiment sequence, the subsequent experiment groups inherit the **B4** setting unless an override is stated.

4.1.4 C. Flow-model family

These experiments keep the same backbone and data representation but vary the learned target, model-specific hyperparameters, and inference rule. Standard flow, data-parameterized flow, and consistency models use a single time-like input, while Mean-Flow-style models use two time-like inputs (t, h) with $h = t - r$. AlphaMeanFlow shares this Mean-Flow-style interface and differs through its scheduled target, so it is reported as a separate C-series ablation rather than as a separate model family. The mathematical description in Chapter 2 uses forward intervals $r < t$, so the corresponding implementation offset is likewise taken to be $h = t - r \geq 0$ for non-degenerate intervals. Unless a model-specific override is stated, training times are sampled with the shared time-sampling utility. The default training sampler is logit-normal,

$$t = \sigma(\mathcal{N}(-0.4, 1.0)), \quad (4.1)$$

which was adopted from Mean-Flow-related implementations [21] but is used here as the shared default for all reported model families unless an override is stated. Algorithmic pseudocode is collected in Appendix B.1.

The implemented variants are:

- **C1 Standard FM:** The direct velocity-prediction baseline.
- **C2 Data-parameterized flow:** Direct clean-endpoint prediction instead of velocity prediction.
- **C3 Mean Flow:** Interval-averaged transport using the Mean Flow objective and two time-like inputs (t, h) .

- **C4 Consistency:** The **iCT**-style endpoint-consistency model used in the reported one-step comparison.
- **C5 AlphaMeanFlow:** A scheduled-target Mean-Flow-style ablation with annealed α .

Table 4.2 lists only the remaining explicit C-series parameter settings that are not already clear from the model definitions in the surrounding text. Unless otherwise stated, the non-consistency models use the adaptive MSE described in the Mean Flow paper [21], while the consistency model instead uses the pseudo-Huber consistency loss recommended by improved consistency training [13]. Shared optimizer, dataset, and simulator settings remain in the base training table.

Table 4.2: Explicit C-series parameter settings not already specified by the model definitions.

Parameter	Value	Relevant variants
Adaptive reduction exponent	$p = 1$	C1, C2, C3, C5
Adaptive reduction floor	$\epsilon = 10^{-4}$	C1, C2, C3, C5
Time input	t	C1, C2, C4
Time input	$(t, h = t - r)$	C3, C5
Interval-sampling ratio	0.75	C3, C5
Data scale	1.64	C4
Pseudo-Huber scale	10^{-4}	C4
Timestep sampling	lognormal with mean -1.1 and std 2.0	C4
Discrete-level schedule	10 to 1280 levels	C4
α schedule start	about 35% of training	C5
α schedule end	about 65% of training	C5
α schedule steepness	$\gamma = 25$	C5
α clamp	$\eta = 0.005$	C5

All remaining details for these model families follow the corresponding formulations introduced in Chapters 2 and 3. For model-family-specific details not stated explicitly here, the reader is referred to the original papers for each family, which are cited in the relevant sections of Chapter 2.

4.1.5 D. Backbone comparison

These experiments keep the model family and training setup fixed and change only the neural backbone. The detailed layer structure is described later in this chapter.

- **D1 UNet-Lite:** The default lightweight convolutional backbone.
- **D2 UNet:** A larger convolutional reference with higher width, depth, and attention coverage.
- **D3 SiT-4M:** The transformer-based alternative used in the reported backbone comparison.

UNet-mini is introduced later in the joint-DMRS and final-model stage. It tests the CMT setup on a more aggressively inference-cost-oriented student and is not part of the strict D-series benchmark.

4.1.6 E. CMT implementation

CMT is used in this thesis as a teacher-student stage for the later final-model study rather than as a standalone main-axis result family. The CMT stage starts from a frozen standard-flow teacher checkpoint, rolls it out for 10 Euler steps, converts the resulting teacher states into an interpolated trajectory of 40 points, and supervises the student during the CMT stage. Afterward, the student passes through a post-CMT retraining stage before evaluation.

The standard-flow teacher and the chosen one-step student do not necessarily share the same output parameterization. The teacher velocity field is used only to numerically roll out channel states. The student is then trained against the teacher endpoint rather than against the teacher’s raw velocity output. For a consistency student, this means that the student still does not learn an explicit velocity field, but it is supervised on intermediate states sampled from a teacher-generated flow-trajectory along the same transport path. CMT therefore injects flow information through supervision while keeping the final student as a one-step endpoint model at inference time.

An auxiliary grid-versus-interpolate comparison is reported later in Table A.10. Unless otherwise stated, the implemented CMT pipeline uses the interpolated representation as its default mode in the final-model study.

The only additional implementation setting worth stating here is that the CMT stage and the post-CMT retraining stage use a dedicated student-stage learning-rate override of 2×10^{-3} rather than the base default.

Figure 4.1 summarizes this teacher-student procedure, and CMT changes how the student checkpoint is produced but not how it is inserted into the simulator. After the distillation and retraining stages, the student is evaluated with the same sampler and custom simulator adapter as the non-distilled models. In the main results narrative, CMT is therefore evaluated through the

final integrated model rather than through the auxiliary grid/interpolate mode comparison by itself.

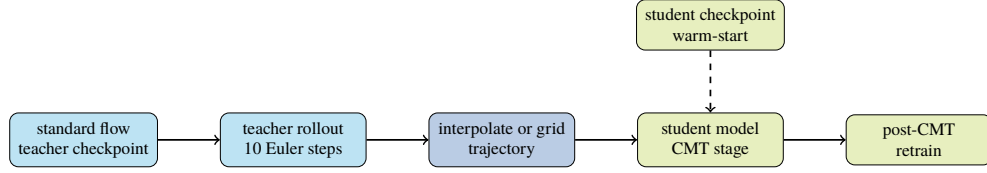


Figure 4.1: Implemented **CMT** pipeline. A standard-flow teacher checkpoint is rolled out with 10 Euler steps to produce either a grid or interpolated trajectory representation. The reported one-step **CMT** runs use this trajectory to form beginning-to-end teacher supervision from the noisy start to the teacher endpoint. The dashed connection indicates an optional student checkpoint or warm-start used to initialize the **CMT** stage before post-**CMT** retraining.

The objective definitions for both consistency-style and Mean-Flow-style **CMT** are given in Section 2.3.4.

4.1.7 F. Training-distribution ablations

These experiments keep the reference denoising setup fixed and change only the training distribution. The shared broad-range base configuration is documented later in Table 4.4. The F-series variants replace only the **SNR** sampler and range unless another override is stated.

The implementation supports two **SNR** samplers. With uniform sampling, the **SNR** is drawn uniformly from the chosen interval. With truncated-normal sampling, samples are drawn from a normal distribution with a chosen mean and standard deviation and then truncated to the same interval. Thus, the truncated-normal setting still respects the chosen low-**SNR** bounds, but places more probability mass around a selected challenging operating point within that band rather than at the extreme lowest-**SNR** tail.

- **F1 Low-**SNR**-only:** Uniform **SNR** sampling over $[-20, 0]$ dB, with the remaining settings inherited from the base configuration unless explicitly changed.
- **F2 Truncated-normal **SNR**:** Truncated-normal **SNR** sampling over $[-20, 0]$ dB with mean -10 dB and standard deviation 3 dB, again inheriting the base configuration otherwise.

The auxiliary-loss definitions and exploratory sweep results are kept in Appendix A.2, since they are supporting experiments rather than one of the main design axes summarized above.

4.1.8 G. Final model implementation

After the sequential main design comparisons, selected settings are combined into final-stage integrated models. This subsection documents those integrated checkpoint forms rather than another one-axis ablation. The warm-start and distillation comparisons are reported only in this final integrated stage rather than as separate E-series experiments.

The final-stage models are reported in two backbone forms:

- **UNet-Lite form:** A focused final consistency line built on the UNet-Lite backbone, using A2 denoising source-state initialization, the B2 log-standard-deviation scalar context, C4 consistency, and the F2 focused low-SNR truncated-normal SNR training distribution.
- **UNet-mini form:** A corresponding focused final line built on the UNet-mini backbone, again using A2, B2, C4, and F2, and then evaluated through the final-stage warm-start and interpolated CMT branches. In this branch, CMT is followed by a 5-epoch final student retraining stage so that both the distilled checkpoint and its retrained version can be evaluated.

Within this final integrated stage, the UNet-mini branch keeps the same B2 scalar context and the same focused low-SNR training regime, but compares direct consistency training from a warm-started student checkpoint against the teacher-distillation route. This comparison is included so that the final integrated stage isolates checkpoint construction rather than context, backbone, or training-distribution changes. The reason for applying CMT only to the smaller UNet-mini branch is discussed later with the result interpretation.

As a final control in this integrated stage, the study also includes a no-flow backbone-only estimator. This control uses the same joint-DMRS data representation and the same backbone family as the learned flow-based models, but predicts the channel directly without a flow or consistency objective. Its purpose is to isolate how much of the final performance comes from the backbone itself and how much comes from the flow-style training objective.

4.2 Backbone implementations

The backbone study compares a convolutional U-Net family adapted from the channel-estimation model in [9] with a transformer-based SiT family adapted from the SiT-style backbone used in [21]. All reported estimators operate on the same real-valued delay-beam tensor of shape $[B, 2, 128, 64]$, where the channel dimension stores real and imaginary parts. The backbone is therefore always a two-dimensional operator over delay and beam indices, while the selected model family determines whether the output is interpreted as a velocity field, an interval-averaged velocity, or a clean endpoint. Time and optional context scalars are embedded with the shared sinusoidal-MLP pipeline, and missing context slots are zero-padded so that closely related checkpoints remain compatible. Because these backbones are adapted rather than proposed as new architectures in this thesis, the discussion below focuses on the concrete implementation choices that affect reproduction and interpretation of the reported results, while the cited papers remain the primary reference for the original designs.

Table 4.3 summarizes the backbone variants documented for the reported experiments.

Table 4.3: Documented neural backbone variants used in the reported experiments.

Family	Variant	Base width / hidden size	Depth	Attention configuration	Parameters
U-Net	UNet-mini	8	1 resblock/level, [1, 1, 2]	middle self-attention only	0.049M
	UNet-Lite	16	1 resblock/level, [1, 1, 2]	middle self-attention only	0.193M
	U-Net	32	2 resblocks/level, [1, 2, 4]	resolution 32 + middle	3.952M
SiT	SiT-4M	192	9 transformer blocks	4 heads, patch size 4	4.137M

4.2.1 U-Net family

The U-Net family follows the diffusion-style encoder-decoder design used in [9]. It is documented in more detail than the appendix-only SiT benchmark because it is the primary backbone family used in the reported main design comparisons and the final integrated model.

Architecture layout. An input convolution maps the two-channel delay-beam tensor to the base width, the downsampling path applies residual blocks and optional attention, the bottleneck contains a residual-attention-residual block, and the upsampling path reconstructs the output with skip connections.

Residual blocks and conditioning. Each residual block uses a pre-activation layout: GroupNorm, SiLU/Swish nonlinearity, a 3×3 convolution, additive injection of the shared time/context embedding, a second GroupNorm followed by SiLU, and a final 3×3 convolution. If the channel dimension changes, the shortcut uses a learned projection. Otherwise it is the identity. The second convolution in each residual block and the final output head use zero-scale initialization so that training starts with weak residual corrections near an identity map. When conditional channels are used, the noisy observation is concatenated only at the input. The time/context embedding is shared across all blocks and is formed by first embedding each scalar with sinusoidal features and a small **MLP** and then summing the resulting vectors. This shared embedding is added to the activations in each residual block after the first convolution, so it conditions the block’s main processing path rather than the input projection.

Attention change relative to the source model. The original architecture in [9] did not use the 2D attention treatment adopted here. In this thesis, self-attention operates directly over the delay-beam grid with GroupNorm pre-normalization, learned 1×1 query/key/value projections, and single-head scaled dot-product attention over the flattened delay-beam tokens. A cached 2D sine-cosine positional encoding is added before the attention projections so the block receives explicit delay and beam location information, and the attention output projection is zero-initialized for stable residual learning.

Reported variants. The UNet-Lite and standard U-Net variants are the two paper-derived configurations carried over as the main convolutional reference settings, while UNet-mini is a thesis-specific reduced variant introduced later for the distillation and inference-cost study. The super-lightweight UNet-mini variant uses base width 8, channel multipliers [1, 1, 2], one residual block per level, GroupNorm with 8 groups, and only the shared middle self-attention block. The default lightweight configuration is UNet-Lite, which uses base width 16 with the same overall layout and the same 8-group normalization. The larger U-Net variant uses base width 32, two residual blocks per level, channel multipliers [1, 2, 4], 32-group normalization, and attention at resolution 32 in addition to the middle block. Relative to [9], the main thesis-specific changes are the scalar-context pathway and the 2D attention treatment over the delay-beam grid.

Figure 4.2 summarizes the implemented U-Net data flow.

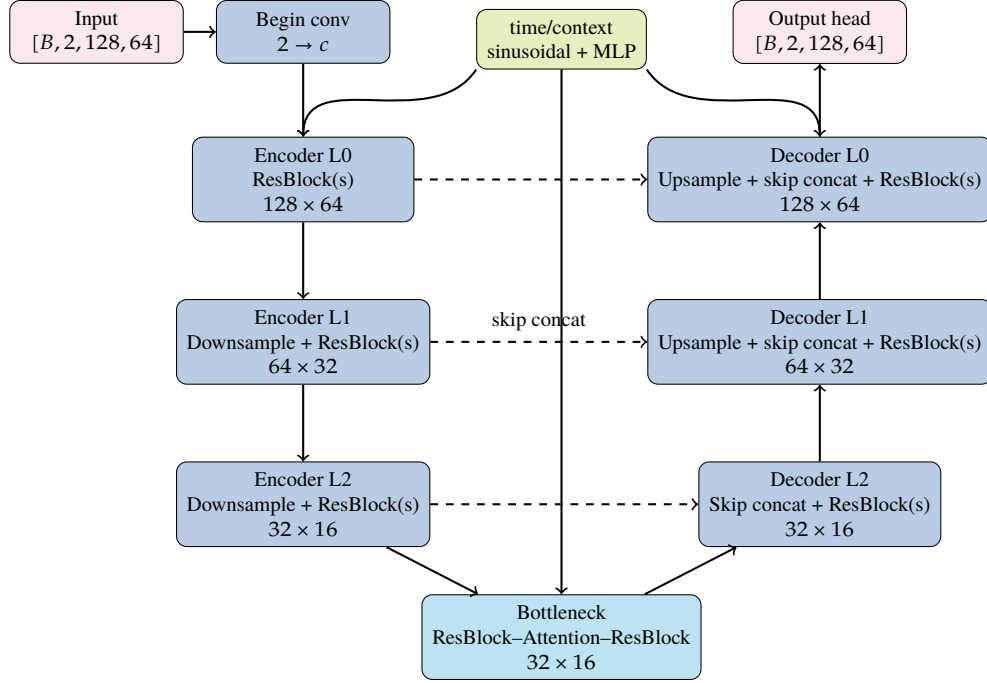


Figure 4.2: Schematic U-Net-style backbone. The architecture is organized as encoder, bottleneck, and decoder stages over the delay-beam grid. Dashed arrows denote skip concatenations, and the time/context embedding is shown schematically rather than repeated for every residual stage. UNet-mini uses $c = 8$, one residual block per level, channel multipliers $[1, 1, 2]$, and only the middle attention block. UNet-Lite uses the same layout with $c = 16$. The standard U-Net uses $c = 32$, two residual blocks per level, multipliers $[1, 2, 4]$, and attention at resolution 32 in addition to the middle block.

4.2.2 SiT transformer family

The transformer alternative is a channel-adapted Scalable Interpolant Transformer following the SiT-style backbone used in [21]. In the reported comparison, this family is represented by SiT-4M, which patchifies the 128×64 delay-beam grid into $32 \times 16 = 512$ spatial tokens, prepends four learned time/context prefix tokens, processes the sequence with transformer blocks, and projects the tokens back to the dense output grid. Internally, each transformer block uses RMSNorm, rotary positional embeddings on the query/key pairs, and a SwiGLU feed-forward sublayer, with zero-initialized residual scaling on both the attention and MLP branches so the network starts near a weak residual update. The final patch-to-output projection is likewise zero-initialized. Because SiT is included only as a secondary backbone

benchmark rather than the default architecture, the detailed schematic and lower-level block description are deferred to Appendix A.7, Figure A.1.

4.3 Training pipeline and experiment management

This section summarizes the concrete environment, shared configuration presets, and training-specific implementation details used to produce the reported experiments.

4.3.1 Software and hardware environment

The flow-model implementation uses Python 3.11 together with PyTorch 2.6.0+cu126, Hugging Face Accelerate 1.13.0, NumPy 2.4.4 and bitsandbytes 0.49.2 for data processing, model training, and offline evaluation. The learned estimator is trained and analyzed offline in this flow-model codebase and is then inserted into a separate TDD uplink simulator codebase for end-to-end SER evaluation. This separation keeps the learning pipeline and the simulator-facing evaluation interface fixed across the reported comparisons.

The experiments were run on a limited GPU cluster with NVIDIA 2080 Ti cards with 11 GB of VRAM. Training typically used at most two GPUs, whereas inference was performed on a single GPU. Throughout the work, 16-bit Floating Point (FP16) mixed precision and the PyTorch Inductor compiler were used to reduce memory pressure and improve throughput. These hardware constraints influenced the choice of lightweight backbones, modest epoch budgets, and the emphasis on one-step inference given the limited available compute.

4.3.2 Configuration presets

Each training run is defined by a common set of settings that selects the backbone, model family, checkpoint initialization, and optimizer loop. Unless a subsection or result table states an override, the reported experiments inherit the remaining shared settings in Table 4.4, while model-design choices such as source-state initialization, scalar context, flow family, and backbone are inherited from the carried-forward experiment line defined in the earlier

subsections. This convention keeps the ablations interpretable by separating shared optimization settings from the specific design choice under test.

The shared training configuration is intentionally modest because the study was run on constrained hardware and because the target use case is simple one-step inference under practical compute constraints. The earliest reference run uses an observation-anchored denoising flow with the lightweight UNet-Lite backbone, but later experiment groups intentionally change the scalar context, flow family, backbone, or training distribution as documented above. During training, **SNR** and **INR** are sampled at batch time by the physics-aware collator, so the same stored clean channels can generate many noisy operating points. Table 4.4 therefore lists only the remaining shared optimization and data settings that are not already specified in the experiment-family subsections above.

Table 4.4: Remaining shared base training settings not already specified above.

Parameter	Value
SNR sampling	Uniform $[-20, 30]$ dB
INR sampling	Uniform per interferer $[0, 20]$ dB
Validation split	0.01
Split seed	42
Effective batch size	64 for all runs, reached with gradient accumulation where needed
Epochs	3
Optimizer	8-bit AdamW
Learning rate	5×10^{-4}
Weight decay	10^{-4}
Scheduler	Warmup 1%, then constant learning rate
Gradient clip	1.0
Gradient checkpointing	Enabled
Exponential Moving Average (EMA) decays	0.99, 0.999
EMA update interval	Every 32 steps
Quick-validation metric	Frequency-antenna NMSE

The evaluation settings are kept fixed across the reported comparisons. The main reported setup uses the **SNR** sweep $[-14, -12, -10, -8, -6]$ dB and **INR** vector $[5.229, 5.229, 5.229, 1.0914]$ dB with batch size 10.

Across this evaluation sweep, all compared estimators use the same

downstream **LMMSE** detector and ideal configuration so that the comparison isolates the channel-estimation module. In all main automated comparisons, only the channel-estimation module is changed. The downstream **LMMSE** detector and its ideal interference-plus-noise covariance input $\hat{R}_{uu}$ are kept fixed. The reported outputs are the per-**SNR SER** curves and the interpolated $\text{Gain}_{0.1}$ relative to hard-windowing. For inference, the default sampler is deterministic and the default inference-cost-oriented setting is one model step, unless otherwise stated.

4.3.3 EMA checkpoint selection

A practical implementation detail is the use of **EMA** checkpoints. An **EMA** model is a smoothed copy of the online model weights, updated as

$$\bar{\theta} \leftarrow \gamma \bar{\theta} + (1 - \gamma)\theta, \quad (4.2)$$

where θ denotes the current online weights and γ is the decay factor. The implementation keeps CPU-side **EMA** copies with $\gamma = 0.99$ and $\gamma = 0.999$, updated every 32 optimizer steps. These smoothed checkpoints reduce the effect of short-term training noise and are especially useful here because one-step Mean Flow and consistency training can be quite unstable on the available compute budget. The final checkpoint is selected by the lowest quick-validation **NMSE** after transforming predictions back to the frequency-antenna domain.

The quick-validation score is computed on a fixed validation split using the same physics-aware collator in deterministic evaluation mode, so checkpoint comparisons are not confounded by changing disturbance realizations.

4.4 Baseline implementations

In addition to the proposed flow-matching models, the reported experiments use two reference channel estimators: the hard-windowing baseline and an ideal estimator. Hard-windowing is the practical classical baseline used throughout the thesis, while the ideal estimator passes the true channel directly to the downstream detection pipeline and therefore serves only as an upper bound and a sanity check for the evaluation pipeline. Standalone **LS** or covariance-based **LMMSE** channel-estimation baselines are not part of the reported comparison set. For transparency, the baseline table still includes one raw **LS** row as a non-competing lower-bound sanity check, but it is not

treated as a practical reported baseline in the later model comparisons. The comparison instead asks whether replacing only the channel-estimation block with a learned estimator improves end-to-end communication performance when the downstream detector and covariance input are kept fixed.

The hard-windowing implementation is inserted as the channel-estimation module in the TDD uplink pipeline, followed by the same downstream **LMMSE** detector used for the learned estimators. It starts from the projected **LS** estimate for the selected Comb-2 pilot symbol, keeps only the occupied pilot subcarriers, and transforms that estimate to the delay domain. The core hard-window step then suppresses the middle of the circular delay grid and retains the two edge regions, which in this setting contain the wrapped low-delay support associated with the desired user.

After this first support-selection step, the simulator estimates an effective pre-**Signal-to-Interference-plus-Noise Ratio (SINR)** from the configured **SNR**, the interferer powers, and the comb-collision pattern. That value is then used to select one of the simulator's delay-domain denoising windows for the chosen multipath profile. The filtered pilot-subcarrier estimate is then mapped back to the simulator's full 192-subcarrier frequency-antenna grid by restoring the observed comb positions and linearly interpolating the missing subcarriers. In short, the reported hard-windowing baseline is simulator-integrated delay-domain support selection followed by profile-matched, **SINR**-adapted denoising.

The ideal estimator is implemented by passing the true simulator channel directly to the evaluation pipeline in place of an estimated channel. In the offline reconstruction diagnostics this yields the target channel itself, and in the end-to-end simulator it provides the performance ceiling corresponding to perfect channel state information. This estimator is not deployment-realistic and is therefore not treated as a practical competing baseline. Its role is instead to verify that the metric and simulator stack behave as expected and to contextualize the remaining gap between hard-windowing and the learned estimators.

Chapter 5

Results and Analysis

This chapter reports the results following the experiment structure introduced in Chapter 3, followed by the final integrated comparison. The main ranking criterion is end-to-end **SER** against hard-windowing, while **NMSE** and the supplementary diagnostics provide supporting context; additional supporting ablations are collected in Appendix A.

5.1 Major results

In Table 5.1, the raw **LS** row is included only as a lower-bound sanity check. Its average SER is 0.3184 across the evaluation grid, so it mainly confirms that the reported learned estimators are doing more than simply forwarding the projected observation. The practical classical baseline is still hard-windowing, which reduces the average SER to 0.1626, while the ideal estimator reaches 0.0686. The ideal row corresponds to a gain of 2.58 dB at $\text{SER} = 0.1$, meaning that the ideal estimator requires substantially less SNR than hard-windowing to reach the same error rate. Interpreted in linear SNR terms, this means about 45% less SNR at the target operating point. As a sanity check, the ideal estimator also achieves an NMSE of -120 dB, confirming that the evaluation pipeline correctly identifies perfect channel estimates.

Table 5.1: Classical and reference simulator baselines. Per-**SNR** SER values are reported across the evaluation grid, together with $\text{Gain}_{0.1}$ (the SNR advantage over hard-windowing at $\text{SER} = 0.1$, where positive means less SNR is required) and **NMSE**.

ID	Method	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		−14	−12	−10	−8	−6		
–	Raw LS	0.5639	0.4603	0.3229	0.1774	0.0676	−2.79	14.30
–	Hard-windowing (HW)	0.3851	0.2496	0.1256	0.0436	0.0092	0.00	6.70
–	Ideal	0.1834	0.1013	0.0434	0.0128	0.0023	2.58	−120.00

For experiment group A, Table 5.2 addresses the first major flow-matching design decision: whether the one-step estimator should start from the noisy observation itself or from a Gaussian source state conditioned on that observation. Neither learned formulation beats hard-windowing at this stage, so the table should be read as an internal model-selection step rather than as evidence of practical improvement over the classical baseline. Within that comparison, the denoising formulation is clearly better at every operating point and is therefore carried forward.

Table 5.2: Generative versus denoising task formulation for one-step standard FM with the UNet-Lite backbone. Per-**SNR** SER, $\text{Gain}_{0.1}$, and **NMSE** are reported across the evaluation grid.

ID	Task formulation	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		−14	−12	−10	−8	−6		
A2	Denoising	0.4125	0.2685	0.1380	0.0508	0.0118	−0.25	−3.59
A1	Generative	0.4453	0.2934	0.1526	0.0565	0.0131	−0.47	−3.09

For experiment group B, Table 5.3 studies whether additional scalar side information improves the denoising baseline. The empirical log-standard-deviation context gives the best **SER** in this group, while adding explicit **SNR** or combined **SNR/INR** context does not improve the default evaluation configuration. Several subsequent ablation groups were nevertheless run with the **SNR/INR**-aware context configuration.

Table 5.3: Context-embedding ablation. Per-**SNR** SER, $\text{Gain}_{0.1}$, and **NMSE** are reported across the evaluation grid. $\text{Gain}_{0.1}$ is measured against hard-windowing using the simulator interpolation convention.

ID	Context	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		−14	−12	−10	−8	−6		
B1	None	0.4028	0.2624	0.1346	0.0499	0.0116	−0.19	−3.60
B2	logstd	0.3992	0.2578	0.1329	0.0487	0.0111	−0.16	−3.63
B3	SNR + logstd	0.4154	0.2687	0.1363	0.0494	0.0113	−0.21	−3.58
B4	INR + SNR + logstd	0.4192	0.2690	0.1349	0.0481	0.0107	−0.18	−3.62

For experiment group C, Table 5.4 compares the main one-step flow formulations against the standard-FM baseline under the shared evaluation setup. An important pattern is that **NMSE** and end-to-end **SER** do not always correlate perfectly: the five-step standard-flow row gives the strongest reconstruction metric in this table, but the one-step consistency model gives the strongest communication result. Among the one-step methods, the consistency model gives the strongest **SER** and $\text{Gain}_{0.1}$ in this table. Mean Flow and AlphaMeanFlow both improve over one-step standard **FM**, while the five-step standard **FM** row recovers much of the performance gap.

Table 5.4: One-step and few-step flow model comparison. Per-**SNR** SER, $\text{Gain}_{0.1}$, and **NMSE** are reported across the evaluation grid.

ID	Model	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		−14	−12	−10	−8	−6		
C1	Standard FM (1 step)	0.4125	0.2685	0.1380	0.0508	0.0118	−0.25	−3.59
C1	Standard FM (5 steps)	0.3574	0.2249	0.1113	0.0389	0.0086	0.31	−3.94
C3	Mean Flow (1 step)	0.3719	0.2361	0.1169	0.0402	0.0088	0.18	−3.91
C4	Consistency (1 step)	0.3529	0.2173	0.1040	0.0354	0.0074	0.51	−2.83
C2	Data-param. flow (1 step)	0.3778	0.2503	0.1297	0.0471	0.0106	−0.10	−3.91
C5	AlphaMeanFlow (1 step)	0.3648	0.2285	0.1115	0.0391	0.0085	0.31	−3.94

For experiment group D, Table 5.5 isolates the strict standard-flow backbone comparison. Moving from the 0.2M-parameter UNet-Lite to the standard 4M-parameter U-Net improves one-step **SER** and **NMSE** consistently across the reported grid, while the SiT-4M result is competitive but does not exceed the standard U-Net on this benchmark.

Table 5.5: Backbone comparison. Per-**SNR** SER, $\text{Gain}_{0.1}$, **NMSE**, and parameter count are reported for the standard-flow backbone variants.

ID	Backbone	Params (M)	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
			-14	-12	-10	-8	-6		
D1	UNet-Lite	0.195	0.4125	0.2685	0.1380	0.0508	0.0118	-0.25	-3.59
D3	SiT-4M	4.137	0.3739	0.2379	0.1191	0.0425	0.0094	0.13	-4.22
D2	UNet	3.952	0.3554	0.2256	0.1129	0.0403	0.0091	0.27	-4.36

For experiment group F, Table 5.6 reports the auxiliary training-distribution study with the architecture and model family held fixed. Compared with the broad-range reference, both low-**SNR**-focused variants improve all three main metrics, and the truncated-normal sampler gives the strongest result in this group. It is also the first standard-flow UNet-Lite configuration to cross slightly above hard-windowing, with positive $\text{Gain}_{0.1}$.

Table 5.6: Focused low-**SNR** training distribution ablation for one-step standard FM with the UNet-Lite backbone. The standard row is included as the wide-range reference. Per-**SNR** SER, $\text{Gain}_{0.1}$, and **NMSE** are reported across the evaluation grid.

ID	Training SNR distribution	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		-14	-12	-10	-8	-6		
-	Standard wide-range	0.4125	0.2685	0.1380	0.0508	0.0118	-0.25	-3.59
F2	Truncnorm on $[-20, 0]$ dB	0.3740	0.2417	0.1232	0.0452	0.0105	0.03	-3.86
F1	Uniform on $[-20, 0]$ dB	0.3958	0.2587	0.1334	0.0492	0.0112	-0.17	-3.68

Table 5.7 gathers the selected final integrated joint-**DMRS** runs reported in the main results narrative. The choice of the joint-**DMRS** interface follows the evaluation setup described in Chapter 3. The U-Net teacher is included in both one-step and 10-step form, with the D-series motivating that backbone as the strongest accuracy-oriented teacher candidate among the tested standard-**FM** models. The table also includes one-step no-flow, standard-**FM**, consistency, and **CMT** rows so that backbone size, objective choice, and teacher-guided checkpoint construction can be compared within the same joint-**DMRS** interface.

The 10-step teacher gives the strongest overall result in the table, while the one-step teacher already provides a strong deployment-style reference. The no-flow UNet-Lite row edges above hard-windowing, but it remains weaker than the better flow-based UNet-Lite rows. At matched backbone size, consistency outperforms standard **FM** for both UNet-Lite and UNet-mini. Within the UNet-mini branch, the **CMT** row improves further over both the

plain consistency and standard-FM baselines, while the retrained CMT row degrades slightly.

Table 5.7: Selected final integrated joint-DMRS runs. Per-SNR SER, $\text{Gain}_{0.1}$, and NMSE are reported across the evaluation grid. The no-flow/standard rows and the consistency/CMT rows are separated for readability, and the best values within each section are shown in bold. Detailed training schedules are documented in Chapter 4.

Variant	SER at SNR (dB) ↓					Gain _{0.1} (dB) ↑	NMSE _{pw} (dB) ↓
	−14	−12	−10	−8	−6		
<i>No-flow / standard</i>							
U-Net teacher (1 step)	0.3319	0.2060	0.1004	0.0353	0.0078	0.61	−4.89
U-Net teacher (10 steps)	0.3118	0.1909	0.0910	0.0308	0.0065	0.80	−4.67
No-flow UNet-Lite	0.3626	0.2362	0.1205	0.0440	0.0101	0.09	−4.06
UNet-Lite standard FM	0.3673	0.2316	0.1138	0.0396	0.0088	0.25	−4.32
UNet-mini standard FM	0.4006	0.2575	0.1293	0.0461	0.0100	−0.08	−3.67
<i>Consistency / CMT</i>							
UNet-Lite consistency	0.3399	0.2091	0.1001	0.0338	0.0071	0.62	−3.09
UNet-mini consistency	0.3520	0.2171	0.1042	0.0352	0.0074	0.50	−2.49
CMT , U-Net teacher / UNet-mini student	0.3319	0.2043	0.0980	0.0333	0.0070	0.66	−3.85
retrained CMT , UNet-mini student	0.3431	0.2107	0.1009	0.0342	0.0073	0.60	−2.82

Figure 5.1 complements Table 5.7 with a direct visual comparison between the strongest reported one-step learned estimator, hard-windowing, and the ideal upper bound across the reported SNR grid. The gap to the ideal estimator remains large, but the distilled UNet-mini row consistently improves over hard-windowing at every reported point.

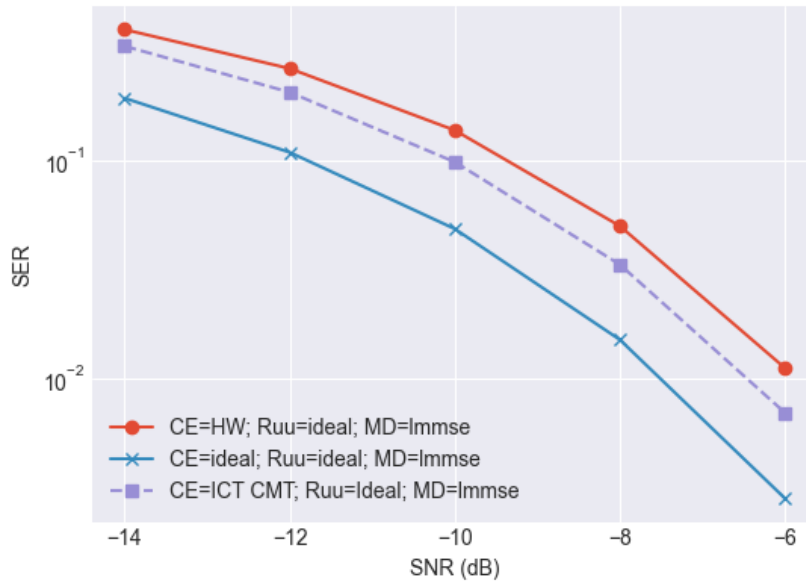


Figure 5.1: End-to-end **SER** versus **SNR** for hard-windowing, the ideal upper-bound estimator, and the strongest reported one-step learned estimator from Table 5.7, namely the **CMT**-distilled UNet-mini. In the legend, CE denotes the channel-estimation module, R_{uu} denotes the covariance estimate used in the detector, and MD=lmmse indicates that the final symbol-estimation stage uses an **LMMSE** detector.

Table 5.8 complements the end-to-end results with a hardware-agnostic complexity view of the backbones used in the joint-**DMRS** comparisons. GMAC is reported here in the sense defined in Section 3.5.1. The arithmetic-cost gap between the three U-Net sizes is large: the standard U-Net is reported at 4.68 GMAC, UNet-Lite at 0.33 GMAC, and UNet-mini at only 0.089 GMAC. For scale, the original ResNet paper reports about 1.8 GMAC and 3.6 GMAC for ImageNet ResNet-18 and ResNet-34, while MobileNetV2 reports about 0.3 GMAC [36, 37]. Under that convention, UNet-Lite is in roughly the MobileNetV2 range, UNet-mini is substantially smaller, and the full U-Net is somewhat above ResNet-34. Because those published reference numbers come from different tasks and input resolutions, they should be read only as rough scale anchors rather than direct runtime comparisons. The broader deployment interpretation of these complexity differences is taken up in Chapter 6.

Table 5.8: Model complexity for the joint-**DMRS** backbones used in the main comparisons, measured with `ptflops` and corrected for the scaled dot-product attention operations omitted by the profiler.

Model	Params (M)	ptflops	Attn. corr.	Reported GMAC
UNet	3.945	4.3079	0.3710	4.6789
UNet-Lite	0.193	0.3211	0.0094	0.3305
UNet-mini	0.049	0.0845	0.0047	0.0892
SiT-4M	4.100	2.0695	0.9177	2.9872

Figure 5.2 combines that complexity proxy with $\text{Gain}_{0.1}$ for selected final neural models. The 10-step teacher remains the strongest accuracy point, but it does so at a much higher approximate channel-estimator cost. The distilled UNet-mini student lies much further to the low-cost side while retaining most of that gain, which makes the one-step deployment trade-off more explicit.

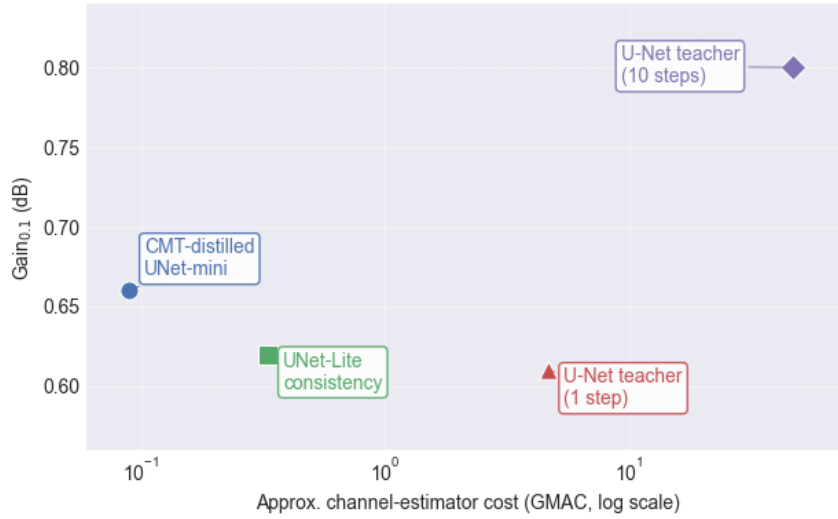


Figure 5.2: Accuracy-cost trade-off for selected final neural models from Table 5.7. The x-axis uses the reported backbone GMAC from Table 5.8; for the 10-step teacher, the plotted cost is approximated as ten times the one-step U-Net cost because the same backbone is evaluated over ten sampling steps. Hard-windowing is omitted because no directly comparable profiler-based GMAC estimate is available for that non-neural baseline.

Supplementary prototype-path runtime measurements are reported in Appendix A.4. They are included only as supporting context because

they reflect the current simulator integration path rather than an optimized deployment implementation.

5.2 Reliability Analysis

The fixed evaluation protocol is described in Chapters 3 and 4. The reliability question in this chapter is therefore whether the observed trends are coherent across the reported result rows. The main trends are reasonably consistent. Early one-step standard-flow models remain below hard-windowing, while later rows that combine focused training, stronger initialization or teacher checkpoints, and consistency-style one-step inference move above it. The final rows in Table 5.7 are also close to each other: all except the UNet-mini standard-FM variant show positive $\text{Gain}_{0.1}$, so the final improvement does not appear to come from a single isolated checkpoint. The repeated mismatch between NMSE and SER rankings also supports the choice to treat simulator SER as the primary metric.

The main reliability limitation is still the training budget. Most configurations were trained once and most ablations used only three epochs, so the tables show controlled single-run trends rather than estimates of mean performance, variance, or convergence. No confidence intervals are reported. Small numerical gaps, especially isolated SER differences in the third decimal place, should therefore not be over-interpreted on their own; the stronger evidence comes from consistent ordering across the reported SNR sweep and from the corresponding $\text{Gain}_{0.1}$ trends. This matters most for Mean Flow, consistency, and CMT, which are sensitive to learning rate, time sampling, EMA choice, teacher quality, trajectory construction, and retraining schedule [12, 13, 14].

5.3 Validity Analysis

The main validity strength of the results is that the primary conclusion is based on end-to-end SER, not only on reconstruction error. Table 5.4 shows why this matters: several rows look strong under NMSE, including the data-parameterized flow and the stronger multi-step standard/AlphaMeanFlow variants, but they do not define the final communication ranking. The result therefore supports the evaluation choice made earlier, namely to use SER and $\text{Gain}_{0.1}$ as the main evidence and NMSE as a supplementary diagnostic.

The same mismatch also matters for checkpoint selection. In the main

runs, checkpoints were selected by quick-validation **NMSE** rather than by repeated full-simulator **SER** ranking, because routine end-to-end simulator evaluation of every saved checkpoint was too expensive under the available compute budget. **SER** is still the primary scientific metric of the thesis, but this means that some reported rows may not correspond to the checkpoint with the very best attainable **SER** within that run. The choice should therefore be read as a practical approximation to **SER**-oriented model selection, not as a fully aligned selection rule.

The hard-windowing and ideal rows also strengthen the result interpretation. Hard-windowing is not a weak baseline, and several early learned estimators fail to beat it. The final improvement is therefore meaningful, but modest: it shows that a carefully aligned learned estimator can become competitive in the studied setting, not that learned estimation is automatically superior. The ideal row confirms that the simulator responds correctly to perfect channel information while also showing that a substantial gap remains after the best learned model.

The internal validity of the ablations is strongest within the shared simulator setup and weakest as evidence of globally optimal model choice. The final integrated table is the outcome of sequential design decisions made after earlier results were observed, so it should be read as the best achieved configuration within the explored design space rather than as an unbiased held-out selection result. External validity is narrower still. The experiments use simulated, non-public **CDL-B** evaluation channels under one pilot-contaminated simulator configuration with **QPSK** pilots, one desired user, four interferers, and 64 receive antennas. At the same time, the evaluation is not confined to a single synthetic source: the model is trained on QuaDRiGa channels and tested on an independent **CDL-B** set with observable spread differences, which provides limited evidence of robustness to simulation-to-simulation distribution shift. That is still not evidence of real-world transfer, however, and measured-channel validation would be needed before making stronger deployment claims. The defensible conclusion is therefore limited to this studied setting: when the design choices are matched to the task, one-step flow-based estimators can show modest gains over hard-windowing in the best reported configurations. The same setting mismatch also limits direct numerical comparison to related work, since many cited papers use different channel models, narrower or cleaner regimes, different inference costs, or reconstruction-focused metrics instead of end-to-end simulator **SER**. Broader robustness across operating regimes, optimized runtime under a production-style implementation, and generalization to measured channels remain open

questions.

Chapter 6

Discussion

As discussed in Chapter 5, the experiments suggest that one-step flow-based models can serve as practical channel estimators in this studied wideband, pilot-contaminated setting, though the conclusion is still qualified by the simulated scope and training budget. This chapter focuses on what the results imply about the design choices behind that outcome.

6.1 Interpretation of the Main Findings

The hard-windowing baseline is an important part of the interpretation. Since hard-windowing is built around the physical delay-domain structure of the channel, it is not a weak reference. The fact that several early learned estimators fail to beat it is therefore meaningful: it shows that replacing a classical estimator with a neural estimator is not automatically beneficial, even when the neural model has access to the same noisy pilot-derived channel observation. In particular, the earliest one-step standard-flow baseline has worse **SER** than hard-windowing, which indicates that one-step learned estimation is more difficult than simply truncating a flow sampler to one network evaluation under the modest early training budget. However, the later experiments show that the same basic one-step UNet-Lite standard-flow can edge above hard-windowing, so the early result is better interpreted as undertraining than as a hard limit of the model family.

The ablation results point toward a more specific lesson. The learned estimator works best when the task is treated as local restoration from the noisy **LS** channel estimate rather than as conditional generation from a generic noise source. This explains why the denoising formulation is stronger than the generative formulation in the early experiments. In the generative case,

the model must learn a harder conditional map from a Gaussian source to a clean channel while still respecting the noisy observation. With the modest model size and training budget used in the first experiments, learning a correction from the noisy channel itself is more feasible than learning the whole conditional transport from noise. Part of the early generative gap may therefore reflect optimization difficulty under that budget rather than only a purely intrinsic disadvantage of the formulation.

The context-input ablation is less decisive, but it still gives a useful practical signal. The empirical log-standard-deviation input performs slightly better than the explicit **SNR** and **SNR/INR** variants in the default comparison, which suggests that much of the disturbance information needed by the model is already present in the observation and can be summarized reasonably well by an observation-derived proxy. At the same time, the gaps are small enough that this result should not be overstated. Later experiment groups run under the **SNR/INR**-aware context should therefore be interpreted mainly as controlled comparisons within that shared setting, not as additional evidence that this context choice is globally best.

The flow-family comparison further supports this interpretation. Methods that are more directly compatible with one-step endpoint prediction, especially the consistency-style model, perform better in the present one-step setting than a naive one-step standard flow. This is expected because standard flow matching learns an instantaneous velocity field whose natural use is numerical integration along a trajectory. Reducing that trajectory to one step introduces a large discretization burden. By contrast, a consistency-style endpoint model is trained to map a noisy point on the path directly toward the clean endpoint, which better matches the direct one-step estimation objective. The no-flow UNet-Lite baseline in the final joint-**DMRS** table is also useful here: it shows that the backbone alone can learn a modest improvement over hard-windowing, but the better consistency and distilled rows still outperform it. This suggests that the flow-style training objectives contribute more than backbone capacity alone.

6.2 Training-Distribution Specialization

Experiment F is best read as an operating-regime specialization rather than as a general claim that narrower training is always better. The broad-range setup remains the more robustness-oriented reference, since it covers a wider span of conditions and also transfers better outside the main target band. Within the target low-**SNR** regime studied here, however, both focused training

distributions improve over the broad-range reference, and the truncated-normal variant gives the strongest result. Under that framing, the results support using a low-SNR-focused training distribution in the final integrated model for this specific setting.

Supporting auxiliary high-SNR simulator diagnostics are reported in Appendix A, Table A.12. On that separate [5,15,25] dB grid, all listed methods already achieve zero SER, so the comparison is visible mainly through reconstruction quality. Those appendix results help qualify the main low-SNR conclusion. Broad-range one-step models such as consistency and Alpha Mean Flow remain ahead of hard-windowing across that high-SNR grid, which suggests that broad-range training generalizes better outside the emphasized low-SNR regime. By contrast, narrowing the training range shifts some accuracy away from much higher SNR values: the low-SNR-focused final models still remain ahead of hard-windowing at 5 dB, just outside the focused band, but give up reconstruction quality at 15 and 25 dB. This is a reasonable trade-off here because the intended operating regime is much lower. The truncated-normal variant should also not be read as emphasizing only the most extreme low-SNR tail. Rather, it concentrates training mass around a representative difficult operating point within that low-SNR band.

6.3 SER Versus Reconstruction Metrics

One of the clearest methodological findings is that reconstruction metrics and communication metrics do not always rank models in the same way. NMSE is useful because it measures physical channel reconstruction error against a known ground truth, and it is practical for checkpoint selection during training. However, the reported results show that the model with the best NMSE does not necessarily give the best end-to-end SER. This appears already in the flow-family comparison, where the data-parameterized flow looks strong under NMSE without giving the best simulator outcome. The same pattern reappears in the final integrated table: the one-step U-Net teacher has the best NMSE among the no-flow/standard rows, yet the 10-step teacher gives better SER and $\text{Gain}_{0.1}$. This mismatch can occur because the detector is sensitive not only to total reconstruction error but also to how the remaining error affects symbol decisions, channel gain calibration, phase structure, and interference handling.

This has an important implication for learned channel estimation. Offline reconstruction metrics are not sufficient by themselves when the goal is end-to-end communication performance. A model that looks strong as a channel

reconstructor can still be less useful for detection than a model with slightly worse **NMSE** but better preservation of the channel properties that matter to the downstream detector. The thesis therefore benefits from integrating the learned estimator into the simulator instead of stopping at offline **NMSE**. This simulator integration is one of the main practical contributions of the work.

6.4 Complexity and Deployment Trade-offs

Backbone capacity clearly matters in the one-step setting, but it is not free. The standard U-Net improves the one-step standard-flow result substantially relative to UNet-Lite, and allowing more sampling steps recovers still more performance. The trade-off is straightforward: better accuracy can be bought either with a larger backbone or with more steps, but both choices raise inference cost. Table 5.8 makes the backbone side of that trade-off concrete, and Figure 5.2 makes the same point more directly for selected final models. The standard U-Net is far more expensive than UNet-Lite and UNet-mini in reported GMAC, while the appendix latency measurements show the same question from the prototype simulator path rather than from an optimized deployment implementation.

The final integrated comparison is most informative when read together with that cost context. The standard U-Net provides the strongest accuracy headroom and is therefore most valuable here as a teacher rather than as the final small deployment-style student. In that sense, the backbone comparison naturally separates roles: the larger U-Net serves as the strongest accuracy-oriented reference among the tested standard-flow backbones, while UNet-Lite and UNet-mini are the more plausible compact students. The SiT-4M row is also useful as a reference point: although it does not beat the standard U-Net on the reported accuracy benchmark, it reaches a similar $\text{Gain}_{0.1}$ with substantially lower reported GMAC. That suggests broader backbone search could still be worthwhile for runtime-sensitive deployments.

The **CMT** result fits naturally into the same trade-off. In the final joint-**DMRS** table, distilling from the stronger U-Net teacher into the extra-small UNet-mini student improves over both the UNet-mini standard-**FM** row and the directly trained UNet-mini consistency row. It also slightly outperforms the larger UNet-Lite one-step rows in communication performance. This is the clearest evidence in the thesis that teacher-student distillation can transfer part of the advantage of a stronger, more expensive model into a much smaller one-step student. The no-flow UNet-Lite row also helps here, since it shows that the gain is not explained by backbone capacity alone.

An auxiliary consistency-student **CMT** mode comparison is reported in Table A.10. That appendix table compares grid and interpolated trajectory representations on the earlier UNet-Lite-teacher setup and is best read as supporting evidence about setup sensitivity rather than as the main verdict on distillation. Its most important lesson is that teacher quality matters. With the earlier UNet-Lite teacher, the **CMT** variants remained weaker than the plain one-step consistency baseline, whereas the stronger U-Net teacher makes distillation into UNet-mini more convincing. The grid-versus-interpolate comparison is still useful as an implementation note, but it is secondary to that broader point. One remaining open issue is the post-**CMT** retraining stage: in the final joint-**DMRS** comparison, retraining slightly degrades the distilled checkpoint rather than improving it. The strongest evidence therefore supports the distilled checkpoint itself, while the retraining extension remains promising but under-tuned.

Chapter 7

Conclusions and Future Work

This chapter summarizes the main conclusions, clarifies the limits of the claims, outlines the most relevant follow-up directions, and reflects on the broader implications of the work.

7.1 Conclusions

The purpose of this thesis was to evaluate whether conditional flow matching can serve as a practical one-step channel-estimation approach in a wideband **MIMO-OFDM** setting with both noise and pilot contamination. The answer in the studied setting is yes, but with some caveats. A naive one-step learned estimator does not automatically outperform a strong hard-windowing baseline. However, the final integrated results show that a carefully aligned one-step learned estimator can, in the best reported one-step setting, show a modest end-to-end **SER** improvement over hard-windowing. In the final joint-**DMRS** comparison, the strongest reported deployment-style one-step result is obtained by combining denoising initialization from the noisy **LS** estimate, consistency-style one-step learning, a strong U-Net teacher, and **CMT** distillation into a compact UNet-mini student. The strongest absolute result in the final comparison is still obtained by the more expensive multi-step teacher, but the main thesis contribution is that a one-step distilled student can recover part of that advantage while showing a modest gain over hard-windowing in the reported comparison.

The thesis goals were largely met. First, conditional flow-matching-based channel estimators were implemented and evaluated against hard-windowing and an ideal upper-bound reference. Second, the ablation studies identified which design choices matter most in this one-step setting: observation-

anchored denoising is stronger than generative initialization from pure noise, one-step endpoint-oriented formulations are better aligned with direct one-step prediction than naive one-step velocity integration, and training-distribution choice materially affects the final communication result. Third, the work produced a well-documented and reusable evaluation framework that combines offline reconstruction metrics with end-to-end simulator **SER**. This proved necessary because **NMSE** alone does not reliably predict downstream communication quality.

These conclusions should still be interpreted within the studied scope. The final gains over hard-windowing are not large in absolute terms, but they are consistent and meaningful given the strength of the baseline and the one-step restriction, while the gap to ideal channel estimation shows that substantial headroom still remains. The thesis therefore does not claim that learned one-step estimation fully replaces other methods. It shows instead that learned estimators can become competitive when the modeling assumptions, objective, backbone, and training regime are matched carefully to the one-step use case.

The clearest practical lessons for similar work are to start from the noisy pilot-derived estimate rather than from an unconditional source, to treat end-to-end communication performance as the primary metric, to tune the training distribution toward the operating regime that matters most, and to use distillation selectively: it is most convincing here when the teacher is clearly stronger and the student is small enough that compression matters. Among the tested extensions, distillation appears to be one of the most promising directions for future improvement of compact one-step students. If this study were to be repeated with the same overall goal, the first priority would be to spend more of the available compute budget on stronger teacher models, broader backbone search, and a cleaner final-stage comparison between direct one-step training, distillation, and post-distillation retraining.

7.2 Limitations

The most important limitation is scope. The reported conclusions are based on a simulated wideband uplink scenario with one desired user, four interferers, fixed pilot structure, ideal covariance interference-plus-noise $\hat{R}_{uu}$ for downstream detection in the main comparisons, and an operating regime that is deliberately emphasized throughout the study. These choices are appropriate for the thesis question, but they limit how far the results can be generalized to other modulation schemes, channel conditions, detection-

pipeline configurations, or real measured channels. The fixed $\widehat{R}_{uu}$ covariance treatment is especially important here: it was held constant as an engineering simplification to isolate the channel-estimation module, so the reported tables do not quantify how alternative, non-ideal covariance estimates would change absolute end-to-end performance. It is plausible that this downstream choice can substantially affect the reported communication metrics, but that sensitivity was not studied systematically enough to support a separate quantitative claim in this thesis.

One important limitation of the study is the restricted hyperparameter exploration. Several of the evaluated one-step flow variants, especially Mean Flow, consistency-based models, and **CMT**, are known to be sensitive to training choices such as learning rate, time sampling, loss weights, and **EMA** settings [12, 13, 14]. Because the experiments had to be run sequentially on limited compute, only a relatively small set of sweeps could be tested. This means that some observed differences between models may partly reflect hyperparameter sensitivity rather than only the intrinsic strengths or weaknesses of the underlying method. The same issue is particularly relevant for the **CMT** auxiliary experiments, where the teacher was not as strong as desired and the distillation setup itself was not exhaustively optimized.

Another limitation is that the final reported model should be read as a best-case result within the explored design space rather than as a completely unbiased held-out model-selection outcome. The chapter structure intentionally follows a sequential design process in which the strongest ideas from earlier comparisons are integrated into the final configuration. This is useful from an engineering perspective, but it means the final rows are partly the product of iterative, informed selection rather than a fully pre-registered experimental plan. However, earlier results still matter for the interpretation because they show that the final one-step model is not the only configuration that beats hard-windowing, and they show that the final gains do not come from a single isolated checkpoint or design choice.

7.3 Future Work

The most valuable next step is to strengthen the evidence around the final one-step estimator by using stronger teachers, broadening the architecture search, and re-evaluating the best candidates under more realistic inference and detection-pipeline assumptions. The most immediate follow-up directions are:

- Testing richer conditioning and guidance mechanisms, for example by combining the current channel-based input with more explicit pilot-domain information, self-conditioning, or pilot-guided constraints or loss.
- Revisiting **CMT** with stronger teachers, better teacher sampling, including iterative guidance or optimization strategies during distillation, and better post-**CMT** retraining schedules, so that the student is guided by a stronger trajectory and is not degraded afterward by the retraining stage.
- Testing additional model families and more advanced consistency formulations, since the consistency model explored in this thesis was intentionally basic.
- Exploring a broader range of computationally efficient backbones, including architectures with sparse convolutions, linear attention, or other operator choices designed to reduce inference cost.
- Extending the experiments to higher-order modulation schemes and to a wider set of system configurations, in order to test how well the conclusions transfer beyond the current setup.

A particularly important follow-up would be to combine these directions with evaluation under less idealized detection-pipeline assumptions, such as non-ideal covariance estimates and other channel conditions. That would make it easier to separate genuine estimator improvements from advantages tied to the current simulator setup and to assess whether the learned estimator remains competitive once the environment becomes less favorable.

7.4 Reflections

This thesis addresses a practical question in next-generation wireless communication: can a learned channel estimator improve communication quality without becoming too slow or too costly? In massive **MIMO** systems, accurate **CSI** is only valuable if it can be obtained within real limits on hardware, processing time, and service quality, so estimator complexity is part of practical performance rather than just an implementation detail [4]. This gives the problem a clear economic dimension: lower-complexity methods are easier to integrate into the physical-layer processing chain, place less demand on compute and hardware resources, and are cheaper to scale across deployed network equipment. That is also why the problem is industrially relevant in

the Huawei Technologies Sweden AB setting where the thesis was conducted. As discussed in Section 1.2.1, the thesis grew out of an industrial question about whether learned channel estimation could be useful in a realistic wireless setting. If such methods can deliver good performance at low complexity, they become easier to turn into deployable products and better suited to advanced commercial services.

The societal relevance follows from the same point. Reliable wireless connectivity supports education, work, public services, healthcare, and economic participation, while broadband access, affordability, and quality affect who can benefit from those opportunities [38]. It also shapes which advanced services are realistic. For instance, healthcare-oriented 5G and 6G work discusses services such as telesurgery, remote monitoring, connected ambulances, and healthcare in rural areas as ways to extend access to care in aging societies [39, 40]. This is where the economic and societal perspectives meet: if industry can deploy lower-complexity communication components at scale, such services become more realistic outside a few ideal settings. Robustness therefore matters not only for better average metrics, but for whether communication remains dependable under noise and interference. Lower computational cost matters socially for the same reason, since it makes advanced services easier to deploy outside heavily provisioned environments, including remote and rural ones [38, 40, 41]. The broader ethical questions are about use, responsibility, and consequences. When machine-learning components become part of public or health-related digital infrastructure, issues of safety, security, accountability, privacy, and human oversight become more important [38]. In safety-critical services such as remote care, there can also be a direct trade-off between security and performance, since added protection may introduce delay or overhead in already latency-sensitive communication [39]. The same communication capabilities can also matter in very different contexts, from civilian care and emergency response to battlefield operation, which is why they should be treated as dual-use technologies [39]. Within this thesis, the ethical point is more limited: the data is simulated, so there are no direct concerns related to personal data, consent, or sensitive user information. Also, because the evaluation is performed in an internal simulator under restricted assumptions, it is transparent that the results should be read as evidence that learned one-step channel estimation can work in the studied setting, not as proof that the method is ready for safety-critical deployment.

The environmental side is closely connected. Lower inference cost and lower computational requirements should reduce energy use in deployment

while also improving the economic case for practical adoption. This matters because the radio access network accounts for the largest share of energy consumption in mobile networks, and recent work shows that processing energy can dominate total consumption [42]. Since channel estimation sits in that processing path and is repeated across deployed infrastructure, reducing its cost is worthwhile in itself. From that perspective, one-step inference and teacher-student distillation are not only latency choices but also sustainability choices. A smaller model that reaches useful performance with fewer computations is easier to deploy, cheaper to run, and likely to consume less energy in operation. That fits the broader 6G goal of improving energy efficiency while lowering total cost of ownership [41].

References

- [1] E. Björnson, E. G. Larsson, and T. L. Marzetta, “Massive mimo: ten myths and one critical question,” *IEEE Communications Magazine*, vol. 54, no. 2, p. 114–123, Feb. 2016. doi: 10.1109/mcom.2016.7402270. [Online]. Available: <http://dx.doi.org/10.1109/MCOM.2016.7402270>
- [2] H.-F. Arraño-Scharager and C. Azurdia-Meza, “OFDM: Today and in the future of next generation wireless communications,” in *2016 IEEE Central America and Panama Student Conference (CONESCAPAN)*, 09 2016. doi: 10.1109/CONESCAPAN.2016.8075209 pp. 1–6.
- [3] M. Boloursaz Mashhadi and D. Gündüz, “Deep learning for massive mimo channel state acquisition and feedback,” *Journal of the Indian Institute of Science*, vol. 100, pp. 369–382, 2020. doi: 10.1007/s41745-020-00169-2
- [4] P. K. Gkonis, “A survey on machine learning techniques for massive mimo configurations: Application areas, performance limitations and future challenges,” *IEEE Access*, vol. 11, pp. 67–88, 2023. doi: 10.1109/ACCESS.2022.3232855
- [5] Y. Liu, Z. Tan, H. Hu, L. J. Cimini, and G. Y. Li, “Channel estimation for ofdm,” *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 1891–1908, 2014. doi: 10.1109/COMST.2014.2320074
- [6] M. K. Saeed, A. Khokhar, and S. Ahmed, “Pilot contamination in massive mimo systems: Challenges and future prospects,” in *2024 International Wireless Communications and Mobile Computing (IWCMC)*. IEEE, May 2024. doi: 10.1109/iwcmc61514.2024.10592426 p. 1504–1509. [Online]. Available: <http://dx.doi.org/10.1109/IWCMC61514.2024.10592426>

- [7] M. Arvinte and J. I. Tamir, “MIMO channel estimation using score-based generative models,” *IEEE Transactions on Wireless Communications*, vol. 22, no. 6, pp. 3698–3713, 2023. doi: 10.1109/TWC.2022.3220784. [Online]. Available: <https://doi.org/10.1109/TWC.2022.3220784>
- [8] T. Wu, Z. Chen, D. He, F. Yang, M. Tao, X. Xu, W. Zhang, and P. Zhang, “Icdm: Interference cancellation diffusion models for wireless semantic communications,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.19983>
- [9] Y. Zhang and Q.-U.-A. Nadeem, “Channel estimation in mimo systems using flow matching models,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.13827>
- [10] Huawei, “Huawei launches comprehensive u6ghz portfolio to unlock 5g-a potential and pave the way for 6g,” 2026, accessed 2026-05-28. [Online]. Available: <https://www.huawei.com/en/news/2026/3/mwc-5ga-u6ghz-6g>
- [11] —, “6g: The next horizon,” 2025, accessed 2026-05-28. [Online]. Available: <https://www.huawei.com/en/huaweitech/future-technologies/6g-the-next-horizon/>
- [12] Z. Geng, Y. Lu, Z. Wu, E. Shechtman, J. Z. Kolter, and K. He, “Improved mean flows: On the challenges of fastforward generative models,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.02012>
- [13] Y. Song and P. Dhariwal, “Improved techniques for training consistency models,” in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 15 078–15 097. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/41bd71e7bf7f9fe68f1c936940fd06bd-Paper-Conference.pdf
- [14] Z. Hu, C.-H. Lai, Y. Mitsufuji, and S. Ermon, “CMT: Mid-training for efficient learning of consistency, mean flow, and flow map models,” in *International Conference on Learning Representations*, 2026, published as a conference paper at ICLR 2026. [Online]. Available: <https://arxiv.org/abs/2509.24526>
- [15] H. Alibraheemi and M. al Ibraheemi, “Wireless communication system with frequency selective channel ofdm modulation technique,”

TELKOMNIKA (Telecommunication Computing Electronics and Control), vol. 18, pp. 1206–1211, 06 2020. doi: 10.12928/TELKOMNIKA.v18i3.14683

- [16] A. Kasibovic, B. Fesl, M. Baur, and W. Utschick, “Addressing pilot contamination in channel estimation with variational autoencoders,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.07071>
- [17] Y. Li, L. J. Cimini, and N. R. Sollenberger, “Robust channel estimation for OFDM systems with rapid dispersive fading channels,” *IEEE Transactions on Communications*, vol. 46, no. 7, pp. 902–915, 1998. doi: 10.1109/26.701317
- [18] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [19] W. Liu, N. Ma, J. Chen, X. Qi, and Y. Ma, “Flow matching-based generative models for mimo channel estimation,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.10941>
- [20] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=XVjTT1nw5z>
- [21] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean flows for one-step generative modeling,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.13447>
- [22] H. Zhang, A. Siarohin, W. Menapace, M. Vasilkovsky, S. Tulyakov, Q. Qu, and I. Skorokhodov, “Alphaflow: Understanding and improving meanflow models,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.20771>
- [23] H. Stark, B. Jing, R. Barzilay, and T. Jaakkola, “Harmonic self-conditioned flow matching for joint multi-ligand docking and binding site design,” in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, 2024, pp. 46 468–46 494. [Online]. Available: <https://proceedings.mlr.press/v235/stark24a.html>

- [24] L. Yang, Z. Zhang, Z. Zhang, X. Liu, M. Xu, W. Zhang, C. Meng, S. Ermon, and B. Cui, “Consistency flow matching: Defining straight flows with velocity consistency,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.02398>
- [25] Z. Jiang, F. Zhu, C. Huang, R. Jin, Z. Yang, X. Chen, Z. Zhang, and M. Debbah, “Recursive flow: A generative framework for mimo channel estimation,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.15767>
- [26] Z. Jiang, F. Zhu, S. Jiang, C. Huang, Z. Yang, R. Jin, Z. Zhang, and M. Debbah, “One-step generative channel estimation via average velocity field,” 2026. [Online]. Available: <https://arxiv.org/abs/2512.04501>
- [27] Y. Zhang, P. Yu, Y. Zhu, Y. Chang, F. Gao, Y. Wu, and O. Leong, “Flow priors for linear inverse problems via iterative corrupted trajectory matching,” in *Advances in Neural Information Processing Systems* 37, 2024. doi: 10.52202/079017-1829 pp. 57 389–57 417. [Online]. Available: <https://doi.org/10.52202/079017-1829>
- [28] J. Kim, B. S. Kim, and J. C. Ye, “FlowDPS: Flow-driven posterior sampling for inverse problems,” in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. doi: 10.1109/ICCV51701.2025.01146 pp. 12 328–12 337. [Online]. Available: <https://doi.org/10.1109/ICCV51701.2025.01146>
- [29] M. Pourya, B. E. Rawas, and M. Unser, “Flower: A flow-matching solver for inverse problems,” in *International Conference on Learning Representations*, 2026, published as a conference paper at ICLR 2026. [Online]. Available: <https://arxiv.org/abs/2509.26287>
- [30] A. Tong, K. Fatras, N. Malkin, G. Hugué, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio, “Improving and generalizing flow-based generative models with minibatch optimal transport,” *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=CD9Snc73AW>
- [31] X. Zhang, S. Tan, Q. Nguyen, Q. Dao, L. Han, X. He, T. Zhang, A. Mrdovic, and D. Metaxas, “Flow straighter and faster: Efficient one-step generative modeling via meanflow on rectified trajectories,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.23342>

- [32] Q. Dao, H. Phung, T. T. Dao, D. N. Metaxas, and A. Tran, “Self-corrected flow distillation for consistent one-step and few-step image generation,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 3, pp. 2654–2662, 2025. doi: 10.1609/aaai.v39i3.32269. [Online]. Available: <https://doi.org/10.1609/aaai.v39i3.32269>
- [33] S. Jaeckel, L. Raschkowski, K. Börner, and L. Thiele, “Quadriga: A 3-d multi-cell channel model with time evolution for enabling virtual field trials,” *IEEE Transactions on Antennas and Propagation*, vol. 62, 06 2014. doi: 10.1109/TAP.2014.2310220
- [34] S. Jaeckel *et al.*, *QuaDRiGa Version 2.8.1 documentation*, Fraunhofer Heinrich Hertz Institute, 2021. [Online]. Available: <https://quadriga-channel-model.de/>
- [35] 3GPP, “Study on channel model for frequencies from 0.5 to 100 GHz,” 3rd Generation Partnership Project (3GPP), Technical Report TR 38.901, 2025. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/38_series/38.901/
- [36] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [37] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [38] Broadband Commission for Sustainable Development, “The state of broadband 2024: Leveraging ai for universal connectivity,” <https://www.broadbandcommission.org/publication/state-of-broadband-2024/>, Jun. 2024, annual flagship report of the Broadband Commission.
- [39] Q. Zhang, J. Liu, and G. Zhao, “Towards 5g enabled tactile robotic telesurgery,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.03586>
- [40] G. Cisotto, E. Casarin, and S. Tomasin, “Requirements and enablers of advanced healthcare services over future cellular systems,” *IEEE*

- Communications Magazine*, vol. 58, no. 3, pp. 76–81, 2020. doi: 10.1109/MCOM.001.1900349. [Online]. Available: <https://doi.org/10.1109/MCOM.001.1900349>
- [41] E. Selva, A. Gati, M.-H. Hamon, B. Masood Khorsandi, S. Wunderer, S. Bories, G. Calochira, G. Avino, S. Wänstedt, P. Bergmark, T. Svensson, and M. Matinmikko-Blue, “Towards a 6G embedding sustainability,” in *2023 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2023. doi: 10.1109/ICCWorkshops57953.2023.10283655 pp. 1588–1593. [Online]. Available: <https://doi.org/10.1109/ICCWorkshops57953.2023.10283655>
- [42] U. Tariq, R. Raj, M. Dzaferagic, and D. Kilper, “Energy consumption in next generation radio access networks,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.11899>

Appendix A

Auxiliary Results

This appendix collects supplementary diagnostic metric definitions, companion diagnostic tables for the main Chapter 5 comparisons, supplementary runtime measurements, exploratory loss-ablation details, and auxiliary result tables that are useful for auditing the experiments, but are not used as primary ranking criteria. The primary comparison remains the main evaluation setup reported in Chapter 5.

A.1 Supplementary Diagnostic Metric Definitions

For the n th evaluation sample, let $\hat{H}^{(n)}$ and $H^{(n)}$ denote the predicted and target channels after inverse transformation to the frequency-antenna domain. The supplementary diagnostic metrics are used only to interpret reconstruction behavior. They are not treated as the main model-ranking criteria.

The first diagnostic is signed channel-gain bias,

$$\Delta G_n = 10 \log_{10} \left(\frac{\sum |\hat{H}^{(n)}|^2}{\sum |H^{(n)}|^2} \right). \quad (\text{A.1})$$

This metric measures whether the overall channel power is over- or underestimated. Positive values indicate overestimation and negative values indicate underestimation. The companion tables below report this signed quantity ΔG_n directly.

The second diagnostic is complex cosine similarity,

$$\text{CosSim}_n = \frac{|\langle H^{(n)}, \hat{H}^{(n)} \rangle|}{\|H^{(n)}\|_2 \|\hat{H}^{(n)}\|_2 + \varepsilon}, \quad (\text{A.2})$$

which measures directional alignment between the predicted and target channels. In the present context, it is most useful as a rough structural-alignment check rather than as a power-calibration metric.

The third diagnostic is multi-scale maximum mean discrepancy on flattened channel magnitudes,

$$\text{MMD}^2 = \sum_{\sigma \in \{0.1, 1.0, 10.0\}} [\mathbb{E}k_\sigma(a, a') + \mathbb{E}k_\sigma(b, b') - 2\mathbb{E}k_\sigma(a, b)], \quad (\text{A.3})$$

where a, a' are independent predicted flattened magnitude vectors, b, b' are independent target flattened magnitude vectors, and $k_\sigma(a, b) = \exp(-\|a - b\|_2^2 / (2\sigma^2))$. Low-SNR channel estimation is ill-posed, so pointwise reconstruction metrics such as NMSE can over-penalize estimates that remain plausible under the conditional channel distribution. For that reason, the thesis also includes this distribution-level diagnostic. Here, MMD plays a Fréchet Inception Distance (FID)-like role without requiring an external feature extractor for wireless channels: instead of checking only samplewise agreement, it tests whether the predicted channel magnitudes remain close to the target magnitude distribution. At the same time, the reported estimators are deterministic conditional point estimators and the main task is downstream symbol detection, so MMD is used only as a secondary check. In these experiments, it varies more weakly than SER or NMSE and is therefore not used for primary ranking.

Together, cosine similarity, signed gain bias, and MMD help diagnose cases where two estimators have similar NMSE or SER but differ in structural alignment, power calibration, or distribution-level behavior.

A.1.1 Companion Diagnostic Tables for the Main Results

To keep Chapter 5 focused on the primary ranking metrics, the main tables report SER, $\text{Gain}_{0.1}$, and NMSE. Tables A.1–A.7 mirror those same experiment groups using the supplementary diagnostics defined above. For compactness, only the overall cosine similarity, signed channel-gain bias ΔG , and MMD are repeated here. These tables are intended for interpretation rather

than for primary model ranking.

Table A.1: Supplementary diagnostics for the baseline table in Chapter 5.

Method	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
Raw LS	0.180	15.41	0.737
Hard-windowing (HW)	0.395	8.43	0.737
Ideal	1.000	0.00	0.000

Table A.2: Supplementary diagnostics for the task-formulation table in Chapter 5.

Task formulation	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
A2 Denoising	0.664	-4.36	0.753
A1 Generative	0.624	-4.29	0.744

Table A.3: Supplementary diagnostics for the context-embedding table in Chapter 5.

Context	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
B1 None	0.667	-3.63	0.743
B2 logstd	0.670	-4.20	0.748
B3 SNR + logstd	0.659	-4.62	0.756
B4 INR + SNR + logstd	0.661	-4.62	0.757

Table A.4: Supplementary diagnostics for the flow-model table in Chapter 5.

Model	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
C1 Standard FM (1 step)	0.664	-4.36	0.753
C1 Standard FM (5 steps)	0.706	-2.31	0.739
C3 Mean Flow (1 step)	0.704	-2.52	0.743
C4 Consistency (1 step)	0.656	-0.74	0.737
C2 Data-parameterized flow (1 step)	0.712	-5.06	0.783
C5 AlphaMeanFlow (1 step)	0.706	-2.61	0.743

Table A.5: Supplementary diagnostics for the backbone table in Chapter 5.

Backbone	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
D1 UNet-Lite	0.664	-4.36	0.753
D3 SiT-4M	0.723	-4.40	0.771
D2 U-Net	0.732	-3.81	0.757

Table A.6: Supplementary diagnostics for the focused low-SNR training-distribution table in Chapter 5.

Training SNR distribution	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
Standard wide-range	0.664	-4.36	0.753
F2 Truncnorm on $[-20, 0]$ dB	0.700	-4.48	0.759
F1 Uniform on $[-20, 0]$ dB	0.686	-5.21	0.774

Table A.7: Supplementary diagnostics for the final integrated joint-DMRS table in Chapter 5.

Variant	CosSim $\uparrow$	ΔG (dB)	MMD $\downarrow$
<i>No-flow / standard</i>			
U-Net teacher (1 step)	0.767	-2.37	0.741
U-Net teacher (10 steps)	0.769	-0.86	0.737
No-flow UNet-Lite	0.729	-5.02	0.785
UNet-Lite standard FM	0.726	-2.38	0.740
UNet-mini standard FM	0.675	-2.80	0.739
<i>Consistency / CMT</i>			
UNet-Lite consistency	0.692	-0.25	0.737
UNet-mini consistency	0.649	-0.37	0.737
CMT, U-Net teacher / UNet-mini student	0.724	-1.06	0.737
retrained CMT, UNet-mini student	0.664	-0.63	0.737

Across these companion tables, the signed gain bias reveals a pattern that is less obvious from NMSE alone. The classical baselines show positive bias, which is plausible because raw LS leaves the noisy observation unchanged and hard-windowing mainly suppresses part of the weaker noise rather than fully correcting the retained power. By contrast, most learned flow-based estimators show negative bias, especially the standard FM and data-parameterized variants, which indicates a systematic tendency to underestimate the overall

channel power. The consistency-based rows are typically closest to zero, which suggests that their remaining gains are less about global power calibration and more about recovering the channel structure correctly. The cosine-similarity column adds that structural view more explicitly. For instance, in Table A.4, the consistency model has a smaller gain error that partly compensates for its weaker cosine similarity. At the same time, **MMD** changes only weakly across the compared methods, and the near-tie between raw **LS**, hard-windowing, and the best learned rows shows that this magnitude-distribution diagnostic is much less discriminative than the downstream communication metrics.

A.2 Exploratory Pilot and Huber Auxiliary-Loss Sweeps

This appendix retains only two small auxiliary-loss tests: a pilot-guided reconstruction term and a pseudo-Huber residual term. These choices were not arbitrary. The pilot-guided term was inspired by the Recursive Flow paper, which uses pilot-guided sampling to enforce measurement consistency during iterative inference [25]. Since the present thesis focuses on one-step endpoint models rather than iterative sampling, the analogous idea was tested here as an auxiliary loss instead. The pseudo-Huber term was motivated by the consistency literature, because the strongest one-step family in the main results is the consistency-style model and the underlying paper recommends a pseudo-Huber objective rather than ordinary squared error [13]. This appendix therefore treats the Huber variant as a small ablation of whether the loss itself could explain part of that improvement. Both tests were run only on the standard flow-matching baseline, not across the full model family, and the available compute budget limited them to independent one-epoch trials. The resulting measurements are therefore not strong enough to support model-selection claims. They are kept only to document that these two penalties showed enough tentative promise to be worth more systematic follow-up.

To keep the notation independent of the selected wrapper, let x_t denote the sampled training state on the interpolation path, let x_1 denote the clean target channel, and let $\hat{x}_1$ denote the clean-channel endpoint estimate implied by the model. For the pilot-guided term, define $\hat{H} = \mathcal{T}_{\text{DB}}^{-1}(\hat{x}_1)$ as the predicted channel mapped back from the delay-beam representation to the projected frequency-antenna observation domain. Let y reuse the earlier notation for the projected noisy **LS** observation seen by the estimator, let $\mathcal{L}_{\text{base}}$ denote the

corresponding base training loss from the main text, let λ_{pilot} and λ_{huber} be the auxiliary-loss weights, let $\varepsilon > 0$ prevent division by zero, let $\beta > 0$ be the pseudo-Huber scale parameter, and let $\text{mean}(\cdot)$ denote the empirical mean over all batch elements and tensor entries. For the retained appendix-only tests, the explored auxiliary objective takes the form

$$\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{base}} + \lambda_{\text{pilot}} \mathcal{L}_{\text{pilot}} + \lambda_{\text{huber}} \mathcal{L}_{\text{huber}}. \quad (\text{A.4})$$

The explored penalties were:

- **Pilot-guided loss:**

$$\mathcal{L}_{\text{pilot}} = \frac{\|\hat{H} - y\|_F^2}{\|y\|_F^2 + \varepsilon}. \quad (\text{A.5})$$

This term constrains the predicted channel to remain compatible with the retained projected pilot observation.

- **Pseudo-Huber loss:**

$$\mathcal{L}_{\text{huber}} = \text{mean} \left[\sqrt{\|\hat{x}_1 - x_1\|_2^2 + \beta^2} - \beta \right]. \quad (\text{A.6})$$

This adds a robust pointwise residual penalty.

In all such sweeps, the weights were chosen manually in a small number of SER-oriented trial configurations. Outside these exploratory runs, all auxiliary weights are kept at zero.

Across these one-epoch sensitivity checks, small nonzero pilot and Huber weights sometimes improve average **SER** relative to the corresponding zero-weight row, but the effect is neither large nor stable enough to justify a conclusion about converged performance. These rows are therefore kept only as exploratory appendix material, not as evidence for changing the main training pipeline.

Table A.8 reports the pilot-guided sweep rows in the same evaluation setup used in the main text. The IDs are left unchanged to match the original experiment numbering from Chapter 4.

Table A.8: Exploratory pilot-guided auxiliary-loss sweep for one-step standard flow matching. SER is averaged over -14 to -6 dB. All rows are independent one-epoch sensitivity checks.

ID	Setting	Main metrics		
		SER ↓	Gain _{0.1} (dB) ↑	NMSE _{pw} (dB) ↓
50	Pilot loss weight 0	0.3063	-1.84	-1.32
51	Pilot loss weight 0.01	0.2672	-1.21	-1.43
52	Pilot loss weight 0.03	0.2885	-1.53	-1.74
53	Pilot loss weight 0.05	0.2798	-1.37	-1.40
54	Pilot loss weight 0.1	0.2983	-1.72	-1.60

Table A.9 reports the pseudo-Huber sweep rows. Again, these experiments are appendix-only sensitivity checks rather than converged model-selection evidence, so they are not used in the main ranking of estimators.

Table A.9: Exploratory pseudo-Huber auxiliary-loss sweep for one-step standard flow matching. SER is averaged over -14 to -6 dB. All rows are independent one-epoch sensitivity checks.

ID	Setting	Main metrics		
		SER ↓	Gain _{0.1} (dB) ↑	NMSE _{pw} (dB) ↓
65	Huber loss weight 0	0.2805	-1.43	-1.57
66	Huber loss weight 0.01	0.2445	-1.02	-1.49
67	Huber loss weight 0.03	0.3018	-1.71	-1.27
68	Huber loss weight 0.05	0.2974	-1.72	-0.99
69	Huber loss weight 0.1	0.3210	-1.94	-1.49

A.3 Auxiliary **CMT** Comparisons

Table A.10 reports an auxiliary grid-versus-interpolate **CMT** mode comparison from the earlier UNet-Lite-teacher setup. These rows are kept in the appendix because they mainly support the implementation choice of which trajectory representation to use by default. The grid mode is slightly stronger before retraining, while the retrained interpolated checkpoint is marginally better overall. Both nevertheless remain weaker than the plain one-step consistency baseline from the main text.

- **E1 CMT grid:** The student is supervised from states sampled on the discrete 10-step teacher rollout toward the teacher endpoint.
- **E2 CMT interpolate:** The teacher rollout is first linearly densified so that the student can be supervised from additional intermediate states along the same trajectory toward the teacher endpoint.

Table A.10: Auxiliary **CMT** mode comparison for a consistency student. Per-**SNR** SER, $\text{Gain}_{0.1}$, and **NMSE** are reported across the evaluation grid. These rows are used mainly to compare grid and interpolated trajectory representations before the final integrated **CMT** study.

ID	Setting	SER at SNR (dB) ↓					$\text{Gain}_{0.1}$ (dB) ↑	NMSE_{pw} (dB) ↓
		−14	−12	−10	−8	−6		
E1	CMT grid	0.3580	0.2236	0.1082	0.0368	0.0078	0.40	−3.64
E1	CMT grid + retrain	0.3527	0.2178	0.1049	0.0355	0.0077	0.48	−3.13
E2	CMT interpolate	0.3692	0.2296	0.1109	0.0381	0.0080	0.33	−3.72
E2	CMT interpolate + retrain	0.3504	0.2161	0.1048	0.0356	0.0076	0.49	−3.17

A.4 Supplementary Runtime Measurements

Table A.11 reports representative joint-**DMRS** simulator-path runtime for the hard-windowing reference and selected learned one-step models. These timings are included as supporting context rather than as primary ranking evidence. Their role is to show whether the reported one-step estimators remain in a broadly practical runtime range under the implemented prototype path.

Table A.11: Representative joint-**DMRS** simulator-path latency. $\text{Gain}_{0.1}$ is reported relative to hard-windowing, and latency is the overall per-sample channel-estimation latency reported by the simulator for the full estimator path.

Variant	$\text{Gain}_{0.1}$ (dB) ↑	Latency (ms) ↓
Hard-windowing (HW)	0.00	0.251
U-Net teacher backbone (1 step)	0.53	1.489
UNet-Lite consistency	0.54	0.535
UNet-mini consistency	0.43	0.445

These measurements should not be read as optimized deployment benchmarks. They are intended as representative average runtimes for the

implemented prototype path over the low-SNR evaluation setup, not as a full latency benchmarking study with repeated-run variance or tail-latency analysis. The reported latency covers the full channel-estimation module over all test samples on one GPU and includes adapter overhead, delay-beam transforms, inverse transforms, interpolation back to the simulator grid, and DMRS handling. Hard-windowing is more tightly integrated and does not use the same adapter path, while the neural-network rows typically utilize the GPU much more effectively. The experiments were run on the available 2080 Ti GPU with no separate warm-up phase, so absolute gaps should be interpreted cautiously.

Even with these caveats, the runtime results still support a limited design conclusion. Increasing solver steps is expensive because each step requires another neural function evaluation, and larger backbones also raise the implementation cost of the estimator path. Within that prototype setting, the reported learned one-step estimators remain in the same broad runtime ballpark as hard-windowing while improving the end-to-end communication results. The table is therefore best read as supplementary evidence that the main gains were not obtained simply by resorting to a large iterative estimator.

A.5 Auxiliary High-SNR Simulator Diagnostics

Table A.12 reports selected auxiliary high-SNR simulator runs from the separate [5, 15, 25] dB grid. All listed methods reach zero SER at every reported SNR point on this easier slice, so the table is included only as a reconstruction-quality check outside the low-SNR regime emphasized in the main chapter. Broad-range one-step models such as consistency and Alpha Mean Flow remain better than hard-windowing throughout this grid, while the low-SNR-focused final models give up part of that higher-SNR margin but still remain ahead of hard-windowing at 5 dB, just outside the focused training band.

Table A.12: Auxiliary high-SNR simulator diagnostics on the separate [5, 15, 25] dB grid. Since all listed rows achieve zero SER throughout this grid, the table reports only reconstruction quality through NMSE.

Variant	NMSE _{pw} (dB) at SNR ↓			Overall NMSE _{pw} (dB) ↓
	5 dB	15 dB	25 dB	
Hard-windowing	-7.36	-16.71	-25.03	-21.93
Raw LS	0.18	-9.82	-19.82	-15.50
Consistency	-11.33	-18.70	-25.56	-23.58
Alpha Mean Flow	-12.13	-19.46	-27.16	-24.81
Joint-DMRS standard FM (UNet-Lite)	-11.65	-6.47	-8.66	-8.43
Joint-DMRS consistency (UNet-Lite)	-9.99	-7.04	-6.76	-6.81
Joint-DMRS CMT (U-Net teacher / U-Net-mini student)	-10.44	-8.69	-6.25	-6.44

A.6 No-Interference Sanity Check

As a small auxiliary sanity check, the reported final one-step student from Table 5.7, namely the **CMT**-distilled UNet-mini, was also considered in a no-interference setting. These results are not used for primary model selection, but they help verify that the reported end-to-end trends remain coherent when the structured interference term is removed.

Table A.13: Auxiliary side-by-side low-SNR gain summary for the with-interference and no-interference settings. Gain_{0.1} is measured relative to hard-windowing at the target SER of 0.1.

Variant	Interference Gain _{0.1} (dB) ↑	No-interference Gain _{0.1} (dB) ↑
UNet-mini student	0.661326	1.049822
Ideal	2.577861	1.985509

This behavior is as expected. When interference is removed, or becomes very weak, the estimation and detection task becomes easier for all methods. As a result, the gap between the learned algorithms and the classical baselines becomes smaller than in the main with-interference setting, because all methods operate in a more favorable regime and their performance moves into a closer range.

A.7 Supplementary Backbone Diagram

The main implementation chapter retains the full U-Net schematic because the U-Net family is the default and final backbone used throughout the study. The

SiT-4M diagram is moved here as supplementary material for the secondary transformer benchmark.

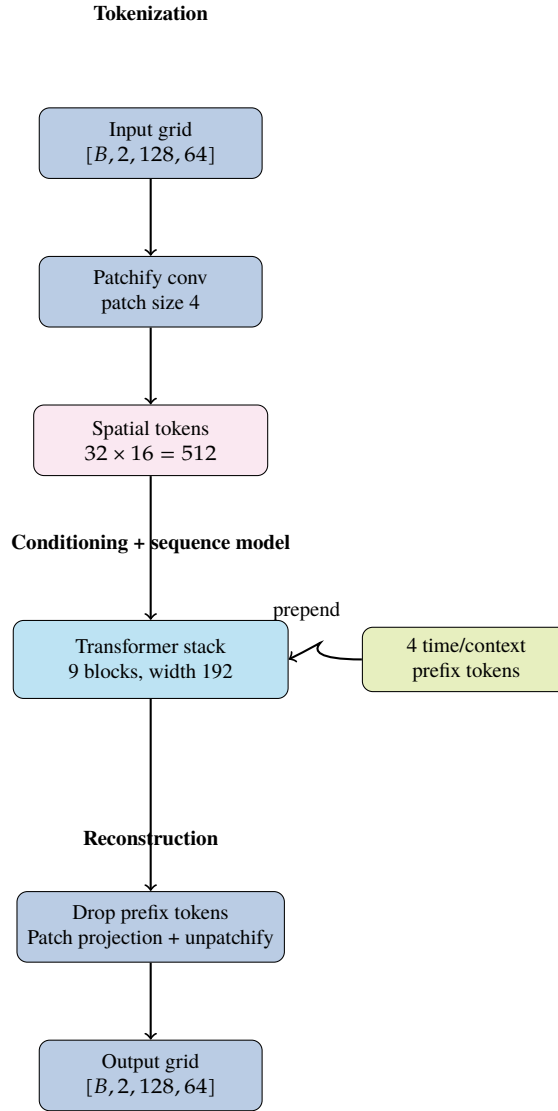


Figure A.1: Supplementary SiT-4M backbone schematic. A patchify convolution converts the delay-beam grid into 512 spatial tokens, four time/context prefix tokens are concatenated before the transformer stack, and the prefix tokens are discarded before patch projection and unpatchifying back to the dense output grid. The transformer blocks use RMSNorm, RoPE attention, SwiGLU feed-forward layers, and zero-initialized residual gates.

A.8 Additional Data and Domain Diagnostics

This section collects supplementary figures that support the data-generation and domain-transformation discussion in Section 3.2. They are not required for the main narrative, but they provide additional evidence about the structural effect of the delay-beam transformation and about representative clean/noisy behavior beyond the figures retained in the main text.

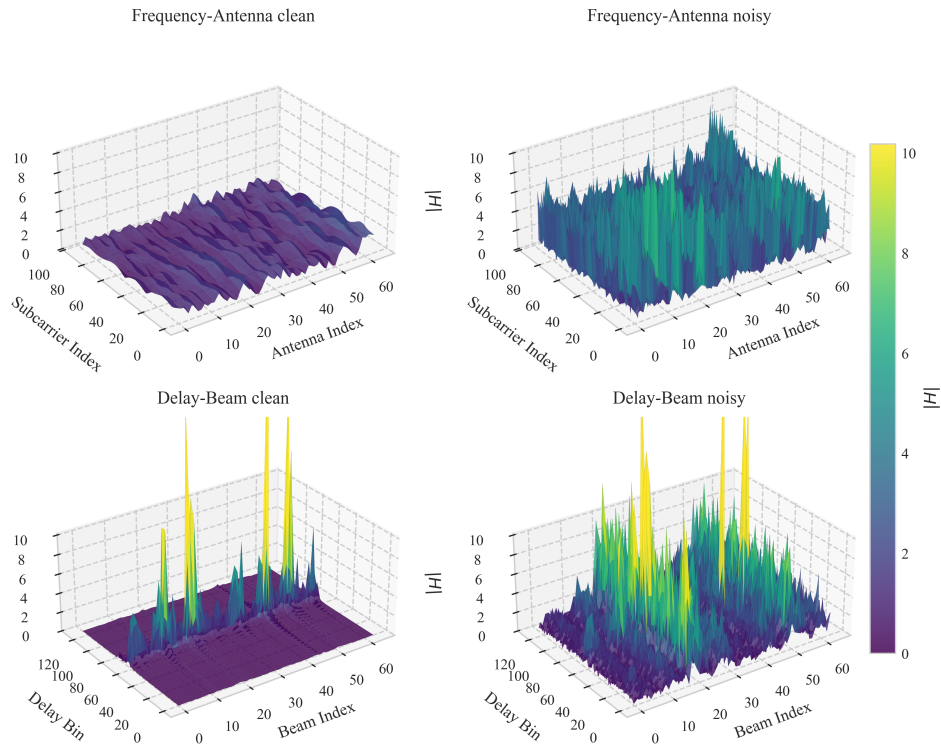
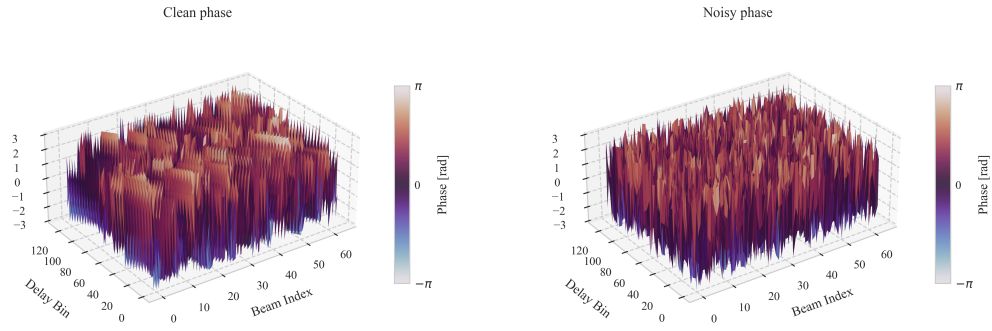


Figure A.2: Supplementary four-panel magnitude summary of the same representative sample in the frequency-antenna and delay-beam domains, for both clean and noisy observations. This figure complements the main-text visualizations by gathering the qualitative comparison into a single overview.



(a) 3D phase of the clean channel.

(b) 3D phase of the noisy channel.

Figure A.3: Supplementary delay-beam phase visualizations for the same representative sample used in the main text. These plots are included for completeness. They show that there is an inherent phase structure in the delay-beam channel representation, and that noise affects this structure in a nontrivial way.

Appendix B

Supporting Materials

B.1 Implementation pseudocode

This appendix collects compact pseudocode for the main implementation paths described in Chapters 3 and 4. It uses the notation e for the source state, x_1 for the clean target channel, and $x_t = (1 - t)e + tx_1$ for the straight source-to-target path. Consistency models are written in the same forward-time notation as the theory section. For Mean-Flow-style models, the interval offset is written as $h = t - r \geq 0$.

Algorithm 1 Physics-collator preprocessing

- 1: Draw target channels, four interferer channels, target pilots, and interference pilots.
 - 2: Sample **SNR** and per-interferer **INR** values from the configured samplers.
 - 3: Scale the desired and interfering channels according to the sampled **SNR/INR**.
 - 4: Run the simulator transmission model and extract the received **DMRS** symbols.
 - 5: Form the masked **LS** channel estimate on pilot positions.
 - 6: **if** the **DMRS** comb is sparse **then**
 - 7: Keep only the active pilot-bearing subcarriers.
 - 8: Compensate for the pilot-power normalization change caused by removing the inactive Comb-2 rows.
 - 9: **end if**
 - 10: Select either one randomly chosen **DMRS** symbol or both jointly, depending on the model variant.
 - 11: Align the clean target to the same **DMRS** configuration.
 - 12: Transform the noisy **LS** estimate and clean target to the delay-beam domain.
 - 13: Return real/imaginary tensors (y, x_1) and scalar context values.
-

Algorithm 2 Standard flow-matching training step

- 1: Draw a minibatch of noisy and clean channel pairs (y, x_1) .
 - 2: Choose the source state e : use $e = y$ for denoising or sample Gaussian noise for generative mode.
 - 3: Sample t with the shared `TimeSampler`.
 - 4: Form $x_t = (1 - t)e + tx_1$ and target velocity $v^* = x_1 - e$.
 - 5: Predict $v_\theta(x_t, t, \text{context}, \text{cond})$ with the wrapped backbone.
 - 6: Minimize $\|v_\theta - v^*\|_2^2$, plus any enabled auxiliary losses.
 - 7: Update the optimizer and the **EMA** checkpoints.
-

Algorithm 3 Mean Flow and AlphaMeanFlow training step

-
- 1: Draw a minibatch (y, x_1) and choose the source state e .
 - 2: Sample the implementation time pair (t, r) with $r < t$ and set $h = t - r$.
 - 3: Form $x_t = (1 - t)e + tx_1$ and $v^* = x_1 - e$.
 - 4: Predict the boundary velocity $v_\theta(x_t, t) = u_\theta(x_t, t, t, \text{context}, \text{cond})$.
 - 5: Predict the interval-average velocity $u_\theta(x_t, r, t, \text{context}, \text{cond})$.
 - 6: Compute the exact **Jacobian-vector product (JVP)** correction $\dot{u}_\theta$ using v_θ as the tangent.
 - 7: Form the improved Mean Flow compound velocity $V_\theta = u_\theta + (t - r) \text{sg}(\dot{u}_\theta)$.
 - 8: **if** AlphaMeanFlow is enabled **then**
 - 9: Minimize the scheduled blend between the easier direct target and the implemented Mean Flow objective.
 - 10: **else**
 - 11: Minimize $\|V_\theta - v^*\|_2^2$.
 - 12: **end if**
 - 13: Update the optimizer and the **EMA** checkpoints.
-

Algorithm 4 Data-parameterized flow training step

-
- 1: Draw a minibatch (y, x_1) , choose source state e , and sample t .
 - 2: Form $x_t = (1 - t)e + tx_1$.
 - 3: Predict the clean endpoint $\hat{x}_1 = f_\theta(x_t, t, \text{context}, \text{cond})$.
 - 4: Minimize $\|\hat{x}_1 - x_1\|_2^2$.
 - 5: Update the optimizer and the **EMA** checkpoints.
-

Algorithm 5 Consistency-model training step

-
- 1: Draw a minibatch (y, x_1) and choose source state e .
 - 2: Sample adjacent levels $t_1 < t_2$ from the discrete schedule.
 - 3: Form $x_{t_1} = (1 - t_1)e + t_1x_1$.
 - 4: Form $x_{t_2} = (1 - t_2)e + t_2x_1$.
 - 5: Evaluate $f_\theta(x_{t_1}, t_1)$ and $f_\theta(x_{t_2}, t_2)$.
 - 6: Stop gradients on the higher- t branch before using it as the target.
 - 7: Minimize the pseudo-Huber consistency loss between the two endpoint estimates.
 - 8: Update the optimizer and the **EMA** checkpoints.
-

Algorithm 6 CMT teacher-student distillation

- 1: Load or train a standard-flow teacher model.
 - 2: Freeze the teacher parameters.
 - 3: Initialize the student either from a student checkpoint or by copying compatible teacher weights.
 - 4: Set the teacher rollout length to $S = 10$ steps.
 - 5: **for** each distillation batch **do**
 - 6: Draw a simulator-aligned noisy input y and context batch.
 - 7: Set $\hat{x}^0 = y$.
 - 8: **for** $s = 0, \dots, S - 1$ **do**
 - 9: Roll out one Euler step:
 - 10: $\hat{x}^{s+1} = \hat{x}^s + S^{-1}v_{\text{teacher}}(\hat{x}^s, s/S, \text{context})$.
 - 11: **end for**
 - 12: Use the stored teacher states, not the raw teacher velocities, to define student labels.
 - 13: **if** `cmt_mode = interpolate` **then**
 - 14: Linearly densify the teacher trajectory to 40 points.
 - 15: **else**
 - 16: Use the 10-step teacher grid directly.
 - 17: **end if**
 - 18: Sample a start index i from the stored teacher states and set j to the teacher endpoint index.
 - 19: **if** the student is Mean-Flow-like **then**
 - 20: Train on the interval-average target $(\hat{x}^j - \hat{x}^i)/(t_j - t_i)$ using the corresponding interval input.
 - 21: **else**
 - 22: Train the student from the sampled start state to predict the teacher endpoint $\hat{x}^j$.
 - 23: For consistency models, pass the sampled start level t_i in the same forward-time notation.
 - 24: **end if**
 - 25: **end for**
 - 26: Retrain or evaluate the student according to the experiment configuration.
-

Algorithm 7 Learned-estimator inference in the simulator adapter

- 1: Form **LS** estimates from the received **DMRS** symbols.
 - 2: Downsample to active pilot subcarriers if the **DMRS** comb is sparse.
 - 3: Compensate for the pilot-power normalization change caused by removing the inactive Comb-2 rows.
 - 4: **if** joint-**DMRS** inference is enabled **then**
 - 5: Stack the two **DMRS**-symbol estimates as the model input.
 - 6: Convert the stacked complex estimate to real/imaginary channels and transform to delay-beam.
 - 7: Initialize the source state e from the noisy **LS** estimate or from Gaussian noise.
 - 8: **if** the wrapper is one-step endpoint or consistency style **then**
 - 9: Evaluate the wrapped model once to obtain both channel estimates.
 - 10: **else**
 - 11: Apply the configured low-step sampler for the requested number of inference steps.
 - 12: **end if**
 - 13: Transform the joint estimate back to the frequency-antenna domain and split the two **DMRS**-symbol estimates.
 - 14: **else**
 - 15: **for** each available **DMRS** symbol **do**
 - 16: Convert the complex estimate to real/imaginary channels and transform to delay-beam.
 - 17: Initialize the source state e from the noisy **LS** estimate or from Gaussian noise.
 - 18: **if** the wrapper is one-step endpoint or consistency style **then**
 - 19: Evaluate the wrapped model once to obtain the channel estimate.
 - 20: **else**
 - 21: Apply the configured low-step sampler for the requested number of inference steps.
 - 22: **end if**
 - 23: Transform the estimate back to the frequency-antenna domain.
 - 24: **end for**
 - 25: **end if**
 - 26: Interpolate active-subcarrier estimates back to the 192-subcarrier grid.
 - 27: Place the two **DMRS**-symbol estimates in their original **OFDM** positions.
 - 28: Fill data-symbol positions with the mean **DMRS** estimate and return the channel tensor to the simulator.
-

B.2 Least-Squares (LS) Projection

We derive the **LS** projection independently on each subcarrier. For one subcarrier k , the linear pilot model is

$$Y_k = H_k P_k + N_k, \quad (\text{B.1})$$

where

$$Y_k \in \mathbb{C}^{N_r \times \tau_p}, \quad H_k \in \mathbb{C}^{N_r \times N_t}, \quad P_k \in \mathbb{C}^{N_t \times \tau_p}, \quad N_k \in \mathbb{C}^{N_r \times \tau_p}.$$

The least-squares (LS) estimate of the channel matrix H_k is obtained by minimizing the Frobenius norm of the residual:

$$\hat{H}_{\text{LS},k} = \arg \min_X \|Y_k - X P_k\|_F^2. \quad (\text{B.2})$$

Using the identity $\|A\|_F^2 = \text{Tr}(A A^H)$, the objective function can be written as

$$J_k(X) = \text{Tr}[(Y_k - X P_k)(Y_k - X P_k)^H]. \quad (\text{B.3})$$

Expanding the expression gives

$$J_k(X) = \text{Tr}(Y_k Y_k^H) - \text{Tr}(Y_k P_k^H X^H) - \text{Tr}(X P_k Y_k^H) + \text{Tr}(X P_k P_k^H X^H). \quad (\text{B.4})$$

Taking the derivative with respect to X and setting it equal to zero yields the normal equation

$$-Y_k P_k^H + X P_k P_k^H = 0. \quad (\text{B.5})$$

Therefore,

$$X P_k P_k^H = Y_k P_k^H. \quad (\text{B.6})$$

Assuming that $P_k P_k^H$ is invertible, which holds if the pilots are orthogonal and $\tau_p \geq N_t$, the LS solution is

$$\boxed{\hat{H}_{\text{LS},k} = Y_k P_k^H (P_k P_k^H)^{-1}.} \quad (\text{B.7})$$

If the pilots are orthogonal and have unit power, then

$$P_k P_k^H = I_{N_t}, \quad (\text{B.8})$$

This simplifies the **LS** estimate to

$$\hat{H}_{\text{LS},k} = Y_k P_k^H. \quad (\text{B.9})$$

Substituting (B.1) into (B.9) gives

$$\begin{aligned} \hat{H}_{\text{LS},k} &= (H_k P_k + N_k) P_k^H \\ &= H_k (P_k P_k^H) + N_k P_k^H \\ &= H_k + \tilde{N}_k, \end{aligned} \quad (\text{B.10})$$

where the effective projected noise is

$$\tilde{N}_k := N_k P_k^H \in \mathbb{C}^{N_r \times N_t}. \quad (\text{B.11})$$

Stacking the per-subcarrier estimates yields the wideband relation used in Section 2.2.1, namely $\hat{H}_{\text{LS}} = H + \tilde{N}$.

In the effective per-user approximation adopted in this thesis, $N_t = 1$. The active slot configuration contains two pilot-carrying **OFDM** symbols, but one such symbol is selected per sample, so the effective pilot length becomes $\tau_p = 1$. Each pilot block therefore reduces to a scalar $p_k \in \mathbb{C}$. In that case,

$$\hat{h}_{\text{LS},k} = h_k + \tilde{n}_k.$$

Since $\tilde{n}_k$ is a linear transformation of complex Gaussian noise, it remains complex Gaussian.

With pilot contamination, the per-subcarrier received pilot block becomes

$$Y_{0,k} = H_{0,k} P_{0,k} + \sum_{i \in \mathcal{I}} H_{i,k} P_{i,k} + N_k.$$

Projecting with respect to the desired pilot gives

$$\hat{H}_{\text{LS},0,k} = H_{0,k} + \sum_{i \in \mathcal{I}} H_{i,k} P_{i,k} P_{0,k}^H (P_{0,k} P_{0,k}^H)^{-1} + \tilde{N}_k.$$

In this effective per-user case with $P_{0,k} P_{0,k}^H = 1$, define the scalar pilot cross-correlation factor

$$\rho_{i0,k} := P_{i,k} P_{0,k}^H.$$

Then the projected observation reduces to

$$\hat{h}_{\text{LS},0,k} = h_{0,k} + \sum_{i \in \mathcal{I}} \rho_{i0,k} h_{i,k} + \tilde{n}_k,$$

which is the per-subcarrier form of the projected pilot-contamination model used in the main text.

B.3 LMMSE Estimator

For completeness, this appendix records the standard vectorized **LMMSE** estimator referred to in Section 2.2.2.1. It is included as an unweighted textbook reference, not as the simulator-integrated channel-estimation baseline. In particular, this expression does not model the simulator-specific pilot-overlap weights or the ideal downstream covariance treatment used in the end-to-end detector. Let $y = \text{vec}(\hat{H}_{\text{LS}})$ denote the vectorized wideband **LS** observation and let $h_0 = \text{vec}(H_0)$ denote the vectorized desired channel. In this notation, the **LMMSE** estimator minimizes $\mathbb{E}[\|\hat{h}_0 - h_0\|^2]$ over linear estimators and yields

$$\hat{h}_{\text{LMMSE},0} = \mu_0 + C_{0,y} C_y^{-1} (y - \mu_y), \quad (\text{B.12})$$

where the total mean μ_y and total covariance C_y of the observation are

$$\mu_y = \mu_0 + \sum_{l \in \mathcal{I}, |\mathcal{I}|=L} \mu_l, \quad (\text{B.13})$$

$$C_y = C_0 + \sum_{l \in \mathcal{I}, |\mathcal{I}|=L} C_l + \frac{1}{\gamma_{\text{SNR}}} I. \quad (\text{B.14})$$

Here, $C_0 = \mathbb{E}[(h_0 - \mu_0)(h_0 - \mu_0)^H]$ is the covariance matrix of the desired channel, $C_{0,y} = \mathbb{E}[(h_0 - \mu_0)(y - \mu_y)^H]$ is the cross-covariance between the desired channel and the observation, γ_{SNR} denotes the **SNR**, and $\mathcal{I}$ is the set of interfering cells. In the thesis itself this estimator is included only as a classical statistical reference, not as a reported baseline.